

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования
«Санкт-Петербургский политехнический университет Петра Великого»
Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
02.03.01 Математика и компьютерные науки

Отчёт по лабораторным работам №1–5

по дисциплине

Теория графов

Разработка программы для генерации графов и
реализации базовых алгоритмов теории графов

Выполнил студент группы 5130201/40003 _____ Джабраилов А. В.

Проверил _____ Востров А. В.

«_____» _____ 2026 г.

Санкт-Петербург, 2026

Содержание

Введение	6
1 Постановка задачи	7
2 Лабораторная работа №1. Генерация графа и базовый анализ	9
2.1 Постановка задачи	9
2.2 Математическое описание	9
2.3 Особенности реализации	10
3 Лабораторная работа №2. Обход в ширину и кратчайшие пути	17
3.1 Постановка задачи	17
3.2 Математическое описание	17
3.3 Особенности реализации	18
4 Лабораторная работа №3. Сети потоков	22
4.1 Постановка задачи	22
4.2 Математическое описание	22
4.3 Особенности реализации	23
5 Лабораторная работа №4. Остовные деревья, код Прюфера и паросочетания	28
5.1 Постановка задачи	28
5.2 Математическое описание	28
5.3 Особенности реализации	29
6 Лабораторная работа №5. Эйлеровы обходы и цикловая структура графа	34
6.1 Постановка задачи	34
6.2 Математическое описание	34
6.3 Особенности реализации	35
7 Структура программной системы	39
7.1 Общая архитектура	39
7.2 Состав модулей	40
7.3 Пользовательский интерфейс	40

8 Особенности и ограничения текущей реализации	42
Заключение	43
Приложение А. Файл AlgorithmComparator.h	45
Приложение Б. Файл AlgorithmComparator.cpp	46
Приложение В. Файл BFS.h	47
Приложение Г. Файл BFS.cpp	48
Приложение Д. Файл Dijkstra.h	51
Приложение Е. Файл Dijkstra.cpp	52
Приложение Ж. Файл Eccentricity.h	57
Приложение З. Файл Eccentricity.cpp	59
Приложение И. Файл EdmondsMatching.h	65
Приложение К. Файл EdmondsMatching.cpp	66
Приложение Л. Файл EulerianCycle.h	77
Приложение М. Файл EulerianCycle.cpp	80
Приложение Н. Файл FlowNetwork.h	94
Приложение О. Файл FlowNetwork.cpp	97
Приложение П. Файл FlowNetworkBuilder.h	104
Приложение Р. Файл FlowNetworkBuilder.cpp	105
Приложение С. Файл FundamentalCuts.h	106
Приложение Т. Файл FundamentalCuts.cpp	108
Приложение У. Файл FundamentalCycles.h	112

Приложение Ф. Файл FundamentalCycles.cpp	114
Приложение Х. Файл Generator.h	122
Приложение Ц. Файл Generator.cpp	124
Приложение Ч. Файл Graph.h	133
Приложение Ш. Файл Graph.cpp	134
Приложение Щ. Файл Kirchhoff.h	138
Приложение Ъ. Файл Kirchhoff.cpp	140
Приложение Ы. Файл Kruskal.h	145
Приложение Ь. Файл Kruskal.cpp	146
Приложение Э. Файл MaxFlowSolver.h	151
Приложение Ю. Файл MaxFlowSolver.cpp	152
Приложение Я. Файл MaxMatching.h	156
Приложение АА. Файл MaxMatching.cpp	157
Приложение АБ. Файл MermaidExporter.h	160
Приложение АВ. Файл MermaidExporter.cpp	161
Приложение АГ. Файл MinCostFlow.h	164
Приложение АД. Файл MinCostFlow.cpp	166
Приложение АЕ. Файл PathCounter.h	172
Приложение АЖ. Файл PathCounter.cpp	173
Приложение АЗ. Файл PruferCode.h	175
Приложение АИ. Файл PruferCode.cpp	177

Приложение АК. Файл ShimbellMethod.h	184
Приложение АЛ. Файл ShimbellMethod.cpp	186
Приложение АМ. Файл main.cpp	193

Введение

Теория графов является разделом дискретной математики, изучающим структуры, состоящие из вершин и рёбер, и отношения между ними. Графовые модели широко применяются для описания самых разных объектов и процессов: транспортных и компьютерных сетей, маршрутов, потоков, зависимостей, социальных связей и структур данных. Практическая ценность теории графов связана с тем, что многие реальные задачи естественным образом сводятся к задачам на графах, таким как поиск путей, анализ связности, построение остовов, исследование циклов, потоков и паросочетаний. Именно поэтому методы теории графов занимают важное место как в математике, так и в информатике. Также теория графов является основой нейронных сетей и искусственного интеллекта.

В рамках цикла лабораторных работ была разработана единая консольная программа на языке C++. Программа позволяет последовательно выполнять генерацию графа, анализ его метрических характеристик, поиск путей, работу с потоками, построение остовов, кодирование деревьев, поиск паросочетаний, а также анализ эйлеровых циклов и базиса циклов.

Практическая ценность такого подхода состоит в том, что результаты предыдущих лабораторных работ не теряются, а переиспользуются в последующих. Например, граф, построенный в первой лабораторной работе, используется при обходе в ширину и поиске кратчайших путей во второй, преобразуется в сеть потоков в третьей, становится основой для построения минимального остова в четвёртой и служит исходными данными для эйлеризации и анализа циклической структуры в пятой. Таким образом, при выполнении каждой лабораторной работы на самом деле происходила модификация уже реализованного кода.

1. Постановка задачи

Целью работы являлась реализация набора алгоритмов теории графов и объединение их в единый программный комплекс с текстовым пользовательским интерфейсом. В ходе выполнения лабораторных работ требовалось решить следующие задачи:

Лабораторная работа №1

1. реализовать генерацию графа на основе биномиального распределения;
2. реализовать вычисление эксцентриситетов вершин;
3. реализовать поиск центра графа;
4. реализовать определение диаметральных вершин графа;
5. реализовать метод Шимбелла;
6. реализовать поиск всех простых путей между двумя вершинами.

Лабораторная работа №2

7. реализовать обход графа в ширину;
8. реализовать классический алгоритм Дейкстры;
9. реализовать сравнение алгоритмов по числу итераций.

Лабораторная работа №3

10. реализовать построение сети потоков по ориентированному графу;
11. реализовать поиск максимального потока;
12. реализовать поиск потока минимальной стоимости.

Лабораторная работа №4

13. реализовать вычисление числа остовных деревьев по теореме Кирхгофа;

14. реализовать построение минимального остова алгоритмом Краскала;
15. реализовать кодирование дерева кодом Прюфера;
16. реализовать декодирование дерева по коду Прюфера;
17. реализовать поиск паросочетания.

Лабораторная работа №5

18. реализовать проверку графа на эйлеровость;
19. реализовать эйлеризацию графа при необходимости;
20. реализовать построение эйлера обхода;
21. реализовать построение фундаментальной системы циклов;
22. реализовать операцию симметрической разности циклов.

Помимо собственно алгоритмов, требовалось реализовать пользовательское меню, проверки входных данных и повторное использование уже построенных структур в рамках одного сеанса работы программы.

2. Лабораторная работа №1. Генерация графа и базовый анализ

2.1. Постановка задачи

Первая лабораторная работа посвящена построению исходного графа и базовым алгоритмам его анализа. Необходимо сгенерировать граф с числом вершин, заданным пользователем, распределить степени и веса по биномиальному закону, затем вычислить эксцентриситеты вершин, центр и диаметр графа, реализовать метод Шимбелла для путей фиксированной длины и определить количество маршрутов между двумя вершинами.

2.2. Математическое описание

В лекциях граф определяется как совокупность двух множеств $G(V, E)$, где V — непустое множество вершин, а E — множество двухэлементных подмножеств множества V . Если элементами E являются упорядоченные пары, то граф является ориентированным, и тогда элементы множества E называются дугами. Вершины, инцидентные одному ребру, называются смежными, а множество вершин, смежных с вершиной v , образует её множество смежности.

Степенью вершины $d(v)$ называется число инцидентных ей рёбер. Для неориентированного графа справедлива лемма о рукопожатиях: сумма степеней всех вершин равна удвоенному числу рёбер, поэтому число вершин нечётной степени всегда чётно. Именно это свойство учитывается при генерации неориентированного графа.

В работе используется помеченный граф, то есть граф, у которого рёбрам сопоставлены веса. Случайные значения степеней и весов формируются на основе биномиального распределения с параметрами n и p .

Маршрутом в графе называется чередующаяся последовательность вершин и рёбер, в которой любые два соседних элемента инцидентны. Если все рёбра маршрута различны, то такой маршрут называется цепью, а если все его вершины различны, то он называется простой цепью. Замкнутая цепь называется циклом, а замкнутая простая цепь — простым циклом. Граф без циклов называется ациклическим.

Две вершины графа называются связанными, если существует соединяющая их простая цепь. Граф, в котором все вершины связаны между собой, называется связным. Число компонент связности графа G обозначается $k(G)$. Расстоянием $d(u, v)$ между вершинами u и v называется длина кратчайшей цепи между ними, измеряемая числом рёбер.

Эксцентриситетом $e(v)$ вершины v в связном графе называется максимальное расстояние от этой вершины до других вершин графа:

$$e(v) = \max_{u \in V} d(v, u).$$

Радиус графа определяется формулой

$$R(G) = \min_{v \in V} e(v),$$

Вершина называется центральной, если её эксцентриситет совпадает с радиусом графа, а множество всех таких вершин образует центр графа. Наиболее эксцентричные вершины являются концами диаметра.

Метод Шимбелла в лекциях используется для выявления маршрутов с заданным количеством рёбер. В лекционном изложении для определения количества маршрутов, состоящих из k рёбер, требуется возвести в k -ую степень матрицу смежности вершин; соответствующий элемент полученной матрицы даёт количество маршрутов длины k . В программе эта идея адаптирована к взвешенному графу: вместо простого подсчёта числа маршрутов дополнительно вычисляются минимальные и максимальные суммарные веса маршрутов фиксированной длины.

Поиск всех путей между двумя вершинами в терминах лекций соответствует поиску всех простых цепей между этими вершинами, то есть всех последовательностей смежных вершин с фиксированными началом и концом, в которых вершины не повторяются.

2.3. Особенности реализации

Ключевой модуль генерации реализован в классе `AcyclicGraphBuilder`. В нём метод `sampleBinomial` вычисляет случайную величину с биномиальным распределением без обращения к готовому

распределению стандартной библиотеки. Метод `generateAcyclicGraph` сначала генерирует целевые степени всех вершин, затем строит рёбра и после этого проверяет связность графа. Если граф оказывается несвязным, генерация повторяется. В коде предусмотрено до 50 попыток регенерации.

Для основных функций этого модуля вход и выход можно описать следующим образом.

Метод `sampleBinomial(n, p)`

Вход: параметры биномиального распределения n и p , а также внутреннее состояние генератора случайных чисел класса.

Выход: одно целое случайное значение, распределённое по биномиальному закону.

Описание: генерирует случайное число по биномиальному распределению.

Код представлен в листинге 1.

```
double u = dist(m_rng);
double cumulative = 0.0;
for (int k = 0; k <= n; ++k) {
    cumulative += std::exp(logCombination(n, k) + k * std::log(p)
        + (n - k) * std::log(1.0 - p));
    if (u <= cumulative) return k;
...

```

Листинг 1: Фрагмент функции `sampleBinomial(n, p)`

Метод `generateAcyclicGraph(vertices, directed, weightSign)`

Вход: количество вершин, признак ориентированности графа, режим генерации весов, а также внутренние параметры биномиального распределения, закреплённые в классе.

Выход: указатель на сгенерированный граф, содержащий вершины, рёбра и веса.

Описание: строит граф с заданным числом вершин и режимом генерации весов.

Код представлен в листинге 2.

```
auto graph = std::make_unique<AdjacencyGraph>(directed);
for (int i = 1; i <= vertexCount; ++i) {
    graph->addVertex(i);
}
for (const auto& [u, v] : selectedEdges) graph->addEdge(u, v,
    generateWeight(weightSign));

```

...

Листинг 2: Фрагмент функции `generateAcyclicGraph(vertices, directed, weightSign)`

Метод `computeDegreeStatistics(graph)`

Вход: уже построенный граф.

Выход: структура со средним значением степени, дисперсией, стандартным отклонением, минимальной и максимальной степенью.

Описание: вычисляет основные статистики по степеням вершин графа.

Код представлен в листинге 3.

```
std::vector<int> degrees;
for (int v : graph.vertexIds()) {
    degrees.push_back(static_cast<int>(graph.neighbors(v).size()));
}
const double mean = std::accumulate(degrees.begin(), degrees.end(), 0.0) / degrees.size();
...
```

Листинг 3: Фрагмент функции `computeDegreeStatistics(graph)`

В ориентированном режиме рёбра добавляются только между вершинами $u < v$, что исключает появление ориентированных циклов и даёт ориентированный ациклический граф. В неориентированном режиме используется та же схема выбора концов ребра, однако сама по себе она не является строгим доказательством отсутствия циклов; фактически она задаёт порядок генерации и исключает петли и дубликаты.

Вычисление эксцентриситетов реализовано в классе `GraphAnalyzer`. Для каждой стартовой вершины выполняется BFS, после чего формируется таблица расстояний по числу рёбер. Далее по этим расстояниям вычисляются радиус, диаметр, центральные вершины и все диаметральные пути.

Для этого класса важны две функции.

Метод `computeEdgeCountPaths()`

Вход: граф, сохранённый во внутреннем состоянии объекта `GraphAnalyzer`.

Выход: таблица расстояний между всеми упорядоченными парами вершин, где расстояние измеряется числом рёбер, а для недостижимых пар фиксируется специальное значение.

Описание: вычисляет расстояния между всеми парами вершин по числу рёбер.

Код представлен в листинге 4.

```
std::map<std::pair<int,int>, int> distances;
auto vertices = m_graph.vertexIds();
for (int start : vertices) {
    std::map<int, int> dist;
    std::queue<int> q;
    ...
}
```

Листинг 4: Фрагмент функции `computeEdgeCountPaths()`

Метод `analyze()`

Вход: граф, переданный объекту при создании класса.

Выход: структура `EccentricityData`, содержащая эксцентриситеты, радиус, диаметр, центр графа, диаметральные вершины и набор диаметральных путей.

Описание: находит основные метрические характеристики графа.

Код представлен в листинге 5.

```
EccentricityData result;
auto vertices = m_graph.vertexIds();
if (vertices.empty()) return result;
auto edgeCounts = computeEdgeCountPaths();
for (int v : vertices) {
    ...
}
```

Листинг 5: Фрагмент функции `analyze()`

Метод Шимбелла реализован в классе `KPathCalculator`. Сначала строится базовая матрица весов для путей длины 1, затем она последовательно перемножается сама на себя в полукольцах $(\min, +)$ и $(\max, +)$. При отсутствии пути используется значение `std::nullopt`, что позволяет явно отделять «пути нет» от нулевого веса.

С точки зрения входа и выхода основные функции здесь таковы.

Метод `buildWeightMatrix()`

Вход: граф, сохранённый внутри объекта `KPathCalculator`.

Выход: базовая матрица весов для путей длины 1.

Описание: строит матрицу весов для путей, состоящих из одного ребра.

Код представлен в листинге 6.

```

WeightTable matrix(m_vertexCount, std::vector<std::optional<int>
    >>(m_vertexCount));
for (int i = 0; i < m_vertexCount; ++i) {
    for (const auto& [to, weight] : m_graph.neighbors(i + 1)) {
        matrix[i][to - 1] = static_cast<int>(weight);
    }
}
...

```

Листинг 6: Фрагмент функции buildWeightMatrix()

Метод multiplyMinPlus(left, right)

Вход: две матрицы весов путей.

Выход: матрица минимальных суммарных весов при композиции путей.

Описание: перемножает две матрицы по правилу (min, +).

Код представлен в листинге 7.

```

WeightTable result(m_vertexCount, std::vector<std::optional<int>
    >>(m_vertexCount));
for (int i = 0; i < m_vertexCount; ++i) {
    for (int j = 0; j < m_vertexCount; ++j) {
        for (int k = 0; k < m_vertexCount; ++k) {
            if (left[i][k] && right[k][j]) result[i][j] =
                minValue(result[i][j], *left[i][k] + *right[k][j])
            ;
        }
    }
}
...

```

Листинг 7: Фрагмент функции multiplyMinPlus(left, right)

Метод multiplyMaxPlus(left, right)

Вход: две матрицы весов путей.

Выход: матрица максимальных суммарных весов при композиции путей.

Описание: перемножает две матрицы по правилу (max, +).

Код представлен в листинге 8.

```

WeightTable result(m_vertexCount, std::vector<std::optional<int>
    >>(m_vertexCount));
for (int i = 0; i < m_vertexCount; ++i) {
    for (int j = 0; j < m_vertexCount; ++j) {
        for (int k = 0; k < m_vertexCount; ++k) {
            if (left[i][k] && right[k][j]) result[i][j] =
                maxValue(result[i][j], *left[i][k] + *right[k][j])
            ;
        }
    }
}
...

```

...

Листинг 8: Фрагмент функции `multiplyMaxPlus(left, right)`

Метод `compute(edgeCount)`

Вход: требуемое число рёбер k и граф, связанный с объектом класса.

Выход: структура `ShimbellOutput`, содержащая матрицу минимальных весов, матрицу максимальных весов и само значение k .

Описание: вычисляет минимальные и максимальные веса путей длины k .

Код представлен в листинге 9.

```
WeightTable baseMatrix = buildWeightMatrix();
WeightTable minMatrix = baseMatrix;
WeightTable maxMatrix = baseMatrix;
for (int step = 2; step <= edgeCount; ++step) {
    minMatrix = multiplyMinPlus(minMatrix, baseMatrix);
    ...
}
```

Листинг 9: Фрагмент функции `compute(edgeCount)`

Поиск всех простых путей вынесен в класс `SimplePathFinder`. Он использует рекурсивный обход в глубину с массивом посещённых вершин и текущим накапливаемым путём. При достижении целевой вершины найденный путь сохраняется в коллекцию.

Для этого алгоритма удобно явно зафиксировать следующее.

Метод `findAllPaths(from, to)`

Вход: номера начальной и конечной вершин, а также граф, сохранённый в объекте `SimplePathFinder`.

Выход: коллекция всех найденных простых путей между указанными вершинами.

Описание: находит все простые пути между двумя вершинами.

Код представлен в листинге 10.

```
PathCollection collectedPaths;
std::unordered_set<int> visited;
std::vector<int> currentPath{from};
visited.insert(from);
dfsExplore(from, to, visited, currentPath, collectedPaths);
...
```

Листинг 10: Фрагмент функции `findAllPaths(from, to)`

Метод `countPaths(from, to)`

Вход: те же две вершины и граф, связанный с объектом класса.

Выход: количество простых путей между ними.

Описание: подсчитывает число простых путей между двумя вершинами.

Код представлен в листинге 11.

```
PathCollection paths = findAllPaths(from, to);
return static_cast<int>(paths.size());
...
```

Листинг 11: Фрагмент функции `countPaths(from, to)`

Метод `dfsExplore(current, target, visited, path, allPaths)`

Вход: текущая вершина обхода, целевая вершина, массив посещённых вершин, текущий путь и контейнер для всех найденных путей.

Выход: модифицированный контейнер `allPaths`, в который добавляются найденные пути; отдельного возвращаемого значения функция не имеет.

Описание: рекурсивно расширяет текущий путь и сохраняет найденные маршруты.

Код представлен в листинге 12.

```
if (current == target) {
    allPaths.push_back(path);
    return;
}
for (const auto& [neighborId, weight] : m_graph.neighbors(current)) {
    if (!visited.count(neighborId)) {
        ...
    }
}
```

Листинг 12: Фрагмент функции `dfsExplore(current, target, visited, path, allPaths)`

В пользовательском интерфейсе первой лабораторной работе соответствуют пункты 1–8. Пользователь может сгенерировать граф, вывести матрицу смежности, матрицу весов для случая $k = 1$, статистику по степеням и результаты всех перечисленных алгоритмов.

3. Лабораторная работа №2. Обход в ширину и кратчайшие пути

3.1. Постановка задачи

Во второй лабораторной работе на уже построенном графе необходимо выполнить обход в ширину, найти кратчайший путь между двумя вершинами классическим алгоритмом Дейкстры и сравнить оба алгоритма по числу итераций.

3.2. Математическое описание

В лекциях поиск в ширину рассматривается как алгоритм Мура. Он строит слоистую структуру графа относительно стартовой вершины s : на нулевом уровне находится сама вершина s , на первом — все вершины, смежные с s , на втором — вершины, достижимые цепью длины 2, и так далее. Поэтому алгоритм Мура находит кратчайшие цепи в графе, если длина цепи измеряется числом рёбер.

Алгоритм Дейкстры, согласно лекциям, находит оптимальные маршруты и их длину между одной конкретной вершиной-источником и всеми остальными вершинами графа. Он корректен для графов и орграфов без рёбер отрицательного веса. Недостаток алгоритма состоит в том, что при наличии отрицательных весов он может работать некорректно.

В классической табличной форме алгоритма Дейкстры используются следующие обозначения:

- $T[v]$ — текущая метка вершины v , то есть текущая оценка длины маршрута от источника до v ;
- $X[v]$ — признак того, что метка вершины стала постоянной;
- $H[v]$ — предшественник вершины v в дереве кратчайших путей.

На каждом шаге выбирается вершина с минимальной временной меткой, после чего выполняется корректировка меток для смежных с ней вершин. После

завершения работы алгоритма по массиву предшественников восстанавливается дерево кратчайших путей и, при необходимости, конкретный маршрут до выбранной вершины.

Таким образом, вторая лабораторная работа сопоставляет два разных критерия длины цепи:

- в алгоритме Мура длина измеряется количеством рёбер;
- в алгоритме Дейкстры длина определяется суммой весов рёбер маршрута.

Если требуется обрабатывать графы с произвольными весами, включая отрицательные, то в лекциях для этого предлагается алгоритм Беллмана–Форда, однако в данной лабораторной работе реализуется именно классический алгоритм Дейкстры.

3.3. Особенности реализации

Обход в ширину реализован в классе `BreadthFirstSearch`. Метод `traverseWithLevels` хранит:

- порядок обхода вершин;
- отображение уровня в список вершин этого уровня;
- число выполненных итераций;
- максимальную глубину обхода.

Таким образом, пользователь получает не только факт достижимости, но и слоистое представление графа.

Формально вход и выход функции можно описать так.

Метод `traverseWithLevels(startVertex)`

Вход: стартовая вершина и граф, заранее сохранённый во внутреннем состоянии класса `BreadthFirstSearch`.

Выход: структура `BFSResult`, содержащая порядок обхода, уровни, число итераций и максимальную глубину.

Описание: выполняет обход в ширину и определяет уровни вершин.

Код представлен в листинге 13.

```

std::queue<int> q;
q.push(startVertex);
result.levels[startVertex] = 0;
while (!q.empty()) {
    int current = q.front(); q.pop();
    ...
}

```

Листинг 13: Фрагмент функции `traverseWithLevels(startVertex)`

Алгоритм Дейкстры реализован в классе `DijkstraAlgorithm`. В коде используется классическая схема без очереди с приоритетом: на каждой итерации линейным просмотром выбирается следующая вершина с минимальной меткой. Это делает реализацию прозрачной и хорошо соответствующей учебному варианту алгоритма. Дополнительно сохраняются предшественники и формируется вектор расстояний от исходной вершины до всех достижимых вершин.

При описании функций этого класса удобно использовать следующий формат.

Метод `findShortestPaths(startVertex, targetVertex)`

Вход: стартовая вершина, необязательная целевая вершина и граф, связанный с объектом `DijkstraAlgorithm`.

Выход: структура `DijkstraResult`, содержащая расстояния до достижимых вершин, предшественников, восстановленный кратчайший путь до цели, итоговое расстояние и число итераций.

Описание: находит кратчайшие пути от стартовой вершины алгоритмом Дейкстры.

Код представлен в листинге 14.

```

result.distances[s] = 0.0;
for (size_t iter = 0; iter < vertexIds.size(); ++iter) {
    int v = extractClosestUnused(result.distances, used);
    if (v == -1) break;
    for (const auto& [to, weight] : m_graph.neighbors(v)) {
        ...
    }
}

```

Листинг 14: Фрагмент функции `findShortestPaths(startVertex, targetVertex)`

Метод `reconstructPath(startVertex, targetVertex, H,`

vertexIds)

Вход: начальная вершина, конечная вершина, массив предшественников и список идентификаторов вершин.

Выход: последовательность вершин, задающая найденный кратчайший путь; если путь не существует, возвращается пустой вектор.

Описание: восстанавливает путь по массиву предшественников.

Код представлен в листинге 15.

```
std::vector<int> path;
for (int current = targetVertex; current != startVertex; current
    = H.at(current)) {
    path.push_back(current);
}
path.push_back(startVertex);
std::reverse(path.begin(), path.end());
...
```

Листинг 15: Фрагмент функции `reconstructPath(startVertex, targetVertex, H, vertexIds)`

Сравнение алгоритмов вынесено в класс `AlgorithmComparator`. Он запускает BFS и алгоритм Дейкстры из одной и той же вершины и сравнивает их по количеству учтённых операций. В программе выводится отношение числа итераций алгоритма Дейкстры к числу итераций BFS.

Соответствующая функция описывается так.

Метод `compare(startVertex)`

Вход: стартовая вершина и граф, сохранённый в объекте `AlgorithmComparator`.

Выход: структура `AlgorithmComparison`, содержащая число итераций BFS, число итераций алгоритма Дейкстры и коэффициент сравнения.

Описание: сравнивает BFS и алгоритм Дейкстры по числу итераций.

Код представлен в листинге 16.

```
BFSResult bfsResult = bfs.traverseWithLevels(startVertex);
DijkstraResult dijkstraResult = dijkstra.findShortestPaths(
    startVertex);
result.bfsIterations = bfsResult.iterations;
result.dijkstraIterations = dijkstraResult.iterations;
result.ratio = static_cast<double>(result.dijkstraIterations) /
    std::max(1, result.bfsIterations);
```

Листинг 16: Фрагмент функции `compare(startVertex)`

Практически важно отметить, что корректность алгоритма Дейкстры обеспечивается только при неотрицательных весах рёбер. Поскольку генератор программы умеет создавать и отрицательные веса, при запуске этой части работы желательно использовать положительный режим генерации весов. Сам код не запрещает запуск алгоритма на смешанных весах, поэтому это ограничение следует учитывать при интерпретации результатов.

Пользовательские пункты второй лабораторной работы — 9, 10 и 11. Они позволяют вывести уровни BFS, кратчайший путь между двумя вершинами и сравнение скорости работы алгоритмов.

4. Лабораторная работа №3. Сети потоков

4.1. Постановка задачи

Третья лабораторная работа основана на ориентированном графе. Требуется случайным образом сформировать матрицы пропускных способностей и стоимостей, построить по ним сеть потоков, найти максимальный поток между выбранными вершинами и затем вычислить поток минимальной стоимости величины, равной двум третям от найденного максимального потока с округлением вниз.

4.2. Математическое описание

С точки зрения лекций сеть потоков задаётся ориентированным графом, в котором по каждой дуге протекает некоторый поток. Каждая дуга характеризуется тремя параметрами:

- нижней границей потока;
- пропускной способностью, то есть верхней границей потока;
- стоимостью передачи единицы потока по данной дуге.

В реализованной модели нижняя граница явно не задаётся, поэтому по умолчанию считается равной нулю.

Если через $f(u, v)$ обозначить поток по дуге (u, v) , через $c(u, v)$ — её пропускную способность, а через $p(u, v)$ — стоимость единицы потока, то поток по каждой дуге должен удовлетворять ограничениям, задаваемым нижней и верхней границами, а в промежуточных вершинах должен выполняться закон сохранения потока.

Максимальный поток ищется на основе теоремы и алгоритма Форда–Фалкерсона. Рассматривается остаточная сеть: если по некоторой дуге ещё можно увеличить поток, то в остаточной сети присутствует прямая дуга; если часть потока можно вернуть назад, то появляется обратная дуга. Наличие увеличивающего пути из истока в сток означает, что значение потока можно увеличить на величину минимальной остаточной пропускной способности вдоль этого пути.

Поток минимальной стоимости в лекциях рассматривается как задача потокового программирования. Требуется выбрать значения потоков по дугам так, чтобы удовлетворялись ограничения по пропускным способностям и сохранению потока, а суммарная стоимость передачи потока была минимальной. В программе для этого используется последовательный поиск кратчайших по стоимости увеличивающих путей в остаточной сети. Сначала определяется величина максимального потока, после чего строится поток минимальной стоимости величины, равной двум третям от найденного максимального потока с округлением вниз.

4.3. Особенности реализации

Построение сети потоков вынесено в класс `FlowNetworkBuilder`. Он не меняет структуру исходного ориентированного графа, а только копирует его топологию. Пропускные способности и стоимости затем генерируются заново с использованием того же генератора биномиального распределения, что и в первой лабораторной работе.

Основная функция этого модуля имеет следующий интерфейс в логическом смысле.

Метод `buildFromGraph(graph)`

Вход: ориентированный граф, задающий топологию сети.

Выход: новая сеть потоков, в которой вершины и дуги унаследованы от исходного графа, а пропускные способности и стоимости сгенерированы автоматически.

Описание: строит сеть потоков по ориентированному графу.

Код представлен в листинге 17.

```
auto network = std::make_unique<FlowNetwork>(true);
for (int vertexId : graph.vertexIds()) {
    network->addVertex(vertexId);
}
for (const WeightedEdge& edge : graph.edges()) {
    network->addEdge(edge.from, edge.to, generateRandomCapacity(),
        generateRandomCost());
}
...
```

Листинг 17: Фрагмент функции `buildFromGraph(graph)`

Класс `FlowNetwork` хранит реальные дуги и умеет на лету строить остаточные дуги. Для этого вводится структура `ResidualArc`, которая содержит направление, остаточную пропускную способность, стоимость и ссылку на исходную дугу. Благодаря этому один и тот же сетевой объект используется и для максимального потока, и для потока минимальной стоимости.

Для используемых в отчёте функций этого класса.

Метод `residualNeighbors(vertex)`

Вход: вершина сети потоков и текущее состояние всех потоков в объекте `FlowNetwork`.

Выход: список остаточных дуг, по которым поток можно увеличить или уменьшить.

Описание: возвращает доступные переходы из вершины в остаточной сети. Код представлен в листинге 18.

```
std::vector<ResidualArc> residuals;
auto outIt = m_outgoing.find(vertex);
if (outIt != m_outgoing.end()) {
    for (int edgeIndex : outIt->second) {
        const FlowEdge& edge = m_edges[edgeIndex];
    }
    ...
}
```

Листинг 18: Фрагмент функции `residualNeighbors(vertex)`

Метод `getCapacityMatrix()`, `getCostMatrix()`, `getFlowMatrix()`

Вход: текущее состояние сети потоков.

Выход: соответствующие матричные представления пропускных способностей, стоимостей и текущего потока.

Описание: возвращает матрицы пропускных способностей, стоимостей и текущего потока.

Код представлен в листинге 19.

```
FlowMatrix matrix = createEmptyMatrix(size());
for (const FlowEdge& edge : m_edges) {
    matrix[indexForVertex(edge.from)][indexForVertex(edge.to)] =
        edge.capacity;
}
return matrix;
```

Листинг 19: Фрагмент функций получения матриц сети потока

Метод `augment(edgeIndex, forward, delta)`

Вход: индекс исходной дуги, признак прямого или обратного прохода и величина изменения потока.

Выход: обновлённое внутреннее состояние сети; отдельного возвращаемого значения функция не имеет.

Описание: изменяет поток по выбранной дуге на заданную величину.

Код представлен в листинге 20.

```
if (edgeIndex < 0 || edgeIndex >= static_cast<int>(m_edges.size())
    ) {
    return;
}
if (forward) {
    m_edges[edgeIndex].flow += delta;
...
}
```

Листинг 20: Фрагмент функции `augment(edgeIndex, forward, delta)`

Максимальный поток реализован в классе `MaxFlowSolver`. По сути, это вариант алгоритма Форда–Фалкерсона с поиском увеличивающего пути в ширину, то есть схема, близкая к алгоритму Эдмондса–Карпа. После каждого успешного шага сохраняются:

- номер шага;
- найденный путь по вершинам;
- величина добавленного потока;
- суммарный поток после увеличения.

Это делает вычисление удобным для трассировки и проверки.

В терминах функций описание выглядит так.

Метод `compute(source, sink)`

Вход: вершины истока и стока, а также сеть потоков, связанная с объектом `MaxFlowSolver`.

Выход: структура `MaxFlowResult`, содержащая значение максимального потока и последовательность увеличивающих шагов.

Описание: находит максимальный поток между истоком и стоком.

Код представлен в листинге 21.

```

MaxFlowResult result{0, {}};
if (source == sink || !m_network.hasVertex(source) || !m_network.
    hasVertex(sink)) {
    return result;
}
m_network.resetFlows();
while (bfs(source, sink, parent)) {
    ...
}

```

Листинг 21: Фрагмент функции `compute(source, sink)` для максимального потока

Метод `bfs(source, sink, parent)`

Вход: исток, сток, остаточная сеть, задаваемая текущим состоянием объекта `FlowNetwork`, и контейнер для предков.

Выход: логическое значение, показывающее, найден ли увеличивающий путь; дополнительно по ссылке заполняется структура предков, описывающая этот путь.

Описание: ищет увеличивающий путь в остаточной сети.

Код представлен в листинге 22.

```

std::queue<int> queue;
std::unordered_set<int> visited;
parent.clear();
queue.push(source);
visited.insert(source);
...

```

Листинг 22: Фрагмент функции `bfs(source, sink, parent)`

Поток минимальной стоимости реализован в классе `MinCostFlowSolver`. В нём на остаточной сети выполняется поиск кратчайшего по стоимости пути, после чего по нему проводится очередное увеличение потока. В программе целевая величина потока вычисляется автоматически как две трети от найденного ранее максимального потока с округлением вниз.

Здесь ключевые функции имеют такой вход и выход.

Метод `compute(source, sink, targetFlow)`

Вход: исток, сток, требуемая величина потока и сеть потоков, закреплённая за объектом `MinCostFlowSolver`.

Выход: структура `MinCostFlowResult`, содержащая целевой и достигнутый поток, полную стоимость и пошаговую историю увеличений.

Описание: находит поток минимальной стоимости заданной величины.

Код представлен в листинге 23.

```
MinCostFlowResult result{targetFlow, 0, 0, false, {}};
if (targetFlow <= 0) {
    result.success = true;
    return result;
}
m_network.resetFlows();
...
```

Листинг 23: Фрагмент функции `compute(source, sink, targetFlow)`

Метод `dijkstra(source, sink, distances, parent)`

Вход: исток, сток, текущее состояние остаточной сети и контейнеры для расстояний и предков.

Выход: логическое значение, показывающее существование допустимого пути минимальной стоимости; дополнительно заполняются отображения расстояний и дуг-предков.

Описание: ищет кратчайший по стоимости путь в остаточной сети.

Код представлен в листинге 24.

```
constexpr int INF = std::numeric_limits<int>::max();
distances.clear();
parent.clear();
if (!m_network.hasVertex(source)) {
    return false;
}
...
```

Листинг 24: Фрагмент функции `dijkstra(source, sink, distances, parent)`

В пользовательском меню лабораторной работе №3 соответствуют пункты 12–17. Пользователь сначала строит сеть потоков, затем может вывести матрицы пропускных способностей и стоимостей, найти максимальный поток, а после этого — поток минимальной стоимости для той же пары «исток–сток».

5. Лабораторная работа №4. Остовные деревья, код Прюфера и паросочетания

5.1. Постановка задачи

Четвёртая лабораторная работа выполняется на неориентированном графе. Требуется вычислить число остовных деревьев по матричной теореме Кирхгофа, построить минимальный остов алгоритмом Краскала, закодировать и декодировать полученное дерево кодом Прюфера, а также определить независимое множество рёбер.

5.2. Математическое описание

Пусть $G(V, E)$ — неориентированный граф. Остовным подграфом графа G называется подграф, содержащий все вершины графа. Остовный подграф, являющийся деревом, называется остовом, или каркасом. Несвязный граф остова не имеет, а связный граф может иметь несколько различных остовов.

Если A — матрица смежности графа, а D — диагональная матрица степеней его вершин, то матрица Кирхгофа определяется формулой

$$B(G) = D - A.$$

По теореме Кирхгофа число остовных деревьев в связном графе порядка $n \geq 2$ равно алгебраическому дополнению любого элемента матрицы $B(G)$. Иными словами, достаточно вычислить любой её кофактор, то есть определитель минора порядка $n - 1$.

Если рёбрам графа приписаны веса, то возникает задача построения кратчайшего остова, то есть остова минимального суммарного веса. В лабораторной работе для этого используется алгоритм Краскала: рёбра перебираются в порядке возрастания их веса, и каждое безопасное ребро добавляется в остов. Безопасным является ребро, добавление которого не образует цикла с уже выбранными рёбрами. В лекциях для этого алгоритма приводится оценка трудоёмкости $O(E \log E)$.

Код Прюфера в лекциях рассматривается как экономное по памяти

представление свободного дерева. Для дерева $T(V, E)$ с вершинами, занумерованными числами от 1 до p , строится последовательность, получаемая последовательным удалением листьев и записью их соседей. В теоретической постановке информация о последнем ребре восстанавливается однозначно. В реализованной версии, помимо самой последовательности, дополнительно сохраняются веса соответствующих рёбер, чтобы можно было декодировать не только структуру дерева, но и его взвешенный вариант.

Паросочетанием называется множество рёбер, не имеющих общих вершин. Если к паросочетанию нельзя добавить ни одного нового ребра, то оно является максимальным по включению. Если же среди всех паросочетаний выбрано паросочетание наибольшей мощности, то оно называется паросочетанием максимальной мощности. В данной работе рассматриваются оба варианта: жадное максимальное по включению паросочетание и точный поиск паросочетания максимальной мощности алгоритмом Эдмондса.

5.3. Особенности реализации

Класс `KirchhoffCounter` строит матрицу Кирхгофа $B(G) = D - A$, а затем вычисляет детерминант минора, полученного удалением первой строки и первого столбца. Для вычисления детерминанта используется метод Гаусса с выбором главного элемента. После вычисления результат округляется, поскольку теоретически он обязан быть целым неотрицательным числом.

С точки зрения входа и выхода здесь используются следующие функции.

Метод `buildKirchhoffMatrix()`

Вход: граф, сохранённый в объекте `KirchhoffCounter`.

Выход: матрица Кирхгофа $B(G) = D - A$.

Описание: строит матрицу Кирхгофа для заданного графа.

Код представлен в листинге 25.

```
const int n = m_graph.size();
std::vector<std::vector<int>> B(n, std::vector<int>(n, 0));
for (const auto& edge : m_graph.edges()) {
    const int i = idToIndex.at(edge.from);
    const int j = idToIndex.at(edge.to);
    ...
}
```

Листинг 25: Фрагмент функции `buildKirchhoffMatrix()`

Метод `determinantOfFirstMinor(matrix)`

Вход: матрица Кирхгофа.

Выход: целое число, равное детерминанту выбранного минора.

Описание: вычисляет определитель минора матрицы Кирхгофа.

Код представлен в листинге 26.

```
const int n = static_cast<int>(matrix.size());
const int m = n - 1;
std::vector<std::vector<double>> M(m, std::vector<double>(m, 0.0)
);
for (int i = 0; i < m; ++i) {
    for (int j = 0; j < m; ++j) {
        ...
    }
}
```

Листинг 26: Фрагмент функции `determinantOfFirstMinor(matrix)`

Метод `compute()`

Вход: граф, связанный с объектом класса.

Выход: структура `KirchhoffResult`, содержащая матрицу Кирхгофа и число остовных деревьев.

Описание: вычисляет число остовных деревьев графа.

Код представлен в листинге 27.

```
KirchhoffResult result;
result.kirchhoffMatrix = buildKirchhoffMatrix();
const int n = m_graph.size();
if (n == 0) {
    result.spanningTreeCount = 0;
    ...
}
```

Листинг 27: Фрагмент функции `compute()` класса `KirchhoffCounter`

Построение минимального остова реализовано в классе `KruskalMST`. Все рёбра сортируются по весу, после чего проходит стандартный алгоритм Краскала на основе структуры `DisjointSetUnion`. Если граф оказывается несвязным, программа возвращает остовный лес и явно сообщает об этом пользователю.

В этом модуле описание выглядит так.

Метод `buildMST()`

Вход: граф, сохранённый в объекте `KruskalMST`.

Выход: структура `KruskalResult`, содержащая построенный остов, список выбранных рёбер, суммарный вес и признак связности исходного графа.

Описание: строит минимальный остов алгоритмом Краскала.

Код представлен в листинге 28.

```
KruskalResult result;
result.totalWeight = 0.0;
result.wasConnected = true;
result.spanningTree = std::make_unique<AdjacencyGraph>(m_graph.
    isDirected());
for (int v : m_graph.vertexIds()) {
    ...
}
```

Листинг 28: Фрагмент функции `buildMST()`

Класс `PruferEncoder` реализует согласованный внутри проекта вариант кода Прюфера. При кодировании на каждом шаге удаляется лист с минимальным номером, в код записывается его единственный сосед, а в параллельный массив — вес соответствующего ребра. При декодировании по оставшейся части кода выбирается минимальная неиспользованная вершина, не встречающаяся в хвосте последовательности, после чего восстанавливается очередное ребро.

Функции этого класса описываются так.

Метод `encode(tree)`

Вход: дерево, представленное объектом `AdjacencyGraph`.

Выход: структура `PruferCode`, содержащая последовательность кода и параллельный массив весов.

Описание: кодирует дерево в код Прюфера.

Код представлен в листинге 29.

```
PruferCode result;
const int p = tree.size();
if (p < 2) {
    return result;
}
std::vector<int> aliveVertices = tree.vertexIds();
...
}
```

Листинг 29: Фрагмент функции encode(tree)

Метод decode(pruferCode, vertexIds)

Вход: код Прюфера с весами и полный список идентификаторов вершин исходного дерева.

Выход: заново построенное дерево как объект AdjacencyGraph.

Описание: восстанавливает дерево по коду Прюфера.

Код представлен в листинге 30.

```
auto tree = std::make_unique<AdjacencyGraph>(false);
for (int v : vertexIds) {
    tree->addVertex(v);
}
const int p = static_cast<int>(vertexIds.size());
if (p < 2) {
    ...
}
```

Листинг 30: Фрагмент функции decode(pruferCode, vertexIds)

Для поиска независимого множества рёбер реализованы два подхода. Класс GreedyMaximalMatching последовательно просматривает рёбра в лексикографическом порядке и добавляет ребро, если обе его вершины пока не покрыты. Класс EdmondsMatching реализует blossom-алгоритм Эдмондса и позволяет находить уже не просто максимальное по включению, а максимальное по мощности паросочетание в общем неориентированном графе.

Логический вход и выход этих функций таков.

Метод GreedyMaximalMatching::compute()

Вход: граф, связанный с объектом класса.

Выход: структура MatchingResult, содержащая выбранные рёбра, непокрытые вершины, размер паросочетания и признак его совершенства.

Описание: строит жадное максимальное по включению паросочетание.

Код представлен в листинге 31.

```
MatchingResult result;
result.matchingSize = 0;
result.isPerfect = false;
std::vector<WeightedEdge> allEdges = m_graph.edges();
std::sort(allEdges.begin(), allEdges.end(),
```


...

Листинг 31: Фрагмент функции `GreedyMaximalMatching::compute()`

Метод `EdmondsMatching::compute()`

Вход: граф, сохранённый во внутреннем состоянии объекта `EdmondsMatching`.

Выход: структура `MatchingResult`, но уже для паросочетания максимальной мощности.

Описание: находит паросочетание максимальной мощности.

Код представлен в листинге 32.

```
MatchingResult result;
result.matchingSize = 0;
result.isPerfect = false;
EdmondsWorker worker(m_graph);
const std::vector<int> match = worker.run();
...
```

Листинг 32: Фрагмент функции `EdmondsMatching::compute()`

Пункты 18–23 пользовательского меню позволяют вывести матрицу Кирхгофа, построить и сохранить минимальный остов, показать его рёбра, выполнить кодирование и декодирование деревьев, а также сравнить жадное и точное паросочетания на исходном графе или на построенном остове.

6. Лабораторная работа №5. Эйлеровы обходы и цикловая структура графа

6.1. Постановка задачи

Пятая лабораторная работа посвящена эйлеровым графам и пространству циклов. Требуется определить, является ли неориентированный граф эйлеровым, при необходимости модифицировать его, построить эйлеров обход, а затем исследовать циклическую структуру графа относительно построенного остова.

6.2. Математическое описание

Если граф имеет цикл, содержащий все рёбра графа, то такой цикл называется эйлеровым циклом, а сам граф — эйлеровым. Если граф имеет цепь, содержащую все рёбра графа, то такая цепь называется эйлеровой цепью, а граф — полуэйлеровым. В лекциях отдельно подчёркивается, что эйлеровым может быть только связный граф.

Для связного неориентированного графа условие эйлеровости формулируется стандартно: граф является эйлеровым тогда и только тогда, когда степени всех его вершин чётны. Если же ровно две вершины имеют нечётную степень, то граф является полуэйлеровым и допускает эйлерову цепь, но не цикл.

Пусть $T(V, E_T)$ — некоторый остов связного графа $G(V, E)$. Остовный подграф, образованный рёбрами $E \setminus E_T$, называется кодеревом, а его рёбра — хордами остова. Каждая хорда остова порождает ровно один простой цикл Z_e . Система всех таких циклов называется фундаментальной системой циклов.

Число циклов в фундаментальной системе называется циклическим рангом, или цикломатическим числом, графа G и обозначается $m(G)$. Для связного графа оно равно

$$m(G) = |E| - |V| + 1.$$

Фундаментальная система циклов является базисом подпространства циклов.

Двойственным понятием является разрез. Разрезом связного графа называется множество рёбер, удаление которых делает граф несвязным. Правильный разрез, определяемый непустым подмножеством вершин $U \subset V$, обозначается $S(U)$. Простым разрезом называется минимальный разрез. В кодовой базе присутствует и модуль фундаментальной системы разрезов, однако в пользовательской части пятой лабораторной работы основной акцент сделан именно на фундаментальной системе циклов.

Сложению в пространстве циклов соответствует операция симметрической разности множеств рёбер. Согласно лекциям совокупность всех симметрических разностей фундаментальных циклов содержит не только все циклы графа, но и некоторые объединения циклов. Поэтому результат симметрической разности двух циклов может быть как одним циклом, так и объединением нескольких циклов, то есть циклическим вектором.

6.3. Особенности реализации

Проверка эйлеровости и построение обхода реализованы в классе `EulerianCycleBuilder`. Сначала входной граф копируется, чтобы не менять исходные данные. Затем выполняются два этапа модификации:

1. соединение компонент связности, содержащих рёбра;
2. попарное исправление нечётных степеней путём добавления новых рёбер.

Каждое добавленное ребро заносится в журнал модификаций. Если исправить нечётность без кратных рёбер невозможно, программа сообщает об этом и предлагает пользователю разрешить переход к мультиграфу. После завершения эйлеризации запускается алгоритм Хиерхольцера, и пользователь получает последовательность вершин эйлерова обхода.

Метод `compute(mode)`

Вход: исходный неориентированный граф, закреплённый за объектом `EulerianCycleBuilder`, и режим эйлеризации, разрешающий или запрещающий мультиграф.

Выход: структура `EulerianCycleResult`, содержащая сведения о необходимости модификаций, журнал добавленных рёбер, модифицированный

граф и итоговый эйлеров обход.

Описание: проверяет граф, при необходимости эйлеризует его и строит эйлеров обход.

Код представлен в листинге 33.

```
EulerianCycleResult result;  
result.success = false;  
result.modifiedGraph = cloneGraph(m_graph);  
AdjacencyGraph& G = *result.modifiedGraph;  
if (G.isDirected() || G.edges().empty()) {  
...  
}
```

Листинг 33: Фрагмент функции `compute(mode)`

Метод `hierholzer(g, start)`

Вход: граф, уже удовлетворяющий условиям существования эйлерова обхода, и стартовая вершина.

Выход: последовательность вершин эйлерова цикла или пути.

Описание: строит эйлеров цикл или путь алгоритмом Иерихольцера.

Код представлен в листинге 34.

```
std::unordered_map<int, std::vector<AdjSlot>> adj;  
std::unordered_set<int> usedEdges;  
std::stack<int> path;  
std::vector<int> trail;  
path.push(start);  
...  
}
```

Листинг 34: Фрагмент функции `hierholzer(g, start)`

Для исследования цикловой структуры используется класс `FundamentalCycleSystem`. На вход он получает исходный граф и ранее построенный минимальный остов. Для каждой хорды строится единственный путь в остове между её концами, после чего этот путь вместе с хордой образует фундаментальный цикл. Далее реализованы:

- вычисление симметрической разности наборов рёбер;
- попытка разложения полученного чётного подграфа на простые циклы.

Метод `compute()`

Вход: исходный граф и остов, сохранённые в объекте

FundamentalCycleSystem.

Выход: список всех фундаментальных циклов.

Описание: вычисляет систему фундаментальных циклов для выбранного остова.

Код представлен в листинге 35.

```
std::vector<FundamentalCycle> cycles;
const auto treeEdgeSet = collectTreeEdges(m_tree);
for (const WeightedEdge& e : m_graph.edges()) {
    const auto key = normEdge(e.from, e.to);
    if (treeEdgeSet.count(key) > 0) {
        ...
    }
}
```

Листинг 35: Фрагмент функции compute()

Метод symmetricDifference(a, b)

Вход: два отсортированных множества рёбер, представляющих циклы.

Выход: отсортированное множество рёбер, принадлежащее ровно одному из двух входов.

Описание: находит симметрическую разность двух циклов.

Код представлен в листинге 36.

```
std::vector<std::pair<int, int>> result;
size_t i = 0;
size_t j = 0;
while (i < a.size() && j < b.size()) {
    if (a[i] == b[j]) {
        ...
    }
}
```

Листинг 36: Фрагмент функции symmetricDifference(a, b)

Метод decomposeIntoCycles(edges)

Вход: множество рёбер чётного подграфа.

Выход: набор восстановленных простых циклов, если такое разложение удалось; в противном случае возвращается пустой результат.

Описание: раскладывает чётный подграф на простые циклы.

Код представлен в листинге 37.

```
std::map<int, std::set<int>> adjacency;
for (const auto& [u, v] : edges) {
    adjacency[u].insert(v);
    adjacency[v].insert(u);
}
```

```
}  
while (true) {  
...
```

Листинг 37: Фрагмент функции `decomposeIntoCycles(edges)`

Следует отметить, что в `lab5` присутствует и отдельный модуль `FundamentalCutSystem`, реализующий фундаментальные разрезы относительно остова. Однако в текущей версии меню задействована именно фундаментальная система циклов, поскольку она напрямую связана с операцией симметрической разности и наглядно демонстрирует структуру пространства циклов.

Пятая лабораторная работа соответствует пунктам меню 24–26. Пользователь может построить эйлеров обход, вывести фундаментальную систему циклов и получить результат симметрической разности выбранных циклов.

7. Структура программной системы

7.1. Общая архитектура

Итоговая программа имеет модульную архитектуру. В качестве базового типа используется класс `AdjacencyGraph`, который хранит граф в виде списка смежности и предоставляет операции добавления вершин и рёбер, получения списка вершин, соседей и матрицы смежности. Для задач третьей лабораторной работы введён отдельный класс `FlowNetwork`, в котором, помимо самих дуг, хранятся пропускные способности, стоимости и текущий поток.

Основные рабочие объекты в `main.cpp` таковы:

- `currentGraph` — текущий граф, сгенерированный пользователем;
- `currentFlowNetwork` — сеть потоков, построенная по текущему ориентированному графу;
- `currentSpanningTree` — минимальный остов или остовный лес;
- `lastPruferCode` — последний полученный код Прюфера;
- `lastFundamentalCycles` — кэш фундаментальной системы циклов.

Такой подход позволяет пользователю сначала сгенерировать граф, а затем поэтапно применять к нему алгоритмы разных лабораторных работ без пересоздания данных.

7.2. Состав модулей

Таблица 1: Основные модули программы

Модуль	Назначение
Graph, Generator	Базовое представление графа, генерация рёбер и весов по биномиальному распределению, вычисление статистики степеней.
Eccentricity, ShimbellMethod, PathCounter	Метрика графа, поиск диаметральных путей, матричный метод Шимбелла, поиск всех простых путей.
BFS, Dijkstra, AlgorithmComparator	Обход графа в ширину, кратчайшие пути, сравнение алгоритмов по числу итераций.
FlowNetwork, FlowNetworkBuilder, MaxFlowSolver, MinCostFlow	Построение сети потоков, остаточная сеть, максимальный поток, поток минимальной стоимости.
Kirchhoff, Kruskal, PruferCode, MaxMatching, EdmondsMatching	Остовные деревья, кодирование деревьев, жадное и максимальное паросочетания.
EulerianCycle, FundamentalCycles, FundamentalCuts	Эйлеризация графа, эйлеров обход, базис циклов; в кодовой базе также присутствует модуль фундаментальных разрезов.
MermaidExporter	Экспорт графа, сети потоков и остова в формат Mermaid для внешней визуализации.

7.3. Пользовательский интерфейс

В файле `lab5/src/main.cpp` реализовано единое текстовое меню. Его пункты сгруппированы по лабораторным работам. Для первой лабораторной доступны генерация графа, вычисление эксцентриситетов, метод Шимбелла, поиск всех путей и вывод матриц. Для второй — BFS, алгоритм Дейкстры и сравнение алгоритмов. Для третьей — построение сети потоков, матрицы пропускных способностей и стоимостей, а также алгоритмы потоков. Для четвёртой — теорема Кирхгофа, минимальный остов, код Прюфера и паросочетания. Для пятой — эйлеров обход и работа с фундаментальной системой циклов.

Отдельно предусмотрены сервисные возможности: скрывание разделов меню и экспорт текущих структур в Mermaid. Это делает программу не только учебной, но и пригодной для наглядной демонстрации результатов.

8. Особенности и ограничения текущей реализации

Разработанный комплекс успешно охватывает все пять лабораторных работ, однако при анализе реализации важно учитывать несколько особенностей.

1. Генератор графов ориентирован на повторное использование в разных лабораторных работах и надёжно обеспечивает связность, отсутствие петель и повторяющихся рёбер. При этом строгая ацикличность гарантируется непосредственно только в ориентированном режиме за счёт добавления дуг по правилу $u < v$.
2. Алгоритм Дейкстры в теории рассчитан на неотрицательные веса. В программе пользователь технически может выбрать и отрицательные веса, поэтому при анализе второй лабораторной работы необходимо использовать корректный режим входных данных.
3. В задаче потока минимальной стоимости используется последовательный поиск кратчайших путей в остаточной сети. Реализация удобна для учебной демонстрации и трассировки шагов, но не претендует на промышленную оптимальность.
4. В кодовой базе пятой лабораторной работы присутствуют и фундаментальные разрезы, и фундаментальные циклы, однако в меню вынесена именно работа с фундаментальной системой циклов.
5. Визуализация не встроена непосредственно в консольное приложение, но предусмотрен экспорт в Mermaid, что упрощает внешнее построение схем графа, сети потоков и остова.

Заключение

В ходе выполнения лабораторных работ был создан единый программный комплекс по теории графов, объединяющий задачи генерации графов, их метрического анализа, обходов, кратчайших путей, сетей потоков, остовных деревьев, кодирования деревьев, паросочетаний и эйлеровых обходов.

Лабораторная работа №1

1. реализована генерация графа на основе биномиального распределения;
2. реализовано вычисление эксцентриситетов вершин;
3. реализован поиск центра графа;
4. реализовано определение диаметральных вершин графа;
5. реализован метод Шимбелла;
6. реализован поиск всех простых путей между двумя вершинами.

Лабораторная работа №2

7. реализован обход графа в ширину;
8. реализован классический алгоритм Дейкстры;
9. реализовано сравнение алгоритмов по числу итераций.

Лабораторная работа №3

10. реализовано построение сети потоков по ориентированному графу;
11. реализован поиск максимального потока;
12. реализован поиск потока минимальной стоимости.

Лабораторная работа №4

13. реализовано вычисление числа остовных деревьев по теореме Кирхгофа;

14. реализовано построение минимального остова алгоритмом Краскала;
15. реализовано кодирование дерева кодом Прюфера;
16. реализовано декодирование дерева по коду Прюфера;
17. реализован поиск паросочетания.

Лабораторная работа №5

18. реализована проверка графа на эйлеровость;
19. реализована эйлеризация графа при необходимости;
20. реализовано построение эйлерова обхода;
21. реализовано построение фундаментальной системы циклов;
22. реализована операция симметрической разности циклов.

Главным результатом является не просто набор отдельных решений, а интегрированная архитектура, где алгоритмы взаимно дополняют друг друга. Граф, сеть потоков, остов и цикловая структура представлены как связанные сущности, а пользователь может шаг за шагом переходить от одной лабораторной работы к другой в рамках одной программы. Такой подход делает выполненный проект удобным как для учебной демонстрации, так и для дальнейшего расширения.

Приложение А. Файл AlgorithmComparator.h

```
#pragma once
#include "include/Graph.h"
#include "include/BFS.h"
#include "include/Dijkstra.h"

/// Comparison result between algorithms
struct AlgorithmComparison {
    int bfsIterations;
    int dijkstraIterations;
    double speedupFactor; // How many times faster BFS is
};

class AlgorithmComparator {
public:
    explicit AlgorithmComparator(const AdjacencyGraph& graph);

    /// Compare BFS and Dijkstra on the same graph
    /// Returns iteration counts and speedup factor
    AlgorithmComparison compare(int startVertex);

private:
    const AdjacencyGraph& m_graph;
};
```

Приложение Б. Файл AlgorithmComparator.cpp

```
#include "include/AlgorithmComparator.h"

AlgorithmComparator::AlgorithmComparator(const AdjacencyGraph&
    graph)
    : m_graph(graph)
{}

AlgorithmComparison AlgorithmComparator::compare(int startVertex)
{
    AlgorithmComparison result;

    // Run BFS
    BreadthFirstSearch bfs(m_graph);
    BFSResult bfsResult = bfs.traverseWithLevels(startVertex);
    result.bfsIterations = bfsResult.iterations;

    // Run Dijkstra
    DijkstraAlgorithm dijkstra(m_graph);
    DijkstraResult dijkstraResult = dijkstra.findShortestPaths(
        startVertex);
    result.dijkstraIterations = dijkstraResult.iterations;

    // Calculate speedup (BFS is typically faster for unweighted
    // traversal)
    if (result.dijkstraIterations > 0) {
        result.speedupFactor = static_cast<double>(result.
            dijkstraIterations) /
            static_cast<double>(result.
                bfsIterations);
    } else {
        result.speedupFactor = 0.0;
    }

    return result;
}
```

Приложение В. Файл BFS.h

```
#pragma once
#include "include/Graph.h"
#include <vector>
#include <map>

/// Result of BFS traversal with level information
struct BFSResult {
    std::vector<int> traversalOrder; // Order of visited vertices
    std::map<int, std::vector<int>> levels;
    int iterations; /// Number of vertices processed
    // int edgeVisits; // Number of edge examinations
    int maxLevel;    // Maximum depth reached
};

class BreadthFirstSearch {
public:
    explicit BreadthFirstSearch(const AdjacencyGraph& graph);

    /// Perform BFS traversal starting from a given vertex with
    /// level tracking
    BFSResult traverseWithLevels(int startVertex);

private:
    const AdjacencyGraph& m_graph;
};
```

Приложение Г. Файл BFS.cpp

```
#include "include/BFS.h"
#include <queue>
#include <vector>
#include <algorithm>
#include <unordered_map>

BreadthFirstSearch::BreadthFirstSearch(const AdjacencyGraph&
    graph)
    : m_graph(graph)
{}

BFSResult BreadthFirstSearch::traverseWithLevels(int startVertex)
{
    BFSResult result;
    result.iterations = 0;
    result.maxLevel = 0;

    if (!m_graph.hasVertex(startVertex)) {
        return result;
    }

    std::vector<int> vertexIds = m_graph.vertexIds();
    std::vector<bool> visited(m_graph.size(), false);

    // Build hash map for O(1) vertex ID to index lookup. FTF (
    now no need,
    // rn all the verteces are indexed from 0 to the last vertex.
    std::unordered_map<int, int> idToIndex;
    for (int i = 0; i < m_graph.size(); ++i) {
        result.iterations++;
        idToIndex[vertexIds[i]] = i;
    }

    // Queue stores {vertex, level}
    std::queue<std::pair<int, int>> queue;
    queue.push({startVertex, 0});
    //result.iterations++;

    int startIndex = idToIndex[startVertex];
```



```

if (startIndex == -1) {
    return result;
}

visited[startIndex] = true;
result.traversalOrder.push_back(startVertex);
result.levels[0].push_back(startVertex);

while (!queue.empty()) {
    auto [current, level] = queue.front();
    queue.pop();
    result.iterations++; // Count: queue pop (O(1) operation)

    result.maxLevel = std::max(result.maxLevel, level);

    /// Visit all neighbors
    for (const auto& [neighbor, weight] : m_graph.neighbors(
        current)) {
        // loop through all vertices connected to current +
        unpacking edge
        // (connected vertex + weight)
        result.iterations++;

        auto it = idToIndex.find(neighbor); // find the index
        of this neighbor vertex
        // (also FTF)
        //if (it != idToIndex.end()) {
            int neighborIndex = it->second; //
            if (!visited[neighborIndex]) {
                visited[neighborIndex] = true;
                result.traversalOrder.push_back(neighbor);
                result.levels[level + 1].push_back(neighbor);
                // next depth
                queue.push({neighbor, level + 1}); // we will
                process its neighbors later
                result.iterations++;
            }
        ///}
    }
}

```

```
    }  
  
    return result;  
}
```

Приложение Д. Файл Dijkstra.h

```
#pragma once
#include "include/Graph.h"
#include <vector>
#include <optional>
#include <map>

/// Result of Dijkstra's algorithm
struct DijkstraResult {
    std::map<int, double> distances;          // Shortest distances
        from start
    std::map<int, int> predecessors; // For path reconstruction
    std::vector<int> shortestPath; // Reconstructed path to
        target
    double targetDistance; // Distance to target vertex
    int iterations; // Number of iterations for comparison
};

class DijkstraAlgorithm {
public:
    explicit DijkstraAlgorithm(const AdjacencyGraph& graph);

    /// Find shortest paths from start vertex to all others
    /// If targetVertex is provided, also reconstruct the path
    DijkstraResult findShortestPaths(int startVertex, std::
        optional<int> targetVertex = std::nullopt);

private:
    const AdjacencyGraph& m_graph;

    /// Reconstruct path from start to target using predecessors
    std::vector<int> reconstructPath(int startVertex, int
        targetVertex,
                                const std::vector<int>& H,
                                const std::vector<int>&
                                    vertexIds) const;
};
```

Приложение Е. Файл `Dijkstra.cpp`

```
#include "include/Dijkstra.h"
#include <limits>
#include <algorithm>
#include <unordered_map>
#include <vector>

DijkstraAlgorithm::DijkstraAlgorithm(const AdjacencyGraph& graph)
    : m_graph(graph)
{}

DijkstraResult DijkstraAlgorithm::findShortestPaths(int
    startVertex, std::optional<int> targetVertex) {
    DijkstraResult result;
    result.iterations = 0;
    result.targetDistance = std::numeric_limits<double>::infinity
        ();

    if (!m_graph.hasVertex(startVertex)) {
        return result;
    }

    constexpr double INF = std::numeric_limits<double>::infinity
        ();

    auto vertexIds = m_graph.vertexIds();
    int p = m_graph.size(); // number of vertices (p in the
        algorithm)

    // Build hash map for vertex ID to index (1-based to match
        algorithm)
    std::unordered_map<int, int> idToIndex;
    for (int i = 0; i < p; ++i) {
        // result.iterations++;
        idToIndex[vertexIds[i]] = i;
    }

    // T[v] -- distance from s to v (T : array [1..p] of real)
    std::vector<double> T(p, INF);
```

```

// X[v] -- mark: 0 = unvisited, 1 = visited (X : array [1..p]
// of 0..1)
std::vector<int> X(p, 0);

// H[v] -- predecessor of v on shortest path (H : array [1..p]
// of 0..p)
std::vector<int> H(p, -1);

int s = startVertex;
int t = targetVertex.value_or(-1);

int startIndex = idToIndex[s];
T[startIndex] = 0;          // T[s] := 0
X[startIndex] = 1;          // X[s] := 1 (s is known)

int v = s;                  // v := s (current vertex)
int vIndex = startIndex;

// Label M: update labels
while (true) {
    //result.iterations++; // Count: main loop iteration

    // for u ∈ Γ(v) do -- examine all neighbors of v
    for (const auto& [neighbor, weight] : m_graph.neighbors(v)) {
        result.iterations++; // Count: examining edge

        auto neighborIt = idToIndex.find(neighbor);
        if (neighborIt == idToIndex.end()) {
            continue;
        }
        int uIndex = neighborIt->second;

        // if X[u] = 0 & T[u] > T[v] + C[v, u] then
        if (X[uIndex] == 0 && T[uIndex] > T[vIndex] + weight)
        {
            T[uIndex] = T[vIndex] + weight; // T[u] := T[v]
            // + C[v, u]
            H[uIndex] = vIndex;              // H[u] := v
            result.distances[neighbor] = T[uIndex];
            result.predecessors[neighbor] = v;
        }
    }
}

```

```

    }
}

//  $m := \infty$ ;  $v := \emptyset$ 
double m = INF;
v = 0;
vIndex = -1;

// for u from 1 to p do -- find vertex with minimum T
for (int u = 0; u < p; ++u) {
    result.iterations++; // Count: finding minimum

    // if  $X[u] = \emptyset$  &  $T[u] < m$  then
    if (X[u] == 0 && T[u] < m) {
        vIndex = u;
        m = T[u];
        v = vertexIds[u];
    }
}

// if  $v = \emptyset$  then stop -- no path from s to t
if (vIndex == -1) {
    break;
}

// if  $v = t$  then stop -- shortest path from s to t found
if (targetVertex.has_value() && v == t) {
    result.targetDistance = T[vIndex];
    result.shortestPath = reconstructPath(s, v, H,
        vertexIds);
    break;
}

X[vIndex] = 1; //  $X[v] := 1$  -- shortest path from s to v
                found
vIndex = vIndex;
}

// If target was specified, reconstruct path
if (targetVertex.has_value()) {
    int target = targetVertex.value();

```

```

        auto targetIt = idToIndex.find(target);
        if (targetIt != idToIndex.end() && T[targetIt->second] !=
            INF) {
            result.targetDistance = T[targetIt->second];
            result.shortestPath = reconstructPath(s, target, H,
                vertexIds);
        }
    }

    // Copy final distances to result
    for (int i = 0; i < p; ++i) {
        if (T[i] != INF) {
            result.distances[vertexIds[i]] = T[i];
        }
    }

    return result;
}

std::vector<int> DijkstraAlgorithm::reconstructPath(int
    startVertex, int targetVertex,

                                                                    const std::
                                                                    vector<
                                                                    int>& H,
                                                                    const std::
                                                                    vector<
                                                                    int>&
                                                                    vertexIds
                                                                    ) const {

    std::vector<int> path;

    if (startVertex == targetVertex) {
        path.push_back(startVertex);
        return path;
    }

    // Find target index
    auto targetIt = std::find(vertexIds.begin(), vertexIds.end(),
        targetVertex);
    if (targetIt == vertexIds.end()) {
        return {};
    }

```

```

    }
    int currentIdx = std::distance(vertexIds.begin(), targetIt);

    // Backtrack using H
    while (currentIdx != -1) {
        path.push_back(vertexIds[currentIdx]);
        if (vertexIds[currentIdx] == startVertex) {
            break;
        }
        currentIdx = H[currentIdx];
    }

    // Reverse to get start -> target order
    std::reverse(path.begin(), path.end());

    // Verify path starts at startVertex
    if (path.empty() || path.front() != startVertex) {
        return {};
    }

    return path;
}

```


Приложение Ж. Файл Eccentricity.h

```
#pragma once
#include "include/Graph.h"
#include "include/BFS.h"
#include <vector>
#include <optional>
#include <map>

/// Eccentricity computation result
struct EccentricityData {
    std::map<int, int> eccentricities; // in number of edges
    std::vector<int> centerVertices;
    std::vector<int> diametricalVertices;
    // Maps unordered vertex pairs to their diametrical paths
    std::map<std::pair<int, int>, std::vector<std::vector<int>>>
        diametricalPathsByPair;
    int radius; // minimum eccentricity (in edges)
    int diameter; // maximum eccentricity (in edges)
};

class GraphAnalyzer {
public:
    explicit GraphAnalyzer(const AdjacencyGraph& graph);

    EccentricityData analyze();

private:
    const AdjacencyGraph& m_graph;
    int m_vertexCount;

    // Compute shortest paths in terms of edge count
    std::map<std::pair<int,int>, int> computeEdgeCountPaths()
        const;

    // Find all shortest paths with exactly k edges between two
    // vertices
    std::vector<std::vector<int>> findAllPathsWithExactEdges(
        int start, int end, int edgeCount,
```

```
const std::map<std::pair<int,int>, int>& edgeCounts)
    const;
};
```

Приложение 3. Файл Eccentricity.cpp

```
#include "include/Eccentricity.h"
#include <limits>
#include <algorithm>
#include <functional>
#include <set>
#include <queue>

GraphAnalyzer::GraphAnalyzer(const AdjacencyGraph& graph)
    : m_graph(graph)
    , m_vertexCount(graph.size())
{}

/// Compute shortest paths in terms of edge count using BFS
std::map<std::pair<int,int>, int> GraphAnalyzer::
computeEdgeCountPaths() const {
    std::map<std::pair<int,int>, int> distances;
    auto vertices = m_graph.vertexIds();

    // For each starting vertex, run BFS to find distances to all
    // others
    for (int start : vertices) {
        std::map<int, int> dist;
        std::queue<int> q;

        // Initialize
        for (int v : vertices) {
            dist[v] = -1; // -1 = unreachable
        }
        dist[start] = 0;
        q.push(start);

        // BFS
        while (!q.empty()) {
            int current = q.front();
            q.pop();

            for (const auto& [neighbor, weight] : m_graph.
                neighbors(current)) {
                if (dist[neighbor] == -1) { // Not visited
```

```

        dist[neighbor] = dist[current] + 1;
        q.push(neighbor);
    }
}

// Store distances
for (int end : vertices) {
    if (dist[end] >= 0) {
        distances[{start, end}] = dist[end];
    } else {
        distances[{start, end}] = -1; // Unreachable
    }
}

return distances;
}

/// Find all paths with exactly k edges between two vertices
using DFS
std::vector<std::vector<int>> GraphAnalyzer::
findAllPathsWithExactEdges(
    int start,
    int end,
    int edgeCount,
    const std::map<std::pair<int,int>, int>& edgeCounts) const
{
    std::vector<std::vector<int>> result;

    // Check if path exists with exactly edgeCount edges
    if (edgeCounts.at({start, end}) != edgeCount) {
        return result; // No such path
    }

    // DFS to find all paths with exactly edgeCount edges
    std::vector<int> currentPath;
    currentPath.push_back(start);

    std::function<void(int, int)> dfs = [&](int current, int
        edgesUsed) {

```

```

        if (edgesUsed == edgeCount) {
            if (current == end) {
                result.push_back(currentPath);
            }
            return;
        }

        // Remaining edges needed
        int remaining = edgeCount - edgesUsed;

        for (const auto& [neighbor, weight] : m_graph.neighbors(
            current)) {
            // Avoid cycles
            if (std::find(currentPath.begin(), currentPath.end(),
                neighbor) != currentPath.end()) {
                continue;
            }

            // Check if we can reach end with exactly remaining
            edges from neighbor
            int distToEnd = edgeCounts.at({neighbor, end});
            if (distToEnd >= 0 && distToEnd == remaining - 1) {
                currentPath.push_back(neighbor);
                dfs(neighbor, edgesUsed + 1);
                currentPath.pop_back();
            }
        }
    };

    dfs(start, 0);
    return result;
}

EccentricityData GraphAnalyzer::analyze() {
    EccentricityData result;
    result.radius = -1;
    result.diameter = -1;

    auto vertices = m_graph.vertexIds();
    if (vertices.empty()) {
        return result;
    }

```

```

}

auto edgeCounts = computeEdgeCountPaths();
bool hasComputedEccentricity = false;

// Compute eccentricities (maximum edge distance for each vertex)
for (int v : vertices) {
    int ecc = -1; // -1 = unreachable to all
    bool hasReachable = false;

    for (int u : vertices) {
        if (u != v) {
            int dist = edgeCounts[{v, u}];
            if (dist >= 0) {
                ecc = std::max(ecc, dist);
                hasReachable = true;
            }
        }
    }

    // If no reachable vertices (end/sink vertex), set eccentricity to 0
    // TO REVERT TO INFINITE ECCENTRICITY FOR END VERTICES:
    // Change: result.eccentricities[v] = hasReachable ? ecc : 0;
    // To:      result.eccentricities[v] = hasReachable ? ecc : -1;
    // And update the radius/diameter check to: if (result.eccentricities[v] >= 0)
    result.eccentricities[v] = hasReachable ? ecc : 0;

    // Only consider vertices with non-negative eccentricity for radius/diameter
    if (result.eccentricities[v] >= 0) {
        if (!hasComputedEccentricity) {
            result.radius = result.eccentricities[v];
            result.diameter = result.eccentricities[v];
            hasComputedEccentricity = true;
        } else {

```

```

        result.radius = std::min(result.radius, result.
            eccentricities[v]);
        result.diameter = std::max(result.diameter,
            result.eccentricities[v]);
    }
}

// Find center and diametrical vertices
for (const auto& [v, ecc] : result.eccentricities) {
    if (ecc == result.radius) {
        result.centerVertices.push_back(v);
    }
    if (ecc == result.diameter) {
        result.diametricalVertices.push_back(v);
    }
}

// Find all diametrical pairs (unique unordered pairs at
// diameter edge distance)
std::set<std::pair<int, int>> diametricalPairs;

for (int start : vertices) {
    for (int end : vertices) {
        if (start != end) {
            int dist = edgeCounts[{start, end}];
            if (dist == result.diameter) {
                // Normalize pair: store as (min, max)
                int first = std::min(start, end);
                int second = std::max(start, end);
                diametricalPairs.insert({first, second});
            }
        }
    }
}

// Find all diametrical paths for each unique pair
for (const auto& [v1, v2] : diametricalPairs) {
    auto paths = findAllPathsWithExactEdges(v1, v2, result.
        diameter, edgeCounts);
    if (!paths.empty()) {

```

```
        result.diametricalPathsByPair[{v1, v2}] = paths;
    }
}

return result;
}
```


Приложение И. Файл EdmondsMatching.h

```
#pragma once
#include "include/Graph.h"
#include "include/MaxMatching.h"

/// Edmonds blossom algorithm for maximum matching in an
    undirected graph.
/// Finds the largest matching by number of edges, not by total
    weight.
class EdmondsMatching {
public:
    explicit EdmondsMatching(const AdjacencyGraph& graph);

    MatchingResult compute() const;

private:
    const AdjacencyGraph& m_graph;
};
```

Приложение К. Файл EdmondsMatching.cpp

```
#include "include/EdmondsMatching.h"
```

```
#include <algorithm>
```

```
#include <unordered_map>
```

```
#include <vector>
```

```
/*Термины
```

```
    , которыеиспользуютсяниже :
```

```
    Matching / паросочетание  $M$ : набор ребер
```

```
        , где никакие два ребра не имеют общей вершины .
```

```
    Matched / covered vertex: вершина
```

```
        , которая уже является концом какого-то ребра из  $M$ .
```

```
    Free / unmatched vertex: вершина
```

```
        , которая пока не покрыта ни одним ребром из  $M$ .
```

```
    Matched edge: ребро
```

```
        , которое уже входит в текущее паросочетание  $M$ .
```

```
    Unmatched edge: ребро графа
```

```
        , которое сейчас не входит в  $M$ .
```

```
    Alternating path / чередующийся путь : путь
```

```
        , где ребра идут по очереди : неиз
```

```
         $M$ , из  $M$ , неиз  $M$ , из  $M$ , ...
```

```
    Augmenting path / аугментирующий путь : чередующийся путь
```

```
        , который начинается и заканчивается в свободных вершинах
```

```
        . Если его перевернуть "", matching станет больше на 1 ребро.
```

```
    Root / корневая вершина : свободная вершина
```

```
        , от которой мы начинаем BFS поиск – аугментирующего пути
```

```
        . В коде это параметр root в bfsAugment(root).
```

```
    Alternating tree / чередующееся дерево :
```

```
        BFS дерево –, которое строится от root. Вне пути от root  
        до любой вершины то же чередуется по ребрам
```

: неиз M , из M , неиз M , ...

Even-level vertex / вершина четного уровня : вершина
, до которой от $root$ идет путь четной длины .
Именно такие вершины лежат в очереди
 BFS в этом коде .

Odd-level vertex / вершина нечетного уровня : вершина
, до которой от $root$ идет путь нечетной длины . Если она покрыта
 $matching$, мы сразу перепрыгиваем "" через ее $matched\ edge$
к парной вершине четного уровня
.

$parent[v]$: откуда мы пришли к вершине
 v в чередующемся дереве . По $parent[]$
потом восстанавливается найденный аугментирующий путь
.

Blossom: нечетный цикл
, найденный внутри чередующегося дерева .
Его можно временно считать одной вершиной
, чтобы BFS не застревал на цикле .

Base / база *blossom*: вершина цикла
, ближайшая к $root$. После сжатия *blossom*
все вершины внутри него имеют одну общую базу в массиве
 $base[]$.

LCA:
Lowest common ancestor,
то есть ближайший общий предок двух вершин в чередующемся дереве
. В этом алгоритме LCA становится базой *blossom*.

Contraction / *shrink*: временное сжатие
blossom в одну супервершину "" .
В коде это делается без создания нового графа
, а перенаправлением $base[]$. Алгоритм Эдмондса

(*blossom*), общая идея : нужно найти

maximum matching, то есть самое большое по числу ребер паросочетание

- Паросочетание M – это набор ребер, где никакие два ребра не имеют общей вершины.
- Алгоритм держит текущее

M ищет augmenting путь. Это путь, который начинается в свободной вершине, заканчивается в другой свободной вершине, а ребра в нем чередуются так:

: неиз

M_1 , из M_1 , неиз M_2 , из M_2 , ..., неиз M_k . Если такой путь найден

- ребра в нем переворачиваются "": те
 - , чтобы в M , удаляются, те
 - , которых не было в M , добавляются.
- После этого размер паросочетания увеличивается на 1.
1. Вдоль этого графа для этого хватает обычного

BFS по чередующимся путям. В общем графе мешают нечетные циклы. Такой нечетный цикл называется blossom. Эдмондс временно сжимает весь blossom в одну вершину, продолжает поиск, а потом по $parent/base$ восстанавливает настоящий путь. Как это сделано именно тут

:

1. *EdmondsWorker* сначала переводит id вершин графа в индексы $0..n-1$. Все массивы ниже работают с этими индексами. В самом конце индексы переводятся обратно в настоящие id вершин.
2. m_match заполняется -1 . Это значит, что изначально паросочетание пустое. Никакого жадного начального шага здесь нет.
3. $run()$ идет по всем вершинам. Если вершина v все еще свободна, вызывается $findAugmentingPath(v)$. Это не значит, что v сразу добавляется в $matching$. Это значит только

: пробуем построить от нее чередующееся *BFS*-дерево.

4. В *bfsAugment(root)* начальный путь "" состоит только из *root*:

```
queue = { root }
```

```
parent[*] = -1
```

```
base[i] = i
```

Отдельным массивом путь во время поиска не хранится

. Вместо этого запоминаются

parent-ссылки.

Сам путь потом восстанавливается назад от найденной свободной конечной вершины

.

5. В очереди *BFS*

лежат вершины четного уровня чередующегося дерева .

Для такой вершины

u смотрим каждое ребро графа (u, v) :

– Если *v* свободна и еще не посещалась , то $root \dots u - v$ уже является аугментирующим путем . *bfsAugment* возвращает *v*.

– Если *v* уже покрыта парой сочетания $x - v$, то продолжать путь можно только через ребро парочетания $v - x$. Поэтому ставим

```
parent[v] = u
```

и кладем $x = m_match[v]$

в очередь как новую вершину четного уровня

.

– Если *v* уже лежит в этом же чередующемся дереве так , что получается нечетный цикл

, найден *blossom*. *contractBlossom(u, v)*

находит его базу через

```
findLCA(),
```

отмечает обе стороны цикла и для всех вершин внутри цикла перенаправляет

```
base[]
```

на общую базу .

6. Когда *bfsAugment* возвращает свободную конечную вершину , *augment(endpoint)* идет назад по

```
parent[]
```

и переворачивает ребра на найденном пути :

```
endpoint <- parent[endpoint] <- matched partner <- ...
```

После этого

matching становится на одно ребро больше .

7. Если от *root* аугментирующий путь не найден , *matching* не меняется . Внешний цикл просто пробует следующую свободную вершину . Важно

: этот алгоритм максимизирует количество ребер .
Вес тут не участвует в выборе
, он нужен только для красивого вывода найденных ребер .

*/

namespace {

// Internal worker: the algorithm uses dense vertex indexes.

class EdmondsWorker {

public:

EdmondsWorker(**const** AdjacencyGraph& graph)

: m_graph(graph)

, m_n(graph.size())

{

// Original id <-> dense index.

m_vertexIds = graph.vertexIds();

for (**int** i = 0; i < m_n; ++i) {

m_idToIndex[m_vertexIds[i]] = i;

}

// Dense adjacency list. Self-loops are useless for matching.

m_adj.assign(m_n, {});

for (**const** WeightedEdge& edge : graph.edges()) {

const int u = m_idToIndex[edge.from];

const int v = m_idToIndex[edge.to];

if (u == v) **continue**; *// skip self-loops*

m_adj[u].push_back(v);

m_adj[v].push_back(u);

}

}

/// Run Edmonds. match[i] == -1 means i is unmatched.

std::vector<**int**> run() {

```

    m_match.assign(m_n, -1);

    // Try to grow the matching from every free vertex.
    for (int v = 0; v < m_n; ++v) {
        if (m_match[v] == -1) {
            findAugmentingPath(v);
        }
    }
    return m_match;
}

```

private:

```

// Graph data in dense indexes.
const AdjacencyGraph& m_graph;
int m_n;
std::vector<int> m_vertexIds;           // dense index ->
    original ID
std::unordered_map<int, int> m_idToIndex; // original ID ->
    dense index
std::vector<std::vector<int>> m_adj;    // dense adjacency

// State for one BFS search.
std::vector<int> m_match;              // match[v] = partner of v in M
    , or -1
std::vector<int> m_parent;             // BFS-tree parent (in the
    alternating forest)
std::vector<int> m_base;               // current blossom base for
    each vertex
std::vector<int> m_q;                 // BFS queue (dense indices)
std::vector<bool> m_inQueue;           // BFS visited flag
std::vector<bool> m_inBlossom;         // marker used when
    contracting a blossom

// Lowest common ancestor in the alternating forest.
int findLCA(int a, int b) {
    std::vector<bool> visitedByA(m_n, false);

    // Mark the path from a to the root.
    int x = a;
    while (true) {
        x = m_base[x];
    }
}

```

```

        visitedByA[x] = true;
        if (m_match[x] == -1) break;
        x = m_parent[m_match[x]];
    }

    // Climb from b until we hit that path.
    int y = b;
    while (true) {
        y = m_base[y];
        if (visitedByA[y]) return y;
        y = m_parent[m_match[y]];
    }
}

// Mark one side of a blossom.
void markBlossomPath(int u, int b, int child) {
    while (m_base[u] != b) {
        m_parent[u] = child;
        child = m_match[u];
        if (!m_inQueue[child]) {
            m_q.push_back(child);
            m_inQueue[child] = true;
        }
        m_inBlossom[m_base[u]] = true;
        m_inBlossom[m_base[child]] = true;
        u = m_parent[child];
    }
}

// Shrink the blossom found through edge (u, v).
void contractBlossom(int u, int v) {
    const int lca = findLCA(u, v);

    std::fill(m_inBlossom.begin(), m_inBlossom.end(), false);
    markBlossomPath(u, lca, v);
    markBlossomPath(v, lca, u);

    // Redirect all marked bases to the blossom base.
    for (int x = 0; x < m_n; ++x) {
        if (m_inBlossom[m_base[x]]) {
            m_base[x] = lca;
        }
    }
}

```



```

    }
}

// BFS in the alternating forest. Returns free endpoint or
// -1.
int bfsAugment(int root) {
    m_parent.assign(m_n, -1);
    m_inQueue.assign(m_n, false);
    m_inBlossom.assign(m_n, false);

    // Each vertex starts as its own base.
    m_base.resize(m_n);
    for (int i = 0; i < m_n; ++i) m_base[i] = i;

    m_q.clear();
    m_q.push_back(root);
    m_inQueue[root] = true;

    // Queue stores even-level vertices.
    for (size_t head = 0; head < m_q.size(); ++head) {
        const int u = m_q[head];
        for (int v : m_adj[u]) {
            // Skip the edge already used in matching.
            if (m_base[u] == m_base[v] || m_match[u] == v)
                continue;

            // Odd cycle in the tree: contract it.
            if (v == root || (m_match[v] != -1 && m_parent[
                m_match[v]] != -1)) {
                contractBlossom(u, v);
            }
            // New vertex in the forest.
            else if (m_parent[v] == -1) {
                m_parent[v] = u;
                if (m_match[v] == -1) {
                    // Free vertex means augmenting path is
                    // found.
                    return v;
                }
                // Jump through the matched edge.
            }
        }
    }
}

```

```

        m_q.push_back(m_match[v]);
        m_inQueue[m_match[v]] = true;
    }
}
}
return -1;
}

// Flip edges along the found augmenting path.
void augment(int v) {
    while (v != -1) {
        const int pv = m_parent[v];
        const int next = (pv == -1) ? -1 : m_match[pv];
        m_match[v] = pv;
        m_match[pv] = v;
        v = next;
    }
}

// One attempt to improve the matching.
void findAugmentingPath(int root) {
    const int endpoint = bfsAugment(root);
    if (endpoint != -1) {
        augment(endpoint);
    }
}

};

}

// Public wrapper: convert dense indexes back to graph vertex ids
.
EdmondsMatching::EdmondsMatching(const AdjacencyGraph& graph)
    : m_graph(graph)
{}

MatchingResult EdmondsMatching::compute() const {
    MatchingResult result;
    result.matchingSize = 0;
    result.isPerfect = false;

```

```

const int n = m_graph.size();
if (n == 0) {
    return result;
}

EdmondsWorker worker(m_graph);
const std::vector<int> match = worker.run();

// Report each pair only once.
const std::vector<int> vertexIds = m_graph.vertexIds();
std::unordered_map<int, int> idToIndex;
for (int i = 0; i < n; ++i) idToIndex[vertexIds[i]] = i;

for (int i = 0; i < n; ++i) {
    const int j = match[i];
    if (j == -1 || j < i) continue; // unmatched, or
        duplicate direction

    const int idA = vertexIds[i];
    const int idB = vertexIds[j];

    // Weight is only for output.
    auto weightOpt = m_graph.getEdgeWeight(idA, idB);
    if (!weightOpt.has_value()) {
        weightOpt = m_graph.getEdgeWeight(idB, idA);
    }
    const double w = weightOpt.value_or(0.0);

    result.matchingEdges.push_back({idA, idB, w});
}

// Stable output order.
std::sort(result.matchingEdges.begin(), result.matchingEdges.
    end(),
    [](const WeightedEdge& a, const WeightedEdge& b) {
        if (a.from != b.from) return a.from < b.from;
        return a.to < b.to;
    });

result.matchingSize = static_cast<int>(result.matchingEdges.
    size());

```

```
// Uncovered vertices.
for (int i = 0; i < n; ++i) {
    if (match[i] == -1) {
        result.unmatchedVertices.push_back(vertexIds[i]);
    }
}
result.isPerfect = result.unmatchedVertices.empty();

return result;
}
```

Приложение Л. Файл EulerianCycle.h

```
#pragma once
#include "include/Graph.h"
#include <memory>
#include <string>
#include <vector>

/// One modification step recorded while making the graph
    Eulerian.
/// We may either add an edge or remove one while fixing
    connectivity/parity.
struct EulerianModification {
    int from;
    int to;
    bool added;           // true = edge added, false = edge
        removed
    std::string reason; // human-readable, e.g. "connect
        components" / "fix odd parity"
};

/// How aggressive the eulerization is allowed to be.
/// NonMultigraphOnly: only add edges between vertices that are
    not already
///
    connected by an edge. If the graph
    cannot be
///
    eulerized this way, the builder fails
    with
///
    `requiresMultigraph = true` and lets the
    caller
///
    decide whether to retry in
    AllowMultigraph mode.
/// AllowMultigraph: pair odd-degree vertices regardless of
    whether they
///
    already share an edge -- the resulting
    graph may be
///
    a multigraph.
enum class EulerizationMode {
    NonMultigraphOnly,
    AllowMultigraph,
    DeleteEdgesOnly
}
```

```

};

enum class EulerTraversalKind {
    None,
    Cycle,
    Path
};

struct EulerianCycleResult {
    bool wasAlreadyEulerian;           // true if the
        input already had an Eulerian cycle
    bool wasSemiEulerian;             // true if the
        input already had an Eulerian path (but not a cycle)
    std::vector<EulerianModification> additions; // edge
        modifications, in the order they were applied
    std::vector<int> traversal;         // vertex
        sequence of the Eulerian cycle/path
    std::unique_ptr<AdjacencyGraph> modifiedGraph; // graph
        actually used for the traversal (== input if no changes)
    bool success;                     // false only
        for pathological inputs (e.g. empty graph)
    EulerTraversalKind traversalKind;  // whether `
        traversal` is a cycle or a path
    /// Set to true only in NonMultigraphOnly mode when
        eulerization failed
    /// because the only way to fix odd parities would require
        duplicating
    /// an existing edge (i.e. turning the graph into a
        multigraph).
    /// When this is true, `success` is false and `traversal` is
        empty -- the
    /// caller should ask the user whether to retry in
        AllowMultigraph mode.
    bool requiresMultigraph;
};

class EulerianCycleBuilder {
public:
    explicit EulerianCycleBuilder(const AdjacencyGraph& graph);

```

```

    /// Check the Euler condition on the "edge-bearing" subgraph.
        If the
    /// graph is already Eulerian, build a cycle. If it is semi-
        Eulerian
    /// (exactly two odd-degree vertices), build a path unless
    /// `preferEulerization` is true. Otherwise modify the graph
        if needed
    /// (logging every change), then build an Eulerian cycle.
    ///
    /// `mode` controls whether the builder is allowed to create
        parallel
    /// edges. See EulerizationMode for details.
    EulerianCycleResult compute(
        EulerizationMode mode = EulerizationMode::
            NonMultigraphOnly,
        bool preferEulerization = false) const;

private:
    const AdjacencyGraph& m_graph;
};

```

Приложение М. Файл EulerianCycle.cpp

```
#include "include/EulerianCycle.h"
#include <algorithm>
#include <functional>
#include <set>
#include <stack>
#include <unordered_map>
#include <unordered_set>

EulerianCycleBuilder::EulerianCycleBuilder(const AdjacencyGraph&
    graph)
    : m_graph(graph)
{}

namespace {

// ----- Helpers
-----

// Make a writable copy of the input graph. Eulerization may add
// edges, and
// we do not want to mutate the user's graph in place.
std::unique_ptr<AdjacencyGraph> cloneGraph(const AdjacencyGraph&
    g) {
    auto copy = std::make_unique<AdjacencyGraph>(g.isDirected());
    for (int v : g.vertexIds()) {
        copy->addVertex(v);
    }
    for (const WeightedEdge& e : g.edges()) {
        copy->addEdge(e.from, e.to, e.weight);
    }
    return copy;
}

// Degree of v in the (multi-)graph: just the size of its
// adjacency list, since
// addEdge() pushes one entry per endpoint per occurrence.
int degreeOf(const AdjacencyGraph& g, int v) {
    return static_cast<int>(g.neighbors(v).size());
}
```



```

}

// Find connected components, but only count vertices that touch
// at least one
// edge. An Eulerian cycle does not need to "visit" isolated
// vertices, so
// they should not force us to add bridges to them.
//
// Returns: list of components, each component being a list of
// vertex IDs.
std::vector<std::vector<int>> connectedComponentsWithEdges(const
AdjacencyGraph& g) {
    std::vector<std::vector<int>> components;
    std::unordered_set<int> visited;

    for (int start : g.vertexIds()) {
        if (visited.count(start)) continue;
        if (g.neighbors(start).empty()) continue; // skip
            isolated vertices

        // Standard BFS over the undirected graph.
        std::vector<int> component;
        std::vector<int> queue{start};
        visited.insert(start);
        size_t head = 0;
        while (head < queue.size()) {
            int u = queue[head++];
            component.push_back(u);
            for (const auto& [w, _wt] : g.neighbors(u)) {
                if (!visited.count(w)) {
                    visited.insert(w);
                    queue.push_back(w);
                }
            }
        }
        components.push_back(std::move(component));
    }
    return components;
}

// Hierholzer's algorithm for an undirected (multi-)graph.

```

```

//
// Idea: starting at `start`, walk along edges (each used at most
// once); when
// stuck, append the current vertex to the answer and backtrack.
// If all
// degrees are even, the result is an Eulerian cycle. If exactly
// two vertices
// are odd and `start` is one of them, the same routine yields an
// Eulerian
// path.
//
// Implementation: iterative, using a stack of vertices. We
// maintain, per
// vertex, an "edge iterator" pointing to the next adjacency-list
// slot that
// has not yet been consumed, plus a side `used` set keyed by
// edge IDs so the
// peer endpoint can also see that the edge is gone.
std::vector<int> hierholzer(const AdjacencyGraph& g, int start) {
    // Pull adjacency lists into a mutable structure where each
    // entry has a
    // unique "edge id" so the same edge can be marked used from
    // both sides.
    struct AdjSlot {
        int neighbor;
        int edgeId;
    };

    std::unordered_map<int, std::vector<AdjSlot>> adj;
    int nextId = 0;

    // We need the same edgeId for the (u,v) and (v,u) entries
    // that came from
    // the SAME addEdge call. AdjacencyGraph::edges() lists each
    // undirected
    // edge once with from < to; iterate that list and assign one
    // id per edge.
    for (const WeightedEdge& e : g.edges()) {
        const int id = nextId++;
        adj[e.from].push_back({e.to, id});
        adj[e.to].push_back({e.from, id});
    }
}

```

```

}

std::unordered_set<int> usedEdges;           // edges already
      consumed
std::unordered_map<int, size_t> cursor; // next index to
      inspect in adj[v]

std::stack<int> path;           // current incomplete walk
std::vector<int> trail;        // result, built in reverse order

////////// S := ∅ { стек для хранения вершин }
////////// select v ∈ V { произвольная вершина }
path.push(start);
////////// v → S { положить v в стек S }

// while S ≠ ∅ do
while (!path.empty()) {
    ////////// while S ≠ ∅ do

    int u = path.top();
    ////////// v := top S { v — верхний элемент стека }

    auto& list = adj[u];
    size_t& i = cursor[u];

    // Skip past edges that have already been used (possibly
    // by the other
    // endpoint). This loop ends either at an unused edge or
    // at end().
    while (i < list.size() && usedEdges.count(list[i].edgeId)
        ) {
        ++i;
    }

    if (i == list.size()) {
        // Stuck at u: no outgoing unused edges left. Move u
        // into the
        // result and continue from whatever is below it on
        // the stack.
        // Building the trail this way (popping) naturally
        // handles the

```

```

        // splice case -- subcycles get inserted in the right
        // place.
        trail.push_back(u);
        path.pop();
        ////////// if  $\Gamma[v] = \emptyset$  then
        //////////       $v \leftarrow S$ ; yield  $v$  {
        //очередная вершина эйлерова цикла      }
    } else {
        // Take this edge: walk to the neighbor, mark the
        // edge consumed.
        const AdjSlot slot = list[i];
        usedEdges.insert(slot.edgeId);
        ++i;
        path.push(slot.neighbor);
        ////////// else
        //////////      select  $u \in \Gamma[v]$  {
        //взять первую вершину из списка смежности      }
        //////////       $u \rightarrow S$  { положить  $u$  в стек }
        //////////       $\Gamma[v] := \Gamma[v] - u$ ;  $\Gamma[u] := \Gamma[u] - v$  {
        //удалить ребро  $(v, u)$  }
        ////////// end if
    }
}
}
////////// end while

// We were popping, so the trail is currently reversed.
std::reverse(trail.begin(), trail.end());
return trail;
////////// Выход: последовательность вершин эйлерова цикла
}

// Set of unordered pairs (u,v) representing the edges already
// present in the
// graph. Used to detect when adding (u,v) would duplicate an
// existing edge.
std::set<std::pair<int,int>> existingEdgePairs(const
AdjacencyGraph& g) {
    std::set<std::pair<int,int>> pairs;
    for (const WeightedEdge& e : g.edges()) {
        int a = e.from, b = e.to;
        if (a > b) std::swap(a, b);
    }
}

```

```

        pairs.insert({a, b});
    }
    return pairs;
}

// Try to find a perfect matching of `oddVertices` such that no
    matched pair
// (u,v) already exists in `forbidden`. Returns true on success
    and fills
// `pairs` with the matching. Uses backtracking; the number of
    odd vertices
// in our use cases is small enough for this to be fine.
bool findNonMultigraphMatching(const std::vector<int>&
    oddVertices,

                                const std::set<std::pair<int,int>
                                    >>& forbidden,
                                std::vector<std::pair<int,int>>&
                                    pairs) {
    const size_t n = oddVertices.size();
    std::vector<bool> used(n, false);

    auto isForbidden = [&](int u, int v) {
        int a = u, b = v;
        if (a > b) std::swap(a, b);
        return forbidden.count({a, b}) > 0;
    };

    // Recursive helper: pick the lowest-index unused vertex, try
        every
    // unused partner that does not collide with an existing edge
    .
    std::function<bool()> recurse = [&]() -> bool {
        // Find first unused index.
        size_t i = 0;
        while (i < n && used[i]) ++i;
        if (i == n) return true; // all matched

        used[i] = true;
        for (size_t j = i + 1; j < n; ++j) {
            if (used[j]) continue;

```

```

        if (isForbidden(oddVertices[i], oddVertices[j]))
            continue;

        used[j] = true;
        pairs.push_back({oddVertices[i], oddVertices[j]});
        if (recurse()) return true;
        pairs.pop_back();
        used[j] = false;
    }
    used[i] = false;
    return false;
};

pairs.clear();
return recurse();
}

bool canRemoveEdgeWithoutDisconnecting(const AdjacencyGraph& g,
    int u, int v) {
    auto copy = cloneGraph(g);
    if (!copy->removeEdge(u, v).has_value()) {
        return false;
    }

    if (copy->edges().empty()) {
        return false;
    }

    return connectedComponentsWithEdges(*copy).size() <= 1;
}

bool findDeletionMatching(AdjacencyGraph& graph,
    const std::vector<int>& oddVertices,
    std::vector<std::pair<int,int>>&
        deletions) {
    const size_t n = oddVertices.size();
    std::vector<bool> used(n, false);

    std::function<bool()> recurse = [&]() -> bool {
        size_t i = 0;
        while (i < n && used[i]) ++i;

```

```

        if (i == n) return true;

        used[i] = true;
        for (size_t j = i + 1; j < n; ++j) {
            if (used[j]) continue;

            const int u = oddVertices[i];
            const int v = oddVertices[j];
            if (!graph.getEdgeWeight(u, v).has_value()) continue;
            if (!canRemoveEdgeWithoutDisconnecting(graph, u, v))
                continue;

            const auto removedWeight = graph.removeEdge(u, v);
            if (!removedWeight.has_value()) continue;

            used[j] = true;
            deletions.push_back({u, v});
            if (recurse()) return true;

            deletions.pop_back();
            used[j] = false;
            graph.addEdge(u, v, *removedWeight);
        }

        used[i] = false;
        return false;
    };

    deletions.clear();
    return recurse();
}

} // namespace

EulerianCycleResult EulerianCycleBuilder::compute(
    EulerizationMode mode,
    bool preferEulerization) const {
    EulerianCycleResult result;
    result.success = false;
    result.wasAlreadyEulerian = false;
    result.wasSemiEulerian = false;

```

```

result.requiresMultigraph = false;
result.traversalKind = EulerTraversalKind::None;

// Always work on a copy so we can add edges freely.
result.modifiedGraph = cloneGraph(m_graph);
AdjacencyGraph& G = *result.modifiedGraph;

if (G.isDirected()) {
    // Per the lab spec we only handle undirected graphs.
    return result;
}

if (G.edges().empty()) {
    // No edges -> trivially "Eulerian" with an empty cycle,
    but there is
    // nothing meaningful to walk. Surface this as not-
    success so the
    // caller can print a sensible message.
    return result;
}

// Fast path: if the original graph is already connected on
the edge-
// bearing subgraph, then we may be able to build an Euler
traversal
// without adding any edges at all.
const auto originalComponents = connectedComponentsWithEdges(
    G);
if (originalComponents.size() <= 1) {
    std::vector<int> originalOddVertices;
    for (int v : G.vertexIds()) {
        if (degreeOf(G, v) % 2 == 1) {
            originalOddVertices.push_back(v);
        }
    }
    std::sort(originalOddVertices.begin(),
        originalOddVertices.end());

    if (originalOddVertices.empty()) {
        int start = -1;
        for (int v : G.vertexIds()) {

```



```

        if (!G.neighbors(v).empty()) {
            start = v;
            break;
        }
    }
    if (start == -1) {
        return result;
    }

    result.traversal = hierholzer(G, start);
    result.traversalKind = EulerTraversalKind::Cycle;
    result.wasAlreadyEulerian = true;
    result.success = true;
    return result;
}

if (originalOddVertices.size() == 2 && !
preferEulerization) {
    result.traversal = hierholzer(G, originalOddVertices.
        front());
    result.traversalKind = EulerTraversalKind::Path;
    result.wasSemiEulerian = true;
    result.success = true;
    return result;
}
}

// ---- Step 1. Make sure the edge-bearing part of the graph
// is connected.
//
// If we have several components that each contain edges, we
// must add
// bridge edges between them. Connecting k components needs
// k-1 bridges.
// We always bridge the first vertex of each component (
// deterministic).
//
// Bridging two different components can never create a
// parallel edge
// (the endpoints had no path between them, let alone a
// direct edge), so

```

```

// step 1 is safe in either mode.
while (true) {
    auto components = connectedComponentsWithEdges(G);
    if (components.size() <= 1) break;

    const int u = components[0].front();
    const int v = components[1].front();
    G.addEdge(u, v, 1.0);
    result.additions.push_back({u, v, true, "connect
        components"});
}

// ---- Step 2. Fix odd-degree vertices by pairing.
//
// Handshaking Lemma: there is always an even number of odd-
// degree
// vertices, so pairing always succeeds. Each added edge
// flips the
// parity of its two endpoints, turning two odd vertices into
// even ones.
std::vector<int> oddVertices;
for (int v : G.vertexIds()) {
    if (degreeOf(G, v) % 2 == 1) {
        oddVertices.push_back(v);
    }
}

// Sort for reproducibility -- same input should always pair
// the same way.
std::sort(oddVertices.begin(), oddVertices.end());

// Try to pair odd vertices. In NonMultigraphOnly mode we
// look for a
// perfect matching that avoids any pair already connected by
// an edge.
// If no such matching exists, we cannot fix the graph
// without making it
// a multigraph -- bail out and let the caller decide whether
// to retry.
if (!oddVertices.empty()) {
    std::vector<std::pair<int,int>> pairs;

```

```

bool foundClean = false;

if (mode == EulerizationMode::NonMultigraphOnly) {
    const auto forbidden = existingEdgePairs(G);
    foundClean = findNonMultigraphMatching(oddVertices,
        forbidden, pairs);

    if (!foundClean) {
        // Cannot eulerize without creating parallel
        // edges. Surface
        // this to the caller; do NOT mutate the graph
        // any further
        // and do NOT compute a cycle.
        result.requiresMultigraph = true;
        result.success = false;
        // Roll back the modifiedGraph to the post-step-1
        // state we
        // already have, but leave additions intact so
        // the caller can
        // see the bridge edges that would still be
        // needed.
        return result;
    }
} else if (mode == EulerizationMode::DeleteEdgesOnly) {
    std::vector<std::pair<int,int>> deletions;
    const bool foundDeletionFix = findDeletionMatching(G,
        oddVertices, deletions);
    if (!foundDeletionFix) {
        result.requiresMultigraph = true;
        result.success = false;
        return result;
    }

    for (const auto& [u, v] : deletions) {
        result.additions.push_back({u, v, false, "fix odd
            parity by deleting edge"});
    }
} else {
    // AllowMultigraph: just pair adjacent odd vertices
    // in the sorted

```

```

        // list. This matches the original behavior and is
        // guaranteed to
        // succeed (handshaking Lemma).
        for (size_t i = 0; i + 1 < oddVertices.size(); i +=
            2) {
            pairs.push_back({oddVertices[i], oddVertices[i +
                1]});
        }
    }

    // Apply the chosen pairing.
    for (const auto& [u, v] : pairs) {
        G.addEdge(u, v, 1.0);
        result.additions.push_back({u, v, true, "fix odd
            parity"});
    }
}

result.wasAlreadyEulerian = result.additions.empty();

// ---- Step 3. Pick a start vertex (any vertex incident to
// some edge)
// and run Hierholzer's algorithm. After eulerization all
// degrees are
// even, so the traversal is guaranteed to be a cycle.
int start = -1;
for (int v : G.vertexIds()) {
    if (!G.neighbors(v).empty()) {
        start = v;
        break;
    }
}
if (start == -1) {
    // Should not happen: we returned earlier on edge-empty
    // graphs.
    return result;
}

result.traversal = hierholzer(G, start);
result.traversalKind = EulerTraversalKind::Cycle;
result.success = true;

```

```
    return result;  
}
```

Приложение Н. Файл FlowNetwork.h

```
#pragma once

#include <optional>
#include <ostream>
#include <unordered_map>
#include <vector>

// one real edge in flow network
struct FlowEdge {
    int from;
    int to;
    int capacity; // how much can pass through this edge at max
    int cost; // cost of sending 1 unit of flow through this edge
    int flow; // how much flow is currently pushed through this
               edge
};

// edge from residual network, not from the original one
// needed because max-flow / min-cost-flow work with "what can
// still be changed"
struct ResidualArc {
    int from;
    int to;
    int edgeIndex; // refers to the original edge from FlowEdge
                   from m_edges
    bool forward; // true: forward through original edge;
                  // false: back (opportunity to reduce already
                  // pushed flow)
    int residualCapacity; // if it's forward: cap - flow
                          // if it's back: flow
    int cost; // forward: just cost
              // back: negative (we're taking flow back)
};

/*
Type for:
- capacity matrix
- cost matrix
- flow matrix
*/
```

```

'optional' because there might be no edge between two vertices.
    if exists: num
    if not: std::nullopt
why this and not 0:
    - 0 might be normal value
    - I used it before to show the absence of an edge
*/
using FlowMatrix = std::vector<std::vector<std::optional<int>>>;

class FlowNetwork {
public:
    explicit FlowNetwork(bool directed = true);

    void addVertex(int id);
    bool addEdge(int from, int to, int capacity, int cost);

    bool hasVertex(int id) const;
    bool isDirected() const;
    int size() const;

    std::vector<int> vertexIds() const;
    std::vector<FlowEdge> edges() const;
    // build neighbors for residual network "on the fly"
    // forward residual edge: we still can push cap - flow
    // backward residual edge: we can take already pushed flow
    back
    std::vector<ResidualArc> residualNeighbors(int vertex) const;

    // accessors for one real edge
    std::optional<FlowEdge> getEdge(int from, int to) const;
    std::optional<int> getCapacity(int from, int to) const;
    std::optional<int> getCost(int from, int to) const;
    std::optional<int> getFlow(int from, int to) const;

    // make matrix views for printing
    FlowMatrix getCapacityMatrix() const;
    FlowMatrix getCostMatrix() const;
    FlowMatrix getFlowMatrix() const;

    void resetFlows(); // set all current flows to 0
    /*

```

```

    update real edge using residual move
        if forward: flow += delta
        if back:     flow -= delta
    generally connects the res. arcs logic to flows of real arcs
    */
    void augment(int edgeIndex, bool forward, int delta);

private:
    std::optional<int> findEdgeIndex(int from, int to) const; //
        find real edge in m_edges
    int indexForVertex(int id) const; // for matrices

    bool m_directed;
    std::vector<int> m_vertices; // list of ids as they were
        added
    std::unordered_map<int, int> m_idToIndex; // id -> position
        in m_vertices / matrices
    std::vector<FlowEdge> m_edges; // all real edges live here
    std::unordered_map<int, std::vector<int>> m_outgoing; //
        vertex -> indexes of outgoing edges in m_edges
    std::unordered_map<int, std::vector<int>> m_incoming; //
        vertex -> indexes of incoming edges in m_edges
};

void printFlowMatrix(std::ostream& out, const FlowMatrix& matrix,
                    const std::vector<int>& vertexIds);

```


Приложение О. Файл FlowNetwork.cpp

```
#include "include/FlowNetwork.h"

#include <iomanip>

namespace {
// helper so all 3 matrix builders use the same empty "square" (
// filled w/ nullopt)
FlowMatrix createEmptyMatrix(int size) {
    return FlowMatrix(size, std::vector<std::optional<int>>(size,
        std::nullopt));
}
}

FlowNetwork::FlowNetwork(bool directed)
    : m_directed(directed) {}

void FlowNetwork::addVertex(int id) {
    if (hasVertex(id)) {
        return;
    }

    // index is just current position in vertex list
    m_idToIndex[id] = static_cast<int>(m_vertices.size());
    m_vertices.push_back(id);
}

bool FlowNetwork::addEdge(int from, int to, int capacity, int
cost) {
    // no negative capacities and no duplicate directed edge
    if (capacity < 0 || findEdgeIndex(from, to).has_value()) {
        return false;
    }

    // add endpoints if needed
    addVertex(from);
    addVertex(to);

    // flow starts from 0, algorithms will change it later
    const int edgeIndex = static_cast<int>(m_edges.size());
```

```

        m_edges.push_back({from, to, capacity, cost, 0});

        // store only indexes into m_edges, so we don't duplicate the
            edge itself
        m_outgoing[from].push_back(edgeIndex);
        m_incoming[to].push_back(edgeIndex);
        return true;
    }

    bool FlowNetwork::hasVertex(int id) const {
        return m_idToIndex.find(id) != m_idToIndex.end();
    }

    bool FlowNetwork::isDirected() const {
        return m_directed;
    }

    int FlowNetwork::size() const {
        return static_cast<int>(m_vertices.size());
    }

    std::vector<int> FlowNetwork::vertexIds() const {
        return m_vertices;
    }

    std::vector<FlowEdge> FlowNetwork::edges() const {
        return m_edges;
    }

    std::vector<ResidualArc> FlowNetwork::residualNeighbors(int
        vertex) const {
        std::vector<ResidualArc> residuals;

        // 1) forward residual edges:
        // if edge still has free capacity, we can push more flow
            forward
        auto outIt = m_outgoing.find(vertex);
        if (outIt != m_outgoing.end()) {
            for (int edgeIndex : outIt->second) {
                const FlowEdge& edge = m_edges[edgeIndex];

```

```

        const int residualCapacity = edge.capacity - edge.
            flow;
        if (residualCapacity > 0) {
            residuals.push_back({
                edge.from,
                edge.to,
                edgeIndex,
                true,
                residualCapacity,
                edge.cost
            });
        }
    }
}

// 2) backward residual edges:
// if some flow was already pushed, we may "take it back"
auto inIt = m_incoming.find(vertex);
if (inIt != m_incoming.end()) {
    for (int edgeIndex : inIt->second) {
        const FlowEdge& edge = m_edges[edgeIndex];
        if (edge.flow > 0) {
            residuals.push_back({
                edge.to,
                edge.from,
                edgeIndex,
                false,
                edge.flow,
                -edge.cost
            });
        }
    }
}

return residuals;
}

std::optional<FlowEdge> FlowNetwork::getEdge(int from, int to)
const {
    auto edgeIndex = findEdgeIndex(from, to);
    if (!edgeIndex.has_value()) {

```

```

        return std::nullopt;
    }

    // return a copy, not reference
    return m_edges[edgeIndex.value()];
}

std::optional<int> FlowNetwork::getCapacity(int from, int to)
    const {
    auto edge = getEdge(from, to);
    if (!edge.has_value()) {
        return std::nullopt;
    }
    return edge->capacity;
}

std::optional<int> FlowNetwork::getCost(int from, int to) const {
    auto edge = getEdge(from, to);
    if (!edge.has_value()) {
        return std::nullopt;
    }
    return edge->cost;
}

std::optional<int> FlowNetwork::getFlow(int from, int to) const {
    auto edge = getEdge(from, to);
    if (!edge.has_value()) {
        return std::nullopt;
    }
    return edge->flow;
}

FlowMatrix FlowNetwork::getCapacityMatrix() const {
    FlowMatrix matrix = createEmptyMatrix(size());

    // only real edges are written here
    for (const FlowEdge& edge : m_edges) {
        matrix[indexForVertex(edge.from)][indexForVertex(edge.to)] = edge.capacity;
    }
    return matrix;
}

```

```

}

FlowMatrix FlowNetwork::getCostMatrix() const {
    FlowMatrix matrix = createEmptyMatrix(size());

    for (const FlowEdge& edge : m_edges) {
        matrix[indexForVertex(edge.from)][indexForVertex(edge.to)] = edge.cost;
    }
    return matrix;
}

FlowMatrix FlowNetwork::getFlowMatrix() const {
    FlowMatrix matrix = createEmptyMatrix(size());

    for (const FlowEdge& edge : m_edges) {
        matrix[indexForVertex(edge.from)][indexForVertex(edge.to)] = edge.flow;
    }
    return matrix;
}

void FlowNetwork::resetFlows() {
    // keep graph / capacities / costs, only clear current flow state
    for (FlowEdge& edge : m_edges) {
        edge.flow = 0;
    }
}

void FlowNetwork::augment(int edgeIndex, bool forward, int delta)
{
    // safety check, just ignore bad index
    if (edgeIndex < 0 || edgeIndex >= static_cast<int>(m_edges.size())) {
        return;
    }

    // forward residual move => add flow
    // backward residual move => reduce previously pushed flow
    if (forward) {

```

```

        m_edges[edgeIndex].flow += delta;
    } else {
        m_edges[edgeIndex].flow -= delta;
    }
}

std::optional<int> FlowNetwork::findEdgeIndex(int from, int to)
const {
    auto outIt = m_outgoing.find(from);
    if (outIt == m_outgoing.end()) {
        return std::nullopt;
    }

    // scan only outgoing edges from "from", not all edges in
    // graph
    for (int edgeIndex : outIt->second) {
        const FlowEdge& edge = m_edges[edgeIndex];
        if (edge.to == to) {
            return edgeIndex;
        }
    }

    return std::nullopt;
}

int FlowNetwork::indexForVertex(int id) const {
    return m_idToIndex.at(id);
}

void printFlowMatrix(std::ostream& out, const FlowMatrix& matrix,
                    const std::vector<int>& vertexIds) {
    const size_t n = matrix.size();
    if (n == 0) {
        return;
    }

    out << "      ";
    for (int vertexId : vertexIds) {
        out << std::setw(10) << vertexId;
    }
    out << "\n";
}

```

```

out << "          ";
for (size_t i = 0; i < n; ++i) {
    out << "-----";
}
out << "\n";

for (size_t row = 0; row < n; ++row) {
    out << std::setw(5) << vertexIds[row] << " |";
    for (size_t col = 0; col < n; ++col) {
        if (matrix[row][col].has_value()) {
            out << std::setw(10) << matrix[row][col].value();
        } else {
            // same convention as in other parts of the
            // project
            out << std::setw(10) << "i";
        }
    }
    out << "\n";
}
}

```

Приложение П. Файл FlowNetworkBuilder.h

```
#pragma once

#include "include/Generator.h"
#include "include/FlowNetwork.h"
#include "include/Graph.h"

#include <memory>

// takes already generated graph and builds flow network from it
class FlowNetworkBuilder {
public:
    // graph gives us only structure (who is connected with whom)
    // capacities and costs are generated here separately from
    // first generation
    std::unique_ptr<FlowNetwork> buildFromGraph(const
        AdjacencyGraph& graph);

private:
    int generateRandomCapacity();
    int generateRandomCost();

    AcyclicGraphBuilder m_generator;
};
```


Приложение Р. Файл FlowNetworkBuilder.cpp

```
#include "include/FlowNetworkBuilder.h"

std::unique_ptr<FlowNetwork> FlowNetworkBuilder::buildFromGraph(
    const AdjacencyGraph& graph) {
    // flow network is always directed in this lab
    auto network = std::make_unique<FlowNetwork>(true);

    // copy vertex set as is
    for (int vertexId : graph.vertexIds()) {
        network->addVertex(vertexId);
    }

    // keep only topology from original graph
    // cap and cost are generated randomly here
    for (const WeightedEdge& edge : graph.edges()) {
        network->addEdge(edge.from, edge.to,
            generateRandomCapacity(), generateRandomCost());
    }

    return network;
}

int FlowNetworkBuilder::generateRandomCapacity() {
    return m_generator.sampleBinomial(BINOMIAL_N_WEIGHT,
        BINOMIAL_P_WEIGHT);
}

int FlowNetworkBuilder::generateRandomCost() {
    return m_generator.sampleBinomial(BINOMIAL_N_WEIGHT,
        BINOMIAL_P_WEIGHT);
}
```

Приложение С. Файл FundamentalCuts.h

```
#pragma once
#include "include/Graph.h"
#include <set>
#include <vector>

/// One fundamental cut: associated with one tree edge.
/// Removing the tree edge splits the spanning tree into two
    components
/// (`sideA` and `sideB`); the cut is every graph edge with
    exactly one
/// endpoint on each side. The tree edge itself is always in the
    cut.
struct FundamentalCut {
    int treeEdgeFrom;           // endpoints of the tree edge
                               that defines this cut
    int treeEdgeTo;
    std::set<int> sideA;        // vertex IDs on one side after
                               removing the tree edge
    std::set<int> sideB;        // vertex IDs on the other side
    std::vector<std::pair<int,int>> edges; // edges in the cut,
                               normalized so first < second
};

class FundamentalCutSystem {
public:
    /// `originalGraph` is the full (undirected) graph; `
        spanningTree` must be
    /// a spanning tree built from the same vertex set (e.g.
        Kruskal's MST).
    FundamentalCutSystem(const AdjacencyGraph& originalGraph,
                        const AdjacencyGraph& spanningTree);

    /// Build all fundamental cuts. There is exactly one per
        tree edge, so
    /// the result has size (n - 1) for an n-vertex connected
        graph.
    std::vector<FundamentalCut> compute() const;

    /// Symmetric difference of edge sets:
```

```
/// present in exactly one of `a`, `b`.
/// This is the operation that combines cuts in the cut-space
    basis.
static std::vector<std::pair<int,int>> symmetricDifference(
    const std::vector<std::pair<int,int>>& a,
    const std::vector<std::pair<int,int>>& b);

private:
    const AdjacencyGraph& m_graph;
    const AdjacencyGraph& m_tree;
};
```

Приложение Т. Файл FundamentalCuts.cpp

```
#include "include/FundamentalCuts.h"
#include <algorithm>
#include <unordered_set>

FundamentalCutSystem::FundamentalCutSystem(const AdjacencyGraph&
    originalGraph,
                                           const AdjacencyGraph&
                                           spanningTree)
    : m_graph(originalGraph)
    , m_tree(spanningTree)
{}

namespace {

// Normalize an edge so the smaller endpoint comes first. This
// lets us treat
// (u,v) and (v,u) as the same edge when comparing cuts.
std::pair<int,int> normEdge(int u, int v) {
    if (u > v) std::swap(u, v);
    return {u, v};
}

// BFS over the tree starting at `root`, but pretending the edge
// (forbiddenA, forbiddenB) does not exist. Returns the set of
// reached
// vertices -- this is one of the two components after removing
// that edge.
std::set<int> reachableInTreeWithoutEdge(const AdjacencyGraph&
    tree,
                                           int root,
                                           int forbiddenA,
                                           int forbiddenB) {

    std::set<int> visited;
    std::vector<int> queue{root};
    visited.insert(root);

    size_t head = 0;
    while (head < queue.size()) {
        int u = queue[head++];
```

```

        for (const auto& [v, _w] : tree.neighbors(u)) {
            // Skip the edge we are pretending to remove (in
            // either direction).
            const bool isForbidden =
                (u == forbiddenA && v == forbiddenB) ||
                (u == forbiddenB && v == forbiddenA);
            if (isForbidden) continue;
            if (visited.count(v)) continue;
            visited.insert(v);
            queue.push_back(v);
        }
    }
    return visited;
}

} // namespace

std::vector<FundamentalCut> FundamentalCutSystem::compute() const
{
    std::vector<FundamentalCut> cuts;

    // Walk every tree edge and build its fundamental cut.
    // AdjacencyGraph::edges() lists each undirected edge once.
    for (const WeightedEdge& te : m_tree.edges()) {
        FundamentalCut cut;
        cut.treeEdgeFrom = te.from;
        cut.treeEdgeTo = te.to;

        // The two sides of the partition: BFS from te.from in T
        // \ {te},
        // everyone else is on the other side.
        cut.sideA = reachableInTreeWithoutEdge(m_tree, te.from,
            te.from, te.to);
        for (int v : m_tree.vertexIds()) {
            if (cut.sideA.count(v) == 0) {
                cut.sideB.insert(v);
            }
        }
    }

    // Now scan all ORIGINAL graph edges; an edge is in the
    // cut iff its

```

```

        // two endpoints fall on opposite sides of the partition.
        for (const WeightedEdge& e : m_graph.edges()) {
            const bool aHasFrom = cut.sideA.count(e.from) > 0;
            const bool aHasTo   = cut.sideA.count(e.to)   > 0;
            if (aHasFrom != aHasTo) { // exactly one endpoint in
                sideA
                cut.edges.push_back(normEdge(e.from, e.to));
            }
        }
        // Sort for deterministic display and to make symmetric
        difference easier.
        std::sort(cut.edges.begin(), cut.edges.end());
        cuts.push_back(std::move(cut));
    }

    return cuts;
}

std::vector<std::pair<int,int>> FundamentalCutSystem::
symmetricDifference(
    const std::vector<std::pair<int,int>>& a,
    const std::vector<std::pair<int,int>>& b)
{
    // Both inputs are sorted (compute() sorts each cut), so the
    symmetric
    // difference is a single linear merge: keep elements that
    are in exactly
    // one of the two lists.
    std::vector<std::pair<int,int>> result;
    size_t i = 0;
    size_t j = 0;
    while (i < a.size() && j < b.size()) {
        if (a[i] == b[j]) {
            // present in both -> drop
            ++i;
            ++j;
        } else if (a[i] < b[j]) {
            result.push_back(a[i++]);
        } else {
            result.push_back(b[j++]);
        }
    }
}

```

```
    }  
    while (i < a.size()) result.push_back(a[i++]);  
    while (j < b.size()) result.push_back(b[j++]);  
    return result;  
}
```

Приложение У. Файл FundamentalCycles.h

```
#pragma once
#include "include/Graph.h"
#include <vector>

/// One fundamental cycle: associated with one non-tree edge ("
    chord").
/// In a spanning tree T of G, every edge of G that is NOT in T
    forms a
/// unique cycle when added to T -- the chord plus the unique
    tree path
/// connecting its two endpoints. That cycle is the fundamental
    cycle
/// of the chord.
struct FundamentalCycle {
    int chordFrom; // endpoints of the
        chord that defines this cycle
    int chordTo;
    std::vector<int> vertexSequence; // cycle as v0, v1,
        ..., vk = v0 (for display)
    std::vector<std::pair<int,int>> edges; // edges in the
        cycle, normalized so first < second; sorted
};

class FundamentalCycleSystem {
public:
    /// `originalGraph` is the full (undirected) graph; `
    spanningTree` must be
/// a spanning tree built from the same vertex set (e.g.
    Kruskal's MST).
    FundamentalCycleSystem(const AdjacencyGraph& originalGraph,
        const AdjacencyGraph& spanningTree);

    /// Build all fundamental cycles. There is exactly one per
    chord, so
/// the result has size m - n + 1 for a connected n-vertex, m
    -edge graph.
    std::vector<FundamentalCycle> compute() const;

    /// Symmetric difference of edge sets:
```



```

    /// present in exactly one of `a`, `b`.
    /// Combining fundamental cycles by symmetric difference
        gives every cycle of G
    /// (or a disjoint union of cycles, i.e. an "even subgraph").
    /// Both inputs are expected to be sorted (compute() returns
        sorted edge
    /// lists), and the output is sorted too.
    static std::vector<std::pair<int,int>> symmetricDifference(
        const std::vector<std::pair<int,int>>& a,
        const std::vector<std::pair<int,int>>& b);

    /// Decompose an undirected edge set from the cycle space
        into edge-disjoint
    /// simple cycles. If the set is itself one simple cycle,
        the result has
    /// exactly one vertex sequence  $v_0, v_1, \dots, v_k = v_0$ .
    static std::vector<std::vector<int>> decomposeIntoCycles(
        const std::vector<std::pair<int,int>>& edges);

private:
    const AdjacencyGraph& m_graph;
    const AdjacencyGraph& m_tree;
};

```

Приложение Ф. Файл FundamentalCycles.cpp

```
#include "include/FundamentalCycles.h"
#include <algorithm>
#include <functional>
#include <map>
#include <set>
#include <unordered_map>

FundamentalCycleSystem::FundamentalCycleSystem(const
    AdjacencyGraph& originalGraph,
                                                    const
                                                    AdjacencyGraph&
                                                    spanningTree)
    : m_graph(originalGraph)
    , m_tree(spanningTree)
{}

namespace {

// Normalize an edge so the smaller endpoint comes first. This
// lets us treat
// (u,v) and (v,u) as the same edge when comparing cycles.
std::pair<int,int> normEdge(int u, int v) {
    if (u > v) std::swap(u, v);
    return {u, v};
}

// Lexicographic order on normalized edges: first by the first
// endpoint, then
// by the second. Making this comparator explicit keeps the
// intended display
// order obvious at the call site.
bool edgeLess(const std::pair<int,int>& lhs, const std::pair<int,
int>& rhs) {
    if (lhs.first != rhs.first) return lhs.first < rhs.first;
    return lhs.second < rhs.second;
}

bool isSimpleCycleSequence(const std::vector<int>& cycle) {
```

```

    if (cycle.size() < 4) return false; // at least 3 edges +
        closing vertex
    if (cycle.front() != cycle.back()) return false;

    std::set<int> seen;
    for (size_t i = 0; i + 1 < cycle.size(); ++i) {
        if (!seen.insert(cycle[i]).second) {
            return false;
        }
    }
    return true;
}

std::vector<int> findAnySimpleCycle(
    const std::map<int, std::set<int>>& adjacency)
{
    std::set<int> visited;
    std::vector<int> stack;
    std::unordered_map<int, size_t> position;
    std::vector<int> cycle;

    std::function<bool(int, int)> dfs = [&](int u, int parent) {
        visited.insert(u);
        position[u] = stack.size();
        stack.push_back(u);

        auto it = adjacency.find(u);
        if (it != adjacency.end()) {
            for (int v : it->second) {
                if (v == parent) continue;

                if (position.count(v) > 0) {
                    cycle.assign(stack.begin() + static_cast<std::
                        ::ptrdiff_t>(position[v]),
                        stack.end());
                    cycle.push_back(v);
                    return true;
                }

                if (visited.count(v) == 0 && dfs(v, u)) {
                    return true;
                }
            }
        }
    };
}

```

```

        }
    }
}

position.erase(u);
stack.pop_back();
return false;
};

for (const auto& [start, neighbors] : adjacency) {
    if (neighbors.empty() || visited.count(start) > 0)
        continue;
    if (dfs(start, -1)) {
        return cycle;
    }
}

return {};
}

// In a tree there is exactly one simple path between any two
vertices.
// BFS from `start` until we hit `target`, recording where we
came from so
// we can reconstruct the path by walking predecessors backwards.
//
// Returns the path as a vertex sequence: [start, ..., target].
Empty if
// target is unreachable (which should not happen for a spanning
tree).
std::vector<int> findTreePath(const AdjacencyGraph& tree, int
start, int target) {
    std::unordered_map<int, int> parent; // child -> parent
    parent[start] = start; // sentinel: start is
        its own parent

    std::vector<int> queue{start};
    size_t head = 0;
    bool found = (start == target);

    while (head < queue.size() && !found) {

```

```

        int u = queue[head++];
        for (const auto& [v, _w] : tree.neighbors(u)) {
            if (parent.count(v)) continue; // already visited
            parent[v] = u;
            if (v == target) {
                found = true;
                break;
            }
            queue.push_back(v);
        }
    }

    if (!found) return {};

    // Walk back from target to start, then reverse.
    std::vector<int> path;
    int cur = target;
    while (cur != start) {
        path.push_back(cur);
        cur = parent[cur];
    }
    path.push_back(start);
    std::reverse(path.begin(), path.end());
    return path;
}

// Collect edges of the spanning tree as a set of normalized (u,v
// ) pairs.
// Used to decide which graph edges are tree edges vs chords.
std::set<std::pair<int,int>> collectTreeEdges(const
AdjacencyGraph& tree) {
    std::set<std::pair<int,int>> result;
    for (const WeightedEdge& e : tree.edges()) {
        result.insert(normEdge(e.from, e.to));
    }
    return result;
}

} // namespace

```

```

std::vector<FundamentalCycle> FundamentalCycleSystem::compute()
const {
    std::vector<FundamentalCycle> cycles;

    const auto treeEdgeSet = collectTreeEdges(m_tree);

    // Walk every graph edge. If it is NOT in the tree, it is a
    // chord, and
    // it defines exactly one fundamental cycle.
    for (const WeightedEdge& e : m_graph.edges()) {
        const auto key = normEdge(e.from, e.to);
        if (treeEdgeSet.count(key) > 0) {
            continue; // tree edge -- skip
        }

        // Chord: build its fundamental cycle.
        FundamentalCycle cycle;
        cycle.chordFrom = key.first;
        cycle.chordTo = key.second;

        // Tree path from one endpoint of the chord to the other.
        // Combined with the chord itself, this closes the cycle.
        std::vector<int> path = findTreePath(m_tree, e.from, e.to
        );
        if (path.empty()) continue; // disconnected tree -- no
            cycle possible

        // Vertex sequence for display: tree path + close back to
            start.
        cycle.vertexSequence = path;
        cycle.vertexSequence.push_back(path.front()); // close
            the loop

        // Edge set: tree-path edges + the chord itself.
        for (size_t i = 0; i + 1 < path.size(); ++i) {
            cycle.edges.push_back(normEdge(path[i], path[i + 1]))
            ;
        }
        cycle.edges.push_back(key); // the chord
    }
}

```

```

        // Sort for deterministic display and to make symmetric
        difference easier.
        std::sort(cycle.edges.begin(), cycle.edges.end(),
            edgeLess);
        cycles.push_back(std::move(cycle));
    }

    std::sort(cycles.begin(), cycles.end(), [](const
        FundamentalCycle& lhs,
                                const
                                FundamentalCycle
                                & rhs) {
        if (lhs.chordFrom != rhs.chordFrom) return lhs.chordFrom
            < rhs.chordFrom;
        return lhs.chordTo < rhs.chordTo;
    });

    return cycles;
}

std::vector<std::pair<int,int>> FundamentalCycleSystem::
symmetricDifference(
    const std::vector<std::pair<int,int>>& a,
    const std::vector<std::pair<int,int>>& b)
{
    // Both inputs are sorted (compute() sorts each cycle), so
    the symmetric
    // difference is a single linear merge: keep elements that
    are in exactly
    // one of the two lists.
    std::vector<std::pair<int,int>> result;
    size_t i = 0;
    size_t j = 0;
    while (i < a.size() && j < b.size()) {
        if (a[i] == b[j]) {
            // present in both -> drop
            ++i;
            ++j;
        } else if (a[i] < b[j]) {
            result.push_back(a[i++]);
        } else {

```

```

        result.push_back(b[j++]);
    }
}
while (i < a.size()) result.push_back(a[i++]);
while (j < b.size()) result.push_back(b[j++]);
return result;
}

std::vector<std::vector<int>> FundamentalCycleSystem::
decomposeIntoCycles(
    const std::vector<std::pair<int,int>>& edges)
{
    std::map<int, std::set<int>> adjacency;
    for (const auto& [u, v] : edges) {
        adjacency[u].insert(v);
        adjacency[v].insert(u);
    }

    std::vector<std::vector<int>> cycles;
    while (true) {
        bool hasEdges = false;
        for (const auto& [vertex, neighbors] : adjacency) {
            if (!neighbors.empty()) {
                hasEdges = true;
                break;
            }
        }
        if (!hasEdges) break;

        std::vector<int> cycle = findAnySimpleCycle(adjacency);
        if (cycle.empty()) {
            cycles.clear();
            return cycles;
        }

        for (size_t i = 0; i + 1 < cycle.size(); ++i) {
            const int u = cycle[i];
            const int v = cycle[i + 1];
            adjacency[u].erase(v);
            adjacency[v].erase(u);
        }
    }
}

```



```
        if (isSimpleCycleSequence(cycle)) {
            cycles.push_back(std::move(cycle));
        } else {
            cycles.clear();
            return cycles;
        }
    }

    return cycles;
}
```

Приложение X. Файл Generator.h

```
#pragma once
#include "include/Graph.h"
#include <random>

constexpr int BINOMIAL_N_DEGREE = 10;
constexpr double BINOMIAL_P_DEGREE = 0.5;

constexpr int BINOMIAL_N_WEIGHT = 10;
constexpr double BINOMIAL_P_WEIGHT = 0.5;

/*Я всё ещё путаюсь
, что это вообще такое P : поэтому это не шаргалка .
k=0:  || (2.8%)
k=1:  ||||| (12.1%)
k=2:  ||||| (23.3%)
k=3:  ||||| (26.7%)
k=4:  ||||| (20.0%)
k=5:  ||||| (10.3%)
k=6:  |||| (3.7%)
k=7:  || (0.9%)
k=8:  | (0.1%)
k=9:  (0.01%)
k=10: (0.0006%)
*/

struct BinomialProperties {
    double mean;
    double variance;
    double stdDev;
    int mode;
    double skewness;
    double kurtosis;
};

struct DegreeStatistics {
    double meanDegree;
    double variance;
    double stdDev;
    int minDegree;
```

```

    int maxDegree;
};

enum class WeightSign {
    Positive,
    Negative,
    Mixed
};

class AcyclicGraphBuilder {
public:
    std::unique_ptr<AdjacencyGraph> generateAcyclicGraph(
        int vertices,
        bool directed,
        WeightSign weightSign = WeightSign::Positive);

    static BinomialProperties getBinomialProperties(int n, double
        p);

    static DegreeStatistics computeDegreeStatistics(const
        AdjacencyGraph& graph);

    int sampleBinomial(int n, double p);

private:
    std::mt19937 m_rng{std::random_device{}()};

    double sampleWeight(WeightSign weightSign);
};

```

Приложение Ц. Файл Generator.cpp

```
#include "include/Generator.h"
#include <cmath>
#include <random>
#include <set>
#include <numeric>
#include <algorithm>
#include <vector>
#include <iostream>

BinomialProperties AcyclicGraphBuilder::getBinomialProperties(int
    n, double p) {
    double q = 1.0 - p;
    double np1 = (n + 1) * p;

    BinomialProperties props{};
    props.mean = n * p;
    props.variance = n * p * q;
    props.stdDev = std::sqrt(props.variance);

    props.mode = static_cast<int>(np1);
    props.skewness = (q - p) / std::sqrt(n * p * q);
    props.kurtosis = (1.0 - 6.0 * p * q) / (n * p * q);

    return props;
}
// ребраневводимвинпут ,
они генерируются на основе степеней автоматически

DegreeStatistics AcyclicGraphBuilder::computeDegreeStatistics(
    const AdjacencyGraph& graph) {
    DegreeStatistics stats{};
    auto vertices = graph.vertexIds();
    int vertexCount = static_cast<int>(vertices.size());

    if (vertexCount == 0) {
        return stats;
    }

    std::vector<int> degreeList;
```

```

degreeList.reserve(vertexCount);

for (int v : vertices) {
    degreeList.push_back(static_cast<int>(graph.neighbors(v).
        size()));
}

double total = std::accumulate(degreeList.begin(), degreeList
    .end(), 0.0);
stats.meanDegree = total / vertexCount;

double sqDiffSum = 0.0;
for (int deg : degreeList) {
    sqDiffSum += (deg - stats.meanDegree) * (deg - stats.
        meanDegree);
}
stats.variance = sqDiffSum / vertexCount;
stats.stdDev = std::sqrt(stats.variance);

auto result = std::minmax_element(degreeList.begin(),
    degreeList.end());
stats.minDegree = *result.first;
stats.maxDegree = *result.second;

return stats;
}

int AcyclicGraphBuilder::sampleBinomial(int n, double p) {
    double q = 1.0 - p;
    double ratio = p / q;
    double probability = std::pow(q, n);

    std::uniform_real_distribution<double> uniform(0.0, 1.0);
    double randomVal = uniform(m_rng);
    int x = 0;

    while (true) {
        randomVal -= probability;
        if (randomVal < 0.0) {
            return x;
        }
    }
}

```

```

        ++x;
        probability *= ratio * (n + 1 - x) / x;
    }
}

namespace {
double makeNonZeroWeight(int baseWeight, WeightSign weightSign,
    std::mt19937& rng) {
    // Edge weights in the lab must never be zero.
    // If the binomial sample produced 0, shift it to 1 before
    applying sign.
    if (baseWeight == 0) {
        baseWeight = 1;
    }

    switch (weightSign) {
        case WeightSign::Positive:
            return static_cast<double>(baseWeight);
        case WeightSign::Negative:
            return -static_cast<double>(baseWeight);
        case WeightSign::Mixed: {
            std::uniform_int_distribution<int> coinFlip(0, 1);
            return coinFlip(rng)
                ? static_cast<double>(baseWeight)
                : -static_cast<double>(baseWeight);
        }
    }

    return static_cast<double>(baseWeight);
}

double AcyclicGraphBuilder::sampleWeight(WeightSign weightSign) {
    int baseWeight = sampleBinomial(BINOMIAL_N_WEIGHT,
        BINOMIAL_P_WEIGHT);
    return makeNonZeroWeight(baseWeight, weightSign, m_rng);
}

// Check if graph is connected using BFS

```

```

// For directed graphs: use WEAK connectivity (ignore edge
    direction)
// For undirected: standard connectivity
static bool checkConnected(const AdjacencyGraph& graph, int
    vertexCount, bool directed) {
    if (vertexCount <= 1) return true;

    std::vector<bool> visited(vertexCount, false);
    std::vector<int> queue;
    int visitedCount = 0;

    queue.push_back(0);
    visited[0] = true;
    visitedCount++;

    size_t head = 0;
    while (head < queue.size()) {
        int current = queue[head++];

        // For directed graphs, we need to check both outgoing
            AND incoming edges
        // (weak connectivity - treat as undirected)
        for (int other = 0; other < vertexCount; ++other) {
            if (visited[other]) continue;

            // Check if there's an edge either direction
            bool hasEdge = false;

            // Check outgoing from current
            for (const auto& [neighbor, weight] : graph.neighbors
                (current)) {
                if (neighbor == other) {
                    hasEdge = true;
                    break;
                }
            }

            // Check incoming to current (for directed graphs)
            if (!hasEdge && directed) {
                for (const auto& [neighbor, weight] : graph.
                    neighbors(other)) {

```

```

        if (neighbor == current) {
            hasEdge = true;
            break;
        }
    }

    if (hasEdge) {
        visited[other] = true;
        visitedCount++;
        queue.push_back(other);
    }
}

return visitedCount == vertexCount;
}

// Try to build graph with given degree targets
static std::unique_ptr<AdjacencyGraph> tryBuildGraph(
    int vertexCount,
    bool directed,
    WeightSign weightSign,
    const std::vector<int>& degreeTargets,
    std::mt19937& rng,
    AcyclicGraphBuilder& builder)
{
    auto graph = std::make_unique<AdjacencyGraph>(directed);

    for (int i = 0; i < vertexCount; ++i) {
        graph->addVertex(i);
    }

    std::vector<int> currentDegree(vertexCount, 0);
    std::set<std::pair<int, int>> addedEdges;

    int edgesAdded = 0;

    // Calculate target edges from degree targets (sum of degrees
    // / 2 for undirected)
    int sumDegrees = 0;

```



```

for (int deg : degreeTargets) {
    sumDegrees += deg;
}
// For undirected: edges = sum(degrees) / 2
// For directed: edges = sum(degrees) (each edge contributes
// to out-degree only)
int targetEdges = directed ? sumDegrees : sumDegrees / 2;

// Calculate max possible edges for acyclic graph (u < v
// constraint)
int maxPossibleEdges = vertexCount * (vertexCount - 1) / 2;
int actualTargetEdges = std::min(targetEdges,
    maxPossibleEdges);

// Base attempts on actual possible edges, not requested
// edges
int attempts = 0;
const int maxAttempts = actualTargetEdges * 100 + 1000;

while (edgesAdded < actualTargetEdges && attempts <
    maxAttempts) {
    ++attempts;

    int u = std::uniform_int_distribution<>(0, vertexCount -
        2)(rng);
    int v = std::uniform_int_distribution<>(u + 1,
        vertexCount - 1)(rng);

    if (addedEdges.count({u, v})) continue;

    // Only check degree targets if they were set (> 0)
    // But don't let them block us if we still need more
    // edges
    bool uBlocked = (degreeTargets[u] > 0 && currentDegree[u]
        >= degreeTargets[u]);
    bool vBlocked = (degreeTargets[v] > 0 && currentDegree[v]
        >= degreeTargets[v]);

    // Skip only if BOTH vertices are at their degree limit
    // This allows flexibility to reach targetEdges
    if (uBlocked && vBlocked) continue;

```

```

        // Generate a non-zero weight using the same binomial
        base.
        int baseWeight = builder.sampleBinomial(BINOMIAL_N_WEIGHT
            , BINOMIAL_P_WEIGHT);
        double weight = makeNonZeroWeight(baseWeight, weightSign,
            rng);

        graph->addEdge(u, v, weight);
        addedEdges.insert({u, v});
        ++currentDegree[u];
        ++currentDegree[v];
        ++edgesAdded;
    }

    if (edgesAdded < actualTargetEdges) {
        std::cout << "[!] Warning: Could only add " << edgesAdded
            << " edges (max possible for " << vertexCount
            << " vertices: " << maxPossibleEdges << ")\n";
    }

    return graph;
}

std::unique_ptr<AdjacencyGraph> AcyclicGraphBuilder::
generateAcyclicGraph(
    int vertexCount,
    bool directed,
    WeightSign weightSign)
{
    const int maxRegenerations = 50;

    for (int attempt = 1; attempt <= maxRegenerations; ++attempt)
    {
        // Step 1: Generate ALL vertex degrees using binomial
        distribution
        std::vector<int> degreeTargets(vertexCount);
        int totalDegree = 0;

        for (int i = 0; i < vertexCount; ++i) {
            degreeTargets[i] = std::min(

```

```

        sampleBinomial(BINOMIAL_N_DEGREE,
                        BINOMIAL_P_DEGREE),
        vertexCount - 1
    );
    totalDegree += degreeTargets[i];
}

// Fix parity for undirected graphs (handshaking Lemma)
if (!directed && totalDegree % 2 != 0) {
    for (int i = 0; i < vertexCount && totalDegree % 2 !=
        0; ++i) {
        if (degreeTargets[i] > 0) {
            --degreeTargets[i];
            --totalDegree;
        }
    }
}

// Step 2: Build graph based on degree targets
auto graph = tryBuildGraph(vertexCount, directed,
    weightSign,
                                degreeTargets, m_rng, *this);

// Step 3: Check if connected (weak connectivity for
// directed)
if (checkConnected(*graph, vertexCount, directed)) {
    // Success! Return connected graph
    return graph;
}

// Step 4: Not connected → discard EVERYTHING and
// regenerate
// (both degrees AND edges)
// Loop continues to next attempt
}

// After max attempts, return last attempt
std::cout << "[!] Warning: Graph may be disconnected after "
    << maxRegenerations << " regeneration attempts\n";
std::cout << "    Try: more edges, or different binomial
    parameters (n, p)\n";

```

```

// One final attempt
std::vector<int> degreeTargets(vertexCount);
for (int i = 0; i < vertexCount; ++i) {
    degreeTargets[i] = std::min(
        sampleBinomial(BINOMIAL_N_DEGREE, BINOMIAL_P_DEGREE),
        vertexCount - 1
    );
}

return tryBuildGraph(vertexCount, directed, weightSign,
    degreeTargets, m_rng, *this);
}

```

Приложение Ч. Файл Graph.h

```
#pragma once
#include <vector>
#include <unordered_map>
#include <optional>
#include <memory>

/// Represents a weighted edge
struct WeightedEdge {
    int from;
    int to;
    double weight;
};

/// Adjacency matrix type (true = edge exists, false = no edge)
using AdjacencyMatrix = std::vector<std::vector<bool>>>;

class AdjacencyGraph {
public:
    explicit AdjacencyGraph(bool directed = true);
    void addVertex(int id);
    void addEdge(int from, int to, double weight); // Checks
        existence FtF
    std::optional<double> removeEdge(int from, int to);
    std::vector<int> vertexIds() const;
    std::vector<WeightedEdge> edges() const;
    std::vector<std::pair<int, double>> neighbors(int v) const;
    std::optional<double> getEdgeWeight(int from, int to) const;
    bool hasVertex(int id) const; // FtF
    bool isDirected() const;
    int size() const;

    AdjacencyMatrix getAdjacencyMatrix() const;

private:
    bool m_directed;
    std::vector<int> m_vertices;
    std::unordered_map<int, std::vector<std::pair<int, double>>>
        m_adj;
};
```

Приложение Ш. Файл Graph.cpp

```
#include "include/Graph.h"
#include <algorithm>

AdjacencyGraph::AdjacencyGraph(bool directed) : m_directed(
    directed) {}

void AdjacencyGraph::addVertex(int id) {
    if (std::find(m_vertices.begin(), m_vertices.end(), id) ==
        m_vertices.end()) {
        m_vertices.push_back(id);
    }
}

void AdjacencyGraph::addEdge(int from, int to, double weight) {
    if (weight == 0.0) {
        weight = 1.0;
    }

    addVertex(from);
    addVertex(to);
    m_adj[from].push_back({to, weight});
    if (!m_directed) {
        m_adj[to].push_back({from, weight});
    }
}

std::optional<double> AdjacencyGraph::removeEdge(int from, int to
) {
    auto fromIt = m_adj.find(from);
    if (fromIt == m_adj.end()) return std::nullopt;

    auto edgeIt = std::find_if(
        fromIt->second.begin(),
        fromIt->second.end(),
        [to](const std::pair<int, double>& edge) { return edge.
            first == to; });
    if (edgeIt == fromIt->second.end()) return std::nullopt;

    const double removedWeight = edgeIt->second;
```

```

        fromIt->second.erase(edgeIt);

        if (!m_directed) {
            auto toIt = m_adj.find(to);
            if (toIt != m_adj.end()) {
                auto backIt = std::find_if(
                    toIt->second.begin(),
                    toIt->second.end(),
                    [from, removedWeight](const std::pair<int, double> & edge) {
                        return edge.first == from && edge.second ==
                            removedWeight;
                    });
                if (backIt != toIt->second.end()) {
                    toIt->second.erase(backIt);
                }
            }
        }

        return removedWeight;
    }

    std::vector<int> AdjacencyGraph::vertexIds() const {
        return m_vertices;
    }

    int AdjacencyGraph::size() const {
        return static_cast<int>(m_vertices.size());
    }

    bool AdjacencyGraph::isDirected() const {
        return m_directed;
    }

    bool AdjacencyGraph::hasVertex(int id) const {
        return std::find(m_vertices.begin(), m_vertices.end(), id) !=
            m_vertices.end();
    }

    std::vector<WeightedEdge> AdjacencyGraph::edges() const {
        std::vector<WeightedEdge> edgeList;

```

```

    for (int u : m_vertices) {
        auto it = m_adj.find(u);
        if (it != m_adj.end()) {
            for (auto const& [v, w] : it->second) {
                if (m_directed || u < v) {
                    edgeList.push_back({u, v, w});
                }
            }
        }
    }
    return edgeList;
}

std::vector<std::pair<int, double>> AdjacencyGraph::neighbors(int
v) const {
    if (m_adj.find(v) == m_adj.end()) return {};
    return m_adj.at(v);
}

std::optional<double> AdjacencyGraph::getEdgeWeight(int from, int
to) const {
    if (m_adj.find(from) == m_adj.end()) return std::nullopt;
    for (auto const& [v, w] : m_adj.at(from)) {
        if (v == to) return w;
    }
    return std::nullopt;
}

AdjacencyMatrix AdjacencyGraph::getAdjacencyMatrix() const {
    // initialize with false (no edges)
    AdjacencyMatrix matrix(m_vertices.size(), std::vector<bool>(
        m_vertices.size(), false));

    // map vertex ID to index (FTF)
    std::unordered_map<int, size_t> idToIndex;
    for (size_t i = 0; i < m_vertices.size(); ++i) {
        idToIndex[m_vertices[i]] = i;
    }

    // Fill edges (1 = edge exists)
    // diagonal stays 0

```



```
for (const auto& edge : edges()) {
    size_t row = idToIndex.at(edge.from);
    size_t col = idToIndex.at(edge.to);
    matrix[row][col] = true;

    // Undirected: symmetric
    if (!m_directed) {
        matrix[col][row] = true;
    }
}

return matrix;
}
```

Приложение Щ. Файл Kirchhoff.h

```
#pragma once
#include "include/Graph.h"
#include <vector>

/// Result of Kirchhoff's matrix-tree theorem
struct KirchhoffResult {
    std::vector<std::vector<int>> kirchhoffMatrix; // B(G) --
        the Kirchhoff (Laplacian) matrix
    long long spanningTreeCount; // number of
        spanning trees
};

class KirchhoffCounter {
public:
    explicit KirchhoffCounter(const AdjacencyGraph& graph);

    /// Build Kirchhoff matrix and count spanning trees via the
        matrix-tree theorem.
    /// The graph is treated as undirected; weights are ignored (
        we count trees, not weight them).
    KirchhoffResult compute() const;

private:
    const AdjacencyGraph& m_graph;

    /// Build the Kirchhoff (Laplacian) matrix B = D - A,
        /// where D is the diagonal degree matrix and A is the
        adjacency matrix.
    std::vector<std::vector<int>> buildKirchhoffMatrix() const;

    /// Compute determinant of the (n-1) x (n-1) minor obtained
        by deleting
    /// the first row and the first column. By the matrix-tree
        theorem, this
    /// determinant equals the number of spanning trees.
    /// Uses Gaussian elimination with partial pivoting for
        numerical stability,
    /// then rounds the result (it must be an integer for an
        integer Laplacian).
```

```
long long determinantOfFirstMinor(const std::vector<std::  
    vector<int>>& matrix) const;  
};
```

Приложение Ъ. Файл Kirchhoff.cpp

```
#include "include/Kirchhoff.h"
#include <algorithm>
#include <cmath>
#include <unordered_map>
#include <vector>

KirchhoffCounter::KirchhoffCounter(const AdjacencyGraph& graph)
    : m_graph(graph)
{}

KirchhoffResult KirchhoffCounter::compute() const {
    KirchhoffResult result;
    result.kirchhoffMatrix = buildKirchhoffMatrix();

    const int n = m_graph.size();

    // Trivial cases:
    // n == 0 -- empty graph, no spanning trees
    // n == 1 -- single vertex, one (trivial) spanning tree
    if (n == 0) {
        result.spanningTreeCount = 0;
        return result;
    }
    if (n == 1) {
        result.spanningTreeCount = 1;
        return result;
    }

    // By the matrix-tree theorem: number of spanning trees =
    // any cofactor of the Kirchhoff matrix. We pick the (1,1)
    // cofactor,
    // which is just det(minor obtained by removing row 0 and
    // column 0).
    result.spanningTreeCount = determinantOfFirstMinor(result.
        kirchhoffMatrix);
    return result;
}
```

```

std::vector<std::vector<int>> KirchhoffCounter::
buildKirchhoffMatrix() const {
    const int n = m_graph.size();
    std::vector<std::vector<int>> B(n, std::vector<int>(n, 0));

    if (n == 0) {
        return B;
    }

    // Map vertex IDs to row/column indices.
    auto vertexIds = m_graph.vertexIds();
    std::unordered_map<int, int> idToIndex;
    for (int i = 0; i < n; ++i) {
        idToIndex[vertexIds[i]] = i;
    }

    // We treat the graph as undirected: every edge (u, v)
    // contributes
    //   B[u][u] += 1, B[v][v] += 1
    //   B[u][v] -= 1, B[v][u] -= 1
    // AdjacencyGraph::edges() already returns each undirected
    // edge once
    // (filtered by u < v internally), so a single pass over
    // edges() is enough.
    //
    // For directed input, edges() returns each arc once; the
    // user is expected
    // to pass an undirected graph for this algorithm. We still
    // symmetrize the
    // contribution so that the Laplacian is well-defined either
    // way.
    for (const auto& edge : m_graph.edges()) {
        const int i = idToIndex.at(edge.from);
        const int j = idToIndex.at(edge.to);
        if (i == j) {
            continue; // ignore self-loops, they don't affect
                       // tree counts
        }
        B[i][i] += 1;
        B[j][j] += 1;
        B[i][j] -= 1;
    }
}

```

```

        B[j][i] -= 1;
    }

    return B;
}

long long KirchhoffCounter::determinantOfFirstMinor(
    const std::vector<std::vector<int>>& matrix) const
{
    const int n = static_cast<int>(matrix.size());
    if (n <= 1) {
        return 0; // shouldn't happen, handled by caller, but be safe
    }

    // Build the (n-1) x (n-1) minor in double precision (drop row 0 and column 0).
    const int m = n - 1;
    std::vector<std::vector<double>> M(m, std::vector<double>(m, 0.0));
    for (int i = 0; i < m; ++i) {
        for (int j = 0; j < m; ++j) {
            M[i][j] = static_cast<double>(matrix[i + 1][j + 1]);
        }
    }

    // Gaussian elimination with partial pivoting.
    // Track sign flips from row swaps so we can recover det's sign at the end.
    double det = 1.0;
    int signFlips = 0;

    for (int col = 0; col < m; ++col) {
        // Find pivot row -- the row at or below `col` with the largest |M[row][col]|.
        int pivot = col;
        double bestAbs = std::fabs(M[col][col]);
        for (int row = col + 1; row < m; ++row) {
            const double currentAbs = std::fabs(M[row][col]);
            if (currentAbs > bestAbs) {
                bestAbs = currentAbs;
            }
        }
        if (pivot != col) {
            std::swap(M[col], M[pivot]);
            signFlips++;
        }
        double invPivot = 1.0 / M[col][col];
        M[col][col] = 1.0;
        for (int row = col + 1; row < m; ++row) {
            double factor = M[row][col];
            for (int k = col + 1; k < m; ++k) {
                M[row][k] -= factor * M[col][k];
            }
        }
    }

    if (signFlips % 2 == 1) {
        det = -det;
    }
    return det;
}

```

```

        pivot = row;
    }
}

// If the pivot is effectively zero, the matrix is
// singular -> det = 0
// (this happens, for example, when the graph is
// disconnected).
if (bestAbs < 1e-12) {
    return 0;
}

// Swap rows if needed and remember the sign flip.
if (pivot != col) {
    std::swap(M[col], M[pivot]);
    ++signFlips;
}

// Multiply running determinant by the pivot.
det *= M[col][col];

// Eliminate below: M[row] -= factor * M[col] for row >
// col.
const double pivotValue = M[col][col];
for (int row = col + 1; row < m; ++row) {
    const double factor = M[row][col] / pivotValue;
    if (factor == 0.0) {
        continue;
    }
    // Start from `col` since columns < col are already
    // 0.
    for (int k = col; k < m; ++k) {
        M[row][k] -= factor * M[col][k];
    }
}
}

if (signFlips % 2 != 0) {
    det = -det;
}

```

```
// The result must be a non-negative integer (number of  
spanning trees).  
// Round and clamp to handle floating-point noise.  
const double rounded = std::round(det);  
if (rounded < 0.0) {  
    return 0;  
}  
return static_cast<long long>(rounded);  
}
```


Приложение Ы. Файл Kruskal.h

```
#pragma once
#include "include/Graph.h"
#include <memory>
#include <vector>

/// Result of Kruskal's algorithm
struct KruskalResult {
    std::unique_ptr<AdjacencyGraph> spanningTree; // MST as an
        undirected graph
    std::vector<WeightedEdge> chosenEdges; // edges in
        the order they were added
    double totalWeight; // sum of
        weights of chosen edges
    bool wasConnected; // false if
        input graph was disconnected

        // (then we
        return a
        spanning
        forest, not
        a tree)
};

class KruskalMST {
public:
    explicit KruskalMST(const AdjacencyGraph& graph);

    /// Build a minimum spanning tree using Kruskal's algorithm.
    /// The graph is treated as undirected. Edges are sorted by
    /// ascending weight;
    /// ties are broken deterministically by (from, to) to make
    /// output reproducible.
    KruskalResult buildMST() const;

private:
    const AdjacencyGraph& m_graph;
};
```

Приложение Б. Файл Kruskal.cpp

```
#include "include/Kruskal.h"
#include <algorithm>
#include <unordered_map>

namespace {

// Disjoint-Set Union with union-by-rank and path compression.
// Operations are amortized  $O(\alpha(n))$ , effectively constant.
// Used by Kruskal to detect whether adding an edge would create
// a cycle.
class DisjointSetUnion {
public:
    explicit DisjointSetUnion(int size)
        : m_parent(size)
        , m_rank(size, 0)
    {
        // Initially every element is its own parent (n singleton
        // sets).
        for (int i = 0; i < size; ++i) {
            m_parent[i] = i;
        }
    }

    // Find the representative of the set containing x, with path
    // compression.
    int find(int x) {
        // Iterative path compression: first walk up to the root,
        // then make every node on the path point directly to it.
        int root = x;
        while (m_parent[root] != root) {
            root = m_parent[root];
        }
        while (m_parent[x] != root) {
            const int next = m_parent[x];
            m_parent[x] = root;
            x = next;
        }
        return root;
    }
};
```

```

    // Union the sets containing x and y. Returns true if they
    // were in
    // different sets (i.e. the union actually happened), false
    // otherwise.
    // Uses union by rank to keep the tree shallow.
    bool unite(int x, int y) {
        int rx = find(x);
        int ry = find(y);
        if (rx == ry) {
            return false; // already in the same set -- adding
                           // edge would create a cycle
        }

        // Attach the shorter tree under the taller one.
        if (m_rank[rx] < m_rank[ry]) {
            m_parent[rx] = ry;
        } else if (m_rank[rx] > m_rank[ry]) {
            m_parent[ry] = rx;
        } else {
            m_parent[ry] = rx;
            ++m_rank[rx];
        }
        return true;
    }

private:
    std::vector<int> m_parent;
    std::vector<int> m_rank;
};

} // namespace

KruskalMST::KruskalMST(const AdjacencyGraph& graph)
    : m_graph(graph)
{}

KruskalResult KruskalMST::buildMST() const {
    KruskalResult result;
    result.totalWeight = 0.0;
    result.wasConnected = true;

```

```

////////// T :=  $\mathbb{R}$ 

const int n = m_graph.size();

// Output tree inherits directedness from the input graph.
// For Lab 4 the user always passes an undirected graph, so
  this
// is effectively always undirected.
result.spanningTree = std::make_unique<AdjacencyGraph>(
  m_graph.isDirected());

// Always add all vertices, even if some end up isolated (
  disconnected case).
for (int v : m_graph.vertexIds()) {
    result.spanningTree->addVertex(v);
}

if (n <= 1) {
    // Empty graph or single vertex: nothing to connect.
    return result;
}

// Map vertex IDs to dense [0..n) indices for the DSU.
auto vertexIds = m_graph.vertexIds();
std::unordered_map<int, int> idToIndex;
for (int i = 0; i < n; ++i) {
    idToIndex[vertexIds[i]] = i;
}

// Collect all edges. AdjacencyGraph::edges() already yields
  each
// undirected edge exactly once (filtered by u < v internally
  ).
std::vector<WeightedEdge> allEdges = m_graph.edges();

// Sort by ascending weight. Tie-break by (from, to) for
  deterministic output:
// running Kruskal twice on the same graph should produce the
  same MST.
std::sort(allEdges.begin(), allEdges.end(),
  [](const WeightedEdge& a, const WeightedEdge& b) {

```

```

        if (a.weight != b.weight) {
            return a.weight < b.weight;
        }
        if (a.from != b.from) {
            return a.from < b.from;
        }
        return a.to < b.to;
    });
////////// Вход: список E рёбер графа G с длинами,
упорядоченный в порядке возрастания длин

DisjointSetUnion dsu(n);
int edgesAdded = 0;
const int targetEdgeCount = n - 1;

int k = 0; // индекс массива allEdges (0-based вместо 1-based)
////////// k := 1 { номер рассматриваемого ребра }

// for i from 1 to p - 1 do
// добавляем( ровно n-1 ребро для остовного дерева )
while (edgesAdded < targetEdgeCount && k < static_cast<int>(
    allEdges.size())) {
    //////////// for i from 1 to p - 1 do

    // while z(T + E[k]) > 0 do
    // проверяем(, образует ли добавление ребра E[k] цикл)
    // z() > 0 означает, что ребро создаёт цикл,
    // поэтому пропускаем его
    while (k < static_cast<int>(allEdges.size())) {
        const WeightedEdge& edge = allEdges[k];
        const int i = idToIndex.at(edge.from);
        const int j = idToIndex.at(edge.to);

        // dsu.unite() возвращает false,
        // если вершины уже в одном множестве
        // т.е. z(T + E[k]) > 0 - добавление создаст цикл )
        if (dsu.unite(i, j)) {
            break; // цикл не образуется, можно добавить ребро
        }
    }
}

```

```

        ++k;
        ////////////  $k := k + 1$  { пропустить это ребро }
    }
    //////////// while  $z(T + E[k]) > 0$  do

    // Если достигли конца списка рёбер, но не набрали  $n-1$  ребро
    if (k >= static_cast<int>(allEdges.size())) {
        break;
    }

    const WeightedEdge& chosenEdge = allEdges[k];
    result.spanningTree->addEdge(chosenEdge.from, chosenEdge.to, chosenEdge.weight);
    result.chosenEdges.push_back(chosenEdge);
    result.totalWeight += chosenEdge.weight;
    ++edgesAdded;
    ////////////  $T := T + E[k]$  { добавить это ребро } SST }

    ++k;
    ////////////  $k := k + 1$  { исключить его из рассмотрения }
}
////////// end for

// If we added fewer than  $n-1$  edges, the input graph was
// disconnected
// and what we produced is a spanning forest, not a spanning
// tree.
if (edgesAdded < targetEdgeCount) {
    result.wasConnected = false;
}

return result;
////////// Выход: множество  $T$  рёбер кратчайшего остова
}

```

Приложение Э. Файл MaxFlowSolver.h

```
#pragma once

#include "include/FlowNetwork.h"

#include <unordered_map>
#include <vector>

// one augmentation step from Ford-Fulkerson
struct MaxFlowStep {
    int iteration; // step number
    std::vector<int> path; // augmenting path by vertices
    int pathFlow; // how much was added through this path
    int totalFlow; // total flow after this step
};

struct MaxFlowResult {
    int maxFlow; // final answer
    std::vector<MaxFlowStep> steps; // useful for printing /
    checking
};

class MaxFlowSolver {
public:
    explicit MaxFlowSolver(FlowNetwork& network);

    // source = where flow starts
    // sink = where flow must end
    MaxFlowResult compute(int source, int sink);

private:
    // search one augmenting path in residual network
    bool bfs(int source, int sink, std::unordered_map<int,
        ResidualArc>& parent) const;

    FlowNetwork& m_network;
};
```

Приложение Ю. Файл MaxFlowSolver.cpp

```
#include "include/MaxFlowSolver.h"

#include <algorithm>
#include <limits>
#include <queue>
#include <unordered_set>

namespace {
// go from sink back to source using "parent" links from bfs
std::vector<ResidualArc> reconstructResidualPath(
    int source,
    int sink,
    const std::unordered_map<int, ResidualArc>& parent)
{
    std::vector<ResidualArc> path;
    int current = sink;

    while (current != source) {
        auto it = parent.find(current);
        if (it == parent.end()) {
            return {};
        }
        path.push_back(it->second);
        current = it->second.from;
    }

    std::reverse(path.begin(), path.end());
    return path;
}

// prettier version of path for output: only vertex ids
std::vector<int> toVertexPath(int source, const std::vector<
ResidualArc>& residualPath) {
    std::vector<int> path;
    path.push_back(source);

    for (const ResidualArc& arc : residualPath) {
        path.push_back(arc.to);
    }
}
```



```

    return path;
}
}

MaxFlowSolver::MaxFlowSolver(FlowNetwork& network)
    : m_network(network) {}

MaxFlowResult MaxFlowSolver::compute(int source, int sink) {
    MaxFlowResult result{0, {}};

    // no sense in running if source/sink are bad
    if (source == sink ||
        !m_network.hasVertex(source) ||
        !m_network.hasVertex(sink)) {
        return result;
    }

    // classic start: zero flow everywhere
    m_network.resetFlows();

    std::unordered_map<int, ResidualArc> parent;
    int iteration = 1;

    // while there exists augmenting path, we can increase flow
    while (bfs(source, sink, parent)) {
        std::vector<ResidualArc> residualPath =
            reconstructResidualPath(source, sink, parent);
        if (residualPath.empty()) {
            break;
        }

        // bottleneck = minimum residual capacity on this path
        int bottleneck = std::numeric_limits<int>::max();
        for (const ResidualArc& arc : residualPath) {
            bottleneck = std::min(bottleneck, arc.
                residualCapacity);
        }

        // apply augmentation to real edges
        for (const ResidualArc& arc : residualPath) {

```

```

        m_network.augment(arc.edgeIndex, arc.forward,
                           bottleneck);
    }

    // remember answer and step info
    result.maxFlow += bottleneck;
    result.steps.push_back({
        iteration++,
        toVertexPath(source, residualPath),
        bottleneck,
        result.maxFlow
    });
}

return result;
}

bool MaxFlowSolver::bfs(int source, int sink,
                        std::unordered_map<int, ResidualArc>&
                        parent) const
{
    // ordinary bfs, but on residual network
    std::queue<int> queue;
    std::unordered_set<int> visited;

    parent.clear();
    queue.push(source);
    visited.insert(source);

    while (!queue.empty()) {
        int current = queue.front();
        queue.pop();

        // only edges that still can change flow are visible here
        for (const ResidualArc& arc : m_network.residualNeighbors
             (current)) {
            if (visited.find(arc.to) != visited.end()) {
                continue;
            }

            // store whole residual arc, not only previous vertex

```

```
        // we'll need it later for augment(...)
        visited.insert(arc.to);
        parent[arc.to] = arc;

        if (arc.to == sink) {
            // first found path is enough for this bfs step
            return true;
        }

        queue.push(arc.to);
    }
}

return false;
}
```

Приложение Я. Файл MaxMatching.h

```
#pragma once
#include "include/Graph.h"
#include <vector>

/// Result of greedy maximal matching.
///
/// "Maximal" here is per the lecture slide: a matching whose
    superset is no
/// longer independent. This is found greedily and is NOT
    necessarily the
/// largest possible matching (that would be "maximum"); see the
    slide on
/// Hall's theorem for the bipartite case.
struct MatchingResult {
    std::vector<WeightedEdge> matchingEdges; //
        выбранные независимые рёбра
    std::vector<int> unmatchedVertices;      // непокрытые
    int matchingSize;                       // = matchingEdges.
        size()
    bool isPerfect;                         // вселипокрыты
};

class GreedyMaximalMatching {
public:
    explicit GreedyMaximalMatching(const AdjacencyGraph& graph);

    /// Build a maximal independent edge set greedily:
    ///   for each edge (u, v) in some order:
    ///       if neither u nor v is matched yet, take this edge
    ///
    /// The graph is treated as undirected. To make output
        reproducible across
    /// runs on the same input, edges are processed in
        lexicographic (from, to) order.
    MatchingResult compute() const;

private:
    const AdjacencyGraph& m_graph;
};
```

Приложение АА. Файл MaxMatching.cpp

```
/*
matching ← ∅
matched ← ∅ // вершины,
уже покрытые матчингом для каждого ребра
(u, v) графа: если
    u ∈ matched и v ∈ matched:
        matching ← matching ∪ {(u, v)}
        matched ← matched ∪ {u, v} вернуть
matching
*/

#include "include/MaxMatching.h"
#include <algorithm>
#include <unordered_set>

GreedyMaximalMatching::GreedyMaximalMatching(const AdjacencyGraph
    & graph)
    : m_graph(graph)
{}

MatchingResult GreedyMaximalMatching::compute() const {
    MatchingResult result;
    result.matchingSize = 0;
    result.isPerfect = false;

    const int n = m_graph.size();
    if (n == 0) {
        return result;
    }

    // Pull all edges. AdjacencyGraph::edges() yields each
    // undirected edge once.
    std::vector<WeightedEdge> allEdges = m_graph.edges();

    // Sort lexicographically by (from, to) so the same input
    // always produces
    // the same matching. The greedy algorithm is correct under
    // any order, but
    // a deterministic order is important for reproducible labs.
```

```

std::sort(allEdges.begin(), allEdges.end(),
    [](const WeightedEdge& a, const WeightedEdge& b) {
        if (a.from != b.from) return a.from < b.from;
        return a.to < b.to;
    });

// matched: set of vertex IDs already covered by the matching
.
// We use unordered_set keyed by vertex ID directly -- IDs
may not be
// 0..n-1 in general, so we don't assume dense indexing here.
std::unordered_set<int> matched;
matched.reserve(static_cast<size_t>(n));

// Greedy pass: take an edge iff both endpoints are still
free.
// After this loop, no edge can be added without sharing an
endpoint
// with one already chosen -> the result is maximal by
inclusion.
for (const WeightedEdge& edge : allEdges) {
    if (edge.from == edge.to) {
        continue; // self-loop: not a valid matching edge,
        skip
    }
    if (matched.count(edge.from) == 0 && matched.count(edge.
to) == 0) {
        result.matchingEdges.push_back(edge);
        matched.insert(edge.from);
        matched.insert(edge.to);
    }
}

result.matchingSize = static_cast<int>(result.matchingEdges.
size());

// Collect vertices not covered. If empty, the matching is
perfect.
for (int v : m_graph.vertexIds()) {
    if (matched.count(v) == 0) {
        result.unmatchedVertices.push_back(v);
    }
}

```

```
        }  
    }  
    result.isPerfect = result.unmatchedVertices.empty();  
  
    return result;  
}
```

Приложение АБ. Файл MermaidExporter.h

```
#pragma once

#include "include/FlowNetwork.h"
#include "include/Graph.h"

#include <filesystem>

class MermaidExporter {
public:
    static std::filesystem::path exportGraph(const AdjacencyGraph
        & graph,
                                           const std::string&
        fileName = "
        generated_graph.
        mmd");
    static std::filesystem::path exportFlowNetwork(
        const FlowNetwork& network,
        const std::string& fileName = "generated_flow_network.mmd
        ");
private:
    static std::filesystem::path executableDirectory();
};
```


Приложение АВ. Файл MermaidExporter.cpp

```
#include "include/MermaidExporter.h"

#include <fstream>
#include <iomanip>
#include <sstream>
#include <string>
#include <stdexcept>

#include <windows.h>

namespace {
std::string formatWeight(double weight) {
    std::ostringstream out;
    out << std::fixed << std::setprecision(2) << weight;
    std::string text = out.str();

    while (!text.empty() && text.back() == '0') {
        text.pop_back();
    }
    if (!text.empty() && text.back() == '.') {
        text.pop_back();
    }
    if (text.empty()) {
        return "0";
    }
    return text;
}

std::string formatWeight(int weight) {
    return std::to_string(weight);
}
}

std::filesystem::path MermaidExporter::exportGraph(const
AdjacencyGraph& graph,
                                                    const std::
string&
fileName) {
```

```

    const std::filesystem::path outputPath = executableDirectory
        () / fileName;
    std::ofstream out(outputPath);
    if (!out) {
        throw std::runtime_error("Failed to create Mermaid file")
            ;
    }

    out << "graph LR\n";

    for (int vertexId : graph.vertexIds()) {
        out << "    v" << vertexId << "[\\"" << vertexId << "\"]\n"
            ;
    }

    const std::string connector = graph.isDirected() ? "-->" : "
        ---";
    for (const WeightedEdge& edge : graph.edges()) {
        out << "    v" << edge.from
            << " " << connector << "|\"" << formatWeight(edge.
                weight) << "\"| "
            << "v" << edge.to << "\n";
    }

    return outputPath;
}

std::filesystem::path MermaidExporter::exportFlowNetwork(const
    FlowNetwork& network,

                                const
                                std::
                                string
                                &
                                fileName
                                ) {
    const std::filesystem::path outputPath = executableDirectory
        () / fileName;
    std::ofstream out(outputPath);
    if (!out) {
        throw std::runtime_error("Failed to create Mermaid file")
            ;
    }

```

```

    }

    out << "graph LR\n";

    for (int vertexId : network.vertexIds()) {
        out << "    v" << vertexId << "[\"\" << vertexId << "\"]\n"
        ";
    }

    for (const FlowEdge& edge : network.edges()) {
        out << "    v" << edge.from
            << " -->|\"cap=\"" << formatWeight(edge.capacity)
            << ", cost=\"" << formatWeight(edge.cost)
            << ", flow=\"" << formatWeight(edge.flow) << "\"| "
            << "v" << edge.to << "\n";
    }

    return outputPath;
}

std::filesystem::path MermaidExporter::executableDirectory() {
    char buffer[MAX_PATH];
    const DWORD length = GetModuleFileNameA(nullptr, buffer,
        MAX_PATH);
    if (length == 0 || length == MAX_PATH) {
        throw std::runtime_error("Failed to resolve executable
            path");
    }

    return std::filesystem::path(buffer).parent_path();
}

```

Приложение АГ. Файл MinCostFlow.h

```
#pragma once

#include "include/FlowNetwork.h"

#include <unordered_map>
#include <vector>

// one augmentation step for min-cost-flow
struct MinCostFlowStep {
    int iteration; // step number
    std::vector<int> path; // chosen shortest path in residual
        network
    int pathFlow; // how much flow was added through this path
    int pathUnitCost; // cost of 1 unit along this path
    int cumulativeFlow; // total sent flow after this step
    int cumulativeCost; // total cost after this step
};

struct MinCostFlowResult {
    int targetFlow; // what we wanted to send
    int achievedFlow; // what was actually sent
    int totalCost; // final price
    bool success; // was target reached fully?
    std::vector<MinCostFlowStep> steps; // for printing /
        checking
};

class MinCostFlowSolver {
public:
    explicit MinCostFlowSolver(FlowNetwork& network);

    // sends exactly targetFlow if possible, but with minimal
    // cost
    MinCostFlowResult compute(int source, int sink, int
        targetFlow);

private:
    // local dijkstra on residual network
    // still follows the same logic as old lab dijkstra
```

```
bool dijkstra(int source, int sink,
              std::unordered_map<int, int>& distances,
              std::unordered_map<int, ResidualArc>& parent)
    const;

FlowNetwork& m_network;
};
```

Приложение АД. Файл MinCostFlow.cpp

```
#include "include/MinCostFlow.h"

#include <algorithm>
#include <limits>
#include <unordered_set>

namespace {
// same idea as in max-flow:
// recover residual path by going from sink back to source
std::vector<ResidualArc> reconstructResidualPath(
    int source,
    int sink,
    const std::unordered_map<int, ResidualArc>& parent)
{
    std::vector<ResidualArc> path;
    int current = sink;

    while (current != source) {
        auto it = parent.find(current);
        if (it == parent.end()) {
            return {};
        }
        path.push_back(it->second);
        current = it->second.from;
    }

    std::reverse(path.begin(), path.end());
    return path;
}

// helper just for pretty output
std::vector<int> toVertexPath(int source, const std::vector<
    ResidualArc>& residualPath) {
    std::vector<int> path;
    path.push_back(source);

    for (const ResidualArc& arc : residualPath) {
        path.push_back(arc.to);
    }
}
```

```

    return path;
}
}

MinCostFlowSolver::MinCostFlowSolver(FlowNetwork& network)
    : m_network(network) {}

MinCostFlowResult MinCostFlowSolver::compute(int source, int sink
, int targetFlow) {
    MinCostFlowResult result{targetFlow, 0, 0, false, {}};

    // sending 0 units is trivially successful
    if (targetFlow <= 0) {
        result.success = true;
        return result;
    }

    // bad source/sink => no answer
    if (source == sink ||
        !m_network.hasVertex(source) ||
        !m_network.hasVertex(sink)) {
        return result;
    }

    // solve min-cost-flow from clean zero state
    m_network.resetFlows();

    std::unordered_map<int, int> distances;
    std::unordered_map<int, ResidualArc> parent;
    int iteration = 1;

    // each iteration:
    // 1) find shortest path by cost in residual network
    // 2) push as much as possible through it
    while (result.achievedFlow < targetFlow &&
        dijkstra(source, sink, distances, parent)) {
        std::vector<ResidualArc> residualPath =
            reconstructResidualPath(source, sink, parent);
        if (residualPath.empty()) {
            break;

```

```

    }

    // initially we can still send only "what is left" to
    // target
    int bottleneck = targetFlow - result.achievedFlow;
    int unitCost = 0;

    // then restrict it by capacities on path
    // also sum full path cost for one unit of flow
    for (const ResidualArc& arc : residualPath) {
        bottleneck = std::min(bottleneck, arc.
            residualCapacity);
        unitCost += arc.cost;
    }

    // just safety
    if (bottleneck <= 0) {
        break;
    }

    // push flow through all arcs of this path
    for (const ResidualArc& arc : residualPath) {
        m_network.augment(arc.edgeIndex, arc.forward,
            bottleneck);
    }

    // update totals
    result.achievedFlow += bottleneck;
    result.totalCost += bottleneck * unitCost;
    result.steps.push_back({
        iteration++,
        toVertexPath(source, residualPath),
        bottleneck,
        unitCost,
        result.achievedFlow,
        result.totalCost
    });
}

result.success = result.achievedFlow >= targetFlow;
return result;

```



```

}

bool MinCostFlowSolver::dijkstra(int source, int sink,
                                std::unordered_map<int, int>&
                                distances,
                                std::unordered_map<int,
                                ResidualArc>& parent) const
{
    constexpr int INF = std::numeric_limits<int>::max();
    distances.clear();
    parent.clear();

    // same guard as in original dijkstra
    if (!m_network.hasVertex(source)) {
        return false;
    }

    auto vertexIds = m_network.vertexIds();
    int p = m_network.size(); // number of vertices (p in the
                             // algorithm)

    // Build hash map for vertex ID to index (1-based to match
    // algorithm)
    std::unordered_map<int, int> idToIndex;
    for (int i = 0; i < p; ++i) {
        idToIndex[vertexIds[i]] = i;
    }

    // T[v] -- distance from s to v (T : array [1..p] of real)
    std::vector<int> T(p, INF);

    // X[v] -- mark: 0 = unvisited, 1 = visited (X : array [1..p]
    // of 0..1)
    std::vector<int> X(p, 0);

    // H[v] -- predecessor of v on shortest path (H : array [1..p]
    // of 0..p)
    std::vector<int> H(p, -1);
    // extra thing for flow lab:
    // remember the exact residual arc, not only predecessor
    // vertex

```

```

std::vector<std::optional<ResidualArc>> parentArcs(p, std::
    nullopt);

int s = source;
int t = sink;

int startIndex = idToIndex[s];
T[startIndex] = 0;           //  $T[s] := 0$ 
X[startIndex] = 1;           //  $X[s] := 1$  ( $s$  is known)

int v = s;                   //  $v := s$  (current vertex)
int vIndex = startIndex;

// Label M: update labels
while (true) {
    //result.iterations++; // Count: main loop iteration

    // for  $u \in \Gamma(v)$  do -- examine all neighbors of  $v$ 
    for (const ResidualArc& arc : m_network.residualNeighbors
        (v)) {
        auto neighborIt = idToIndex.find(arc.to);
        if (neighborIt == idToIndex.end()) {
            continue;
        }
        int uIndex = neighborIt->second;

        // if  $X[u] = 0$  &  $T[u] > T[v] + C[v, u]$  then
        if (X[uIndex] == 0 && T[uIndex] > T[vIndex] + arc.
            cost) {
            T[uIndex] = T[vIndex] + arc.cost; //  $T[u] := T[v]$ 
                 $+ C[v, u]$ 
            H[uIndex] = vIndex;                //  $H[u] := v$ 
            parentArcs[uIndex] = arc;
        }
    }

    //  $m := \infty$ ;  $v := 0$ 
    int m = INF;
    v = 0;
    vIndex = -1;
}

```

```

    // for u from 1 to p do -- find vertex with minimum T
    for (int u = 0; u < p; ++u) {
        // if X[u] = 0 & T[u] < m then
        if (X[u] == 0 && T[u] < m) {
            vIndex = u;
            m = T[u];
            v = vertexIds[u];
        }
    }

    // if v = 0 then stop -- no path from s to t
    if (vIndex == -1) {
        break;
    }

    // if v = t then stop -- shortest path from s to t found
    if (v == t) {
        break;
    }

    X[vIndex] = 1; // X[v] := 1 -- shortest path from s to v
                    found
}

for (int i = 0; i < p; ++i) {
    if (T[i] != INF) {
        distances[vertexIds[i]] = T[i];
    }
    // parent for flow solver = exact residual arc to this
    // vertex
    if (parentArcs[i].has_value()) {
        parent[vertexIds[i]] = parentArcs[i].value();
    }
}

auto sinkIt = idToIndex.find(sink);
return sinkIt != idToIndex.end() && T[sinkIt->second] != INF;
}

```

Приложение АЕ. Файл PathCounter.h

```
#pragma once
#include "include/Graph.h"
#include <vector>

// Matrix storing multiple paths (each row = one path)
using PathCollection = std::vector<std::vector<int>>>;

class SimplePathFinder {
public:
    explicit SimplePathFinder(AdjacencyGraph const& graph);
    int countPaths(int from, int to);
    PathCollection findAllPaths(int from, int to);

private:
    AdjacencyGraph const& m_graph;

    // backtrack
    void dfsExplore(
        int current,
        int target,
        std::vector<bool>& visited,
        std::vector<int>& path,
        PathCollection& allPaths);
};
```

Приложение АЖ. Файл PathCounter.cpp

```
#include "include/PathCounter.h"
#include <unordered_map>

SimplePathFinder::SimplePathFinder(AdjacencyGraph const& graph)
    : m_graph(graph)
{}

int SimplePathFinder::countPaths(int from, int to) { // kostyl' : P
    PathCollection paths = findAllPaths(from, to);
    return static_cast<int>(paths.size());
}

PathCollection SimplePathFinder::findAllPaths(int from, int to) {
    // Validate vertices exist
    if (!m_graph.hasVertex(from) || !m_graph.hasVertex(to)) {
        return {};
    }

    PathCollection collectedPaths;
    std::vector<int> currentPath;
    std::vector<bool> visited(m_graph.size(), false);

    // MBuild vcertex ID → index mapping
    auto vertexList = m_graph.vertexIds();
    std::unordered_map<int, int> idToIndex;
    for (int i = 0; i < static_cast<int>(vertexList.size()); ++i)
    {
        idToIndex[vertexList[i]] = i; // it's for the future
    } // when IDs may not be like "/N"

    dfsExplore(from, to, visited, currentPath, collectedPaths);
    return collectedPaths;
}

void SimplePathFinder::dfsExplore(
    int current,
    int target,
    std::vector<bool>& visited,
```

```

std::vector<int>& path,
PathCollection& allPaths)
{
    // Add current vertex to path
    path.push_back(current);
    visited[current] = true;

    // Check if reached destination
    if (current == target) {
        allPaths.push_back(path);
    } else {
        // Explore unvisited neighbors
        for (const auto& [neighborId, edgeWeight] : m_graph.
            neighbors(current)) {
            if (!visited[neighborId]) {
                dfsExplore(neighborId, target, visited, path,
                    allPaths);
            }
        }
    }

    // Backtrack
    path.pop_back();
    visited[current] = false;
}

```

Приложение А3. Файл PruferCode.h

```
#pragma once
#include "include/Graph.h"
#include <memory>
#include <vector>

/// Prufer code of a free tree, extended with edge weights.
///
/// Per the lecture material (slide 30), the code has length  $p - 1$  (not  $p - 2$ ):
/// the textbook variant that drops the last edge is NOT used
/// here.
///
/// At step  $i$  of encoding we remove the leaf with the smallest ID
/// and write down:
///   - code[i] -- the ID of its only neighbor
///   - weights[i] -- the weight of the edge between them
/// This pairing is what makes weight-preserving encode/decode
/// possible.
struct PruferCode {
    std::vector<int> code;           // length =  $p - 1$ 
    std::vector<double> weights;    // length =  $p - 1$ , parallel to
    `code`
};

class PruferEncoder {
public:
    /// Encode a tree into its Prufer code (with weights
    /// preserved).
    /// Precondition: `tree` is a connected acyclic undirected
    /// graph (a tree).
    /// If preconditions are violated the result is unspecified.
    static PruferCode encode(const AdjacencyGraph& tree);

    /// Decode a Prufer code (with weights) back into a tree.
    ///
    /// Because the code alone does not record which vertex IDs
    /// exist
    /// (e.g. the very last vertex may never appear in the code),
    /// the caller
```

```
/// must also pass `vertexIds` -- the full vertex set of the
    tree to rebuild.
///
/// Returned graph is undirected (free trees are undirected
    by definition).
static std::unique_ptr<AdjacencyGraph> decode(
    const PruferCode& pruferCode,
    const std::vector<int>& vertexIds);
};
```


Приложение АИ. Файл PruferCode.cpp

```
#include "include/PruferCode.h"
#include <algorithm>
#include <map>
#include <unordered_map>
#include <unordered_set>

namespace {

// Helper: find the leaf with the smallest ID in `currentDegree`
// over `aliveVertices`.
// Returns the vertex ID (caller is sure at least one leaf exists
// in a non-trivial tree).
int findSmallestLeaf(
    const std::vector<int>& aliveVertices,
    const std::unordered_map<int, int>& currentDegree)
{
    int smallest = -1;
    for (int v : aliveVertices) {
        if (currentDegree.at(v) == 1) {
            if (smallest == -1 || v < smallest) {
                smallest = v;
            }
        }
    }
    return smallest;
}

} // namespace

PruferCode PruferEncoder::encode(const AdjacencyGraph& tree) {
    PruferCode result;

    const int p = tree.size();
    if (p < 2) {
        // No edges to record. By the slide-30 formula the code
        // length is p-1,
        // which is 0 for p=1 and -1 for p=0 -- we simply return
        // empty.
        return result;
    }
}
```

```

}

// Build a working representation we can mutate as we strip
// leaves:
// - aliveVertices: the set of IDs still present in the
//   tree
// - adjacency:      a multiset-like adjacency map (vertex
//   -> neighbors)
// - edgeWeight:     weight of an unordered edge {u, v},
//   looked up by
//                   (min(u,v), max(u,v)) so direction doesn
//   't matter
// - currentDegree: degree of each vertex in the shrinking
//   tree
//
// We don't mutate the input graph -- everything happens in
// these local
// structures.

std::vector<int> aliveVertices = tree.vertexIds();
////////// V := множество вершин дерева копии ( для удаления )

std::unordered_map<int, std::unordered_set<int>> adjacency;
std::unordered_map<int, int> currentDegree;
for (int v : aliveVertices) {
    adjacency[v] = {};
    currentDegree[v] = 0;
}

// edges() returns each undirected edge once (filtered by
// from < to internally),
// so we just record both directions and the weight once.
auto edgeKey = [](int a, int b) {
    if (a > b) std::swap(a, b);
    return std::pair<int, int>{a, b};
};

std::map<std::pair<int, int>, double> edgeWeight;
for (const WeightedEdge& edge : tree.edges()) {
    adjacency[edge.from].insert(edge.to);
    adjacency[edge.to].insert(edge.from);
}

```

```

    currentDegree[edge.from] += 1;
    currentDegree[edge.to] += 1;
    edgeWeight[edgeKey(edge.from, edge.to)] = edge.weight;
}
//////////  $\Gamma(v)$  – список смежности (adjacency)
//////////  $d(k)$  – степень вершин (currentDegree)

// Main loop: exactly  $p - 1$  iterations, per slide 30.
// Each iteration removes the smallest-ID leaf and writes
// down (neighbor, weight).
result.code.reserve(p - 1);
result.weights.reserve(p - 1);

// for  $i$  from 1 to  $p - 1$  do
for (int i = 0; i < p - 1; ++i) {
    ////////// for  $i$  from 1 to  $p - 1$  do

    // Step 1: find leaf  $v$  with smallest ID (degree == 1).
    //  $v := \min \{k \in V \mid d(k) = 1\}$ 
    const int leaf = findSmallestLeaf(aliveVertices,
        currentDegree);
    //////////  $v := \min \{k \in V \mid d(k) = 1\}$  { выбираем вершину
         $v$  – висячую вершину с наименьшим номером }

    if (leaf == -1) {
        // Shouldn't happen in a valid tree, but bail out
        // gracefully.
        break;
    }

    // Step 2: identify  $v$ 's only neighbor.
    // adjacency[leaf] has exactly one element since  $\deg(\text{leaf}) == 1$ .
    const int neighbor = *adjacency[leaf].begin();

    // Step 3: write  $\text{code}[i] = \text{neighbor}$ ,  $\text{weights}[i] = w(\text{leaf}, \text{neighbor})$ .
    result.code.push_back(neighbor);
    result.weights.push_back(edgeWeight.at(edgeKey(leaf, neighbor)));
}

```

```

////////// A[i] :=  $\Gamma(v)$  {
    записываем в код номер единственной вершины , смежной с v }

    // Step 4: physically remove leaf from the tree.
    // - drop the edge from both adjacency lists
    // - decrement the neighbor's degree
    // - remove leaf from aliveVertices
    adjacency[neighbor].erase(leaf);
    adjacency[leaf].clear();
    currentDegree[neighbor] -= 1;
    currentDegree[leaf] = 0;

    aliveVertices.erase(
        std::find(aliveVertices.begin(), aliveVertices.end(),
            leaf));
    //////////// V := V - v { удаляем вершину v из дерева }
}
////////// end for

return result;
////////// Выход: Массив A : array [1..p-1] of 1..p -
    код Прюфера дерева T
}

std::unique_ptr<AdjacencyGraph> PruferEncoder::decode(
    const PruferCode& pruferCode,
    const std::vector<int>& vertexIds)
{
    auto tree = std::make_unique<AdjacencyGraph>(false); // free
        tree -> undirected

    // Always add all vertices, even if some never appear in the
        code.
    for (int v : vertexIds) {
        tree->addVertex(v);
    }

    const int p = static_cast<int>(vertexIds.size());
    if (p < 2) {
        return tree; // 0 or 1 vertices -> no edges to add
    }
}

```

```

// Sanity: the code must have length  $p - 1$  to match our
// encoder.
// If it doesn't, decode whatever we can without crashing.
const int codeLength = static_cast<int>(pruferCode.code.size
());
const int weightsLength = static_cast<int>(pruferCode.weights
.size());
const int steps = std::min({p - 1, codeLength, weightsLength
});

// B = unused vertex IDs (sorted ascending so "smallest
// unused" is just front()).
std::vector<int> unusedSorted = vertexIds;
std::sort(unusedSorted.begin(), unusedSorted.end());
////////// B := 1..p {
// множество неиспользованных номеров вершин }

std::unordered_set<int> E; // edges will be added to tree
////////// E := ∅ { в начале множество ребер пусто }

// Decoder per slide 31:
// for i from 1 to p - 1:
//     v := min { k in B | for all j >= i, k != A[j] }
//     E := E + (v, A[i])
//     B := B - v
//
// "Smallest unused ID that does NOT appear in the remaining
// suffix of the code".

// for i from 1 to p - 1 do
for (int i = 0; i < steps; ++i) {
    ////////// for i from 1 to p - 1 do

    // Build the set of values present in code[i..end]: that's
    // the forbidden set.
    std::unordered_set<int> remainingInCode;
    for (int j = i; j < codeLength; ++j) {
        remainingInCode.insert(pruferCode.code[j]);
    }
    ////////// { множество вершин ,
    // встречающихся в остатке кода Прюфера A[j] для j >= i }

```

```

// Walk unusedSorted from the front to find the smallest
// k not in remainingInCode.
int chosenIdx = -1;
for (int idx = 0; idx < static_cast<int>(unusedSorted.
    size()); ++idx) {
    if (remainingInCode.find(unusedSorted[idx]) ==
        remainingInCode.end()) {
        chosenIdx = idx;
        break;
    }
}
if (chosenIdx == -1) {
    // Malformed code -- the suffix forbids every
    // remaining vertex.
    // Stop gracefully rather than throw.
    break;
}

const int v = unusedSorted[chosenIdx];
//////////  $v := \min \{k \in B \mid \exists j \geq i (k \neq A[j])\}$  {
    выбираем вершину  $v$  –
    неиспользованную вершину с наименьшим номером
    который не встречается в остатке кода Прюфера
}

const int neighbor = pruferCode.code[i];
const double weight = pruferCode.weights[i];

tree->addEdge(v, neighbor, weight);
E.insert(v);
E.insert(neighbor);
//////////  $E := E + (v, A[i])$  { добавляем ребро  $(v, A[i])$ 
}

// Remove v from unused.
unusedSorted.erase(unusedSorted.begin() + chosenIdx);
//////////  $B := B - v$  { удаляем вершину  $v$ 
    из списка неиспользованных
}
}
////////// end for

```

```

// After  $p - 1$  steps the encoder leaves exactly one vertex in
//  $B$  and one
// unrecorded edge (the last leaf attached to the last
// survivor). Per slide 30
// we DO write down that final edge in the code, so the loop
// above already
// covered all  $p - 1$  edges -- no extra "join the last two
// unused" step needed.

return tree;
////////// Выход: Дерево  $T(V, E)$ , заданное множеством рёбер  $E$ 
}

```

Приложение АК. Файл ShimbellMethod.h

```
#pragma once
#include "include/Graph.h"
#include <vector>
#include <optional>
#include <iostream>

//nullopt = unreachable
using WeightTable = std::vector<std::vector<std::optional<double>>>

struct ShimbellOutput {
    WeightTable minWeights;
    WeightTable maxWeights;
    int edgeCount;
};

void printWeightTable(std::ostream& out, const WeightTable& table
    ,
                    const std::vector<int>& vertexIds);

void printAdjacencyMatrix(std::ostream& out, const
    AdjacencyMatrix& matrix,
                    const std::vector<int>& vertexIds);

class KPathCalculator {
public:
    explicit KPathCalculator(const AdjacencyGraph& graph);
    ShimbellOutput compute(int edgeCount);

private:
    const AdjacencyGraph& m_graph;
    std::vector<int> m_vertexIds; // List of vertex IDs
    int m_vertexCount;

    WeightTable buildWeightMatrix() const;

    WeightTable multiplyMinPlus(const WeightTable& left, const
        WeightTable& right) const;
```



```
WeightTable multiplyMaxPlus(const WeightTable& left, const
    WeightTable& right) const;

int mapVertexToIndex(int vertexId) const; // 0T0
};
```

Приложение АЛ. Файл ShimbellMethod.cpp

```
#include "include/ShimbellMethod.h"
#include <algorithm>
#include <limits>
#include <stdexcept>
#include <iomanip>

KPathCalculator::KPathCalculator(const AdjacencyGraph& graph)
    : m_graph(graph)
    , m_vertexIds(graph.vertexIds())
    , m_vertexCount(graph.size())
{}

ShimbellOutput KPathCalculator::compute(int edgeCount) {
    if (edgeCount < 0) {
        throw std::invalid_argument("Edge count must be non-
            negative");
    }

    if (edgeCount == 0) {
        WeightTable zeroEdgeMatrix(
            m_vertexCount,
            std::vector<std::optional<double>>(m_vertexCount, std
                ::nullopt)
        );

        for (int i = 0; i < m_vertexCount; ++i) {
            zeroEdgeMatrix[i][i] = 0.0;
        }

        return {zeroEdgeMatrix, zeroEdgeMatrix, 0};
    }

    // Step 1: Build the base matrix for 1-edge paths (direct
    // connections)
    WeightTable baseMatrix = buildWeightMatrix();

    // If we only want 1-edge paths, just return the base matrix
    // with diagonal set to 0
```

```

// (diagonal = 0 means no cost for staying at the same vertex
// , but we don't count it as a path)
if (edgeCount == 1) {
    for (int i = 0; i < m_vertexCount; ++i) {
        baseMatrix[i][i] = 0.0;
    }
    return {baseMatrix, baseMatrix, 1};
}

// Step 2: For more edges (k > 1), we "raise the matrix to
// the power k" using special multiplication
// Think of it like: start with 1-edge paths, then multiply
// by the base to get 2-edge paths,
// multiply again for 3-edge paths, and so on, up to k edges.
// We use min-plus multiplication for shortest paths (
// smallest total weight) and
// max-plus for longest paths (biggest total weight).
// This is like matrix exponentiation, but in a "tropical"
// way for graphs!
WeightTable minMatrix = baseMatrix; // Starts as A^1
WeightTable maxMatrix = baseMatrix; // Starts as A^1

// Loop from 2 to k: each time, multiply the current matrix
// by the base matrix
// This builds up paths with exactly 'step' edges
for (int step = 2; step <= edgeCount; ++step) {
    // Right now, minMatrix has paths with exactly (step-1)
    // edges.
    // We multiply it by baseMatrix (1-edge paths) to get
    // paths with exactly 'step' edges.
    // For example: 2-edge paths = (1-edge paths) combined
    // with (1-edge paths)
    minMatrix = multiplyMinPlus(minMatrix, baseMatrix);
    maxMatrix = multiplyMaxPlus(maxMatrix, baseMatrix);
}

// Step 3: Set diagonal to 0 in the final result (no self-
// cost for the paths we found)
for (int i = 0; i < m_vertexCount; ++i) {
    minMatrix[i][i] = 0.0;
    maxMatrix[i][i] = 0.0;
}

```

```

    }

    return {minMatrix, maxMatrix, edgeCount};
}

WeightTable KPathCalculator::buildWeightMatrix() const {
    // Initialize with nullopt (no path)
    WeightTable matrix(
        m_vertexCount,
        std::vector<std::optional<double>>(m_vertexCount, std::
            nullopt)
    );

    // Diagonal = nullopt (no self-loops for exact k-edge paths)
    // This ensures we only find paths with EXACTLY k edges, not
    // reusing old results

    // Fill edge weights (direct edges only)
    for (const auto& edge : m_graph.edges()) {
        int row = mapVertexToIndex(edge.from);
        int col = mapVertexToIndex(edge.to);
        matrix[row][col] = edge.weight;

        // Undirected: symmetric
        if (!m_graph.isDirected()) {
            matrix[col][row] = edge.weight;
        }
    }

    return matrix;
}

WeightTable KPathCalculator::multiplyMinPlus(
    const WeightTable& left,
    const WeightTable& right) const
{
    WeightTable result(
        m_vertexCount,
        std::vector<std::optional<double>>(m_vertexCount, std::
            nullopt)
    );

```

```

// For each cell (i, j)
for (int i = 0; i < m_vertexCount; ++i) {
    for (int j = 0; j < m_vertexCount; ++j) {
        std::optional<double> minimum = std::nullopt;

        // Try all intermediate vertices k
        for (int k = 0; k < m_vertexCount; ++k) {
            if (left[i][k].has_value() && right[k][j].
                has_value()) {
                // Perform min-plus multiplication: sum the
                // weights and track the minimum
                double sum = left[i][k].value() + right[k][j].
                    value();

                if (!minimum.has_value() || sum < minimum.
                    value()) {
                    minimum = sum;
                }
            }
        }

        result[i][j] = minimum;
    }
}

return result;
}

WeightTable KPathCalculator::multiplyMaxPlus(
    const WeightTable& left,
    const WeightTable& right) const
{
    WeightTable result(
        m_vertexCount,
        std::vector<std::optional<double>>(m_vertexCount, std::
            nullopt)
    );

    // For each cell (i, j)
    for (int i = 0; i < m_vertexCount; ++i) {

```

```

    for (int j = 0; j < m_vertexCount; ++j) {
        std::optional<double> maximum = std::nullopt;

        // Try all intermediate vertices k
        for (int k = 0; k < m_vertexCount; ++k) {
            if (left[i][k].has_value() && right[k][j].
                has_value()) {
                // Perform max-plus multiplication: sum the
                // weights and track the maximum
                double sum = left[i][k].value() + right[k][j].
                    value();

                if (!maximum.has_value() || sum > maximum.
                    value()) {
                    maximum = sum;
                }
            }
        }

        result[i][j] = maximum;
    }
}

return result;
}

int KPathCalculator::mapVertexToIndex(int vertexId) const {
    auto iter = std::find(m_vertexIds.begin(), m_vertexIds.end(),
        vertexId);
    return static_cast<int>(std::distance(m_vertexIds.begin(),
        iter));
}

void printWeightTable(std::ostream& out, const WeightTable& table
    ,
        const std::vector<int>& vertexIds) {
    const size_t n = table.size();
    if (n == 0) return;

    // Print header row
    out << "      ";

```

```

    for (int v : vertexIds) {
        out << std::setw(10) << v;
    }
    out << "\n";

    // Print separator
    out << "          ";
    for (size_t i = 0; i < n; ++i) {
        out << "-----";
    }
    out << "\n";

    // Print matrix rows
    for (size_t i = 0; i < n; ++i) {
        out << std::setw(5) << vertexIds[i] << " |";
        for (size_t j = 0; j < n; ++j) {
            if (table[i][j].has_value()) {
                out << std::setw(10) << table[i][j].value();
            } else {
                out << std::setw(10) << "i"; // i = infinity (no
                path)
            }
        }
        out << "\n";
    }
}

void printAdjacencyMatrix(std::ostream& out, const
AdjacencyMatrix& matrix,
                        const std::vector<int>& vertexIds) {
    const size_t n = matrix.size();
    if (n == 0) return;

    // Print header row
    out << "          ";
    for (int v : vertexIds) {
        out << std::setw(10) << v;
    }
    out << "\n";

    // Print separator

```

```

out << "          ";
for (size_t i = 0; i < n; ++i) {
    out << "-----";
}
out << "\n";

// Print matrix rows
for (size_t i = 0; i < n; ++i) {
    out << std::setw(5) << vertexIds[i] << " |";
    for (size_t j = 0; j < n; ++j) {
        // true = 1 (edge exists), false = 0 (no edge)
        out << std::setw(10) << (matrix[i][j] ? 1 : 0);
    }
    out << "\n";
}
}

```


Приложение АМ. Файл main.cpp

```
#include <iostream>
#include <iomanip>
#include <limits>
#include <cmath>
#include <cctype>
#include <sstream>
#include <string>
#include <tuple>
#include <vector>
#include <algorithm>
#include "include/Graph.h"
#include "include/Generator.h"
#include "include/ShimbellMethod.h"
#include "include/PathCounter.h"
#include "include/Eccentricity.h"
#include "include/BFS.h"
#include "include/Dijkstra.h"
#include "include/AlgorithmComparator.h"
#include "include/FlowNetwork.h"
#include "include/FlowNetworkBuilder.h"
#include "include/MaxFlowSolver.h"
#include "include/MinCostFlow.h"
#include "include/MermaidExporter.h"
#include "include/Kirchhoff.h"
#include "include/Kruskal.h"
#include "include/PruferCode.h"
#include "include/MaxMatching.h"
#include "include/EulerianCycle.h"
#include "include/FundamentalCycles.h"
#include "include/EdmondsMatching.h"
```

```
//
```

```
=====
```

```
// Helper Functions
```

```
//
```

```
=====
```

```

void showMenu(bool hideLab1, bool hideLab2, bool hideLab3, bool
hideLab4, bool hideLab5, bool hideCustom) {
    std::cout << "\n===== MAIN MENU =====\n";
    if (!hideLab1) {
        std::cout << "\n   --- Lab 1 ---\n";
        std::cout << "   1. Generate random graph\n";
        std::cout << "   2. Calculate eccentricities & center\n";
        std::cout << "   3. Shimbell's method (k-edge paths)\n";
        std::cout << "   4. Find all paths between vertices\n";
        std::cout << "   5. Show graph details\n";
        std::cout << "   6. Compare distribution statistics\n";
        std::cout << "   7. Show adjacency matrix\n";
        std::cout << "   8. Show weight matrix (Shimbell k=1)\n";
    }
    if (!hideLab2) {
        std::cout << "\n   --- Lab 2 ---\n";
        std::cout << "   9. BFS traversal\n";
        std::cout << "  10. Dijkstra's shortest path\n";
        std::cout << "  11. Compare BFS vs Dijkstra (speed)\n";
    }
    if (!hideLab3) {
        std::cout << "\n   --- Lab 3 ---\n";
        std::cout << "  12. Build flow network from current
            directed graph\n";
        std::cout << "  13. Show capacity matrix\n";
        std::cout << "  14. Show cost matrix\n";
        std::cout << "  15. Find maximum flow\n";
        std::cout << "  16. Find minimum-cost flow for [2/3 * max
            ]\n";
        std::cout << "  17. Show flow network details\n";
    }
    if (!hideLab4) {
        std::cout << "\n   --- Lab 4 ---\n";
        std::cout << "  18. Count spanning trees (Kirchhoff's
            theorem)\n";
        std::cout << "  19. Build minimum spanning tree (Kruskal)
            \n";
        std::cout << "  20. Show spanning tree details\n";
        std::cout << "  21. Encode spanning tree to Prufer code\n
            ";
    }
}

```

```

        std::cout << "    22. Decode last Prufer code back to a
            tree\n";
        std::cout << "    23. Maximal matching (independent edge
            set)\n";
    }
    if (!hideLab5) {
        std::cout << "\n    --- Lab 5 ---\n";
        std::cout << "    24. Build Eulerian cycle/path (modify
            graph if needed)\n";
        std::cout << "    25. Show fundamental cycle system (uses
            MST)\n";
        std::cout << "    26. Combine cycles via symmetric
            difference\n";
    }
    if (!hideCustom) {
        std::cout << "\n    --- Custom ---\n";
        std::cout << "    1001. Toggle hiding Lab 1 in menu\n";
        std::cout << "    1002. Toggle hiding Lab 2 in menu\n";
        std::cout << "    1003. Toggle hiding Lab 3 in menu\n";
        std::cout << "    1004. Toggle hiding Lab 4 in menu\n";
        std::cout << "    1005. Toggle hiding Lab 5 in menu\n";
        std::cout << "    1006. Toggle hiding Custom in menu\n";
        std::cout << "    101. Export current graph to Mermaid\n";
        std::cout << "    102. Export current flow network to
            Mermaid\n";
        std::cout << "    103. Export current spanning tree to
            Mermaid\n";
        std::cout << "    0. Quit\n";
        std::cout << "===== \n";
    }
    std::cout << "> ";
}

int readInteger(const std::string& prompt) {
    std::cout << prompt;
    int value;
    while (!(std::cin >> value)) {
        std::cin.clear();
        std::cin.ignore(std::numeric_limits<std::streamsize>::max
            (), '\n');
        std::cout << "Invalid input. " << prompt;
    }
}

```

```

    }
    return value;
}

int readNonNegativeIntegerStrict(const std::string& prompt, const
std::string& errorMessage) {
    std::cout << prompt;

    while (true) {
        std::string token;
        if (!(std::cin >> token)) {
            std::cin.clear();
            std::cin.ignore(std::numeric_limits<std::streamsize
                >::max(), '\n');
            std::cout << errorMessage;
            continue;
        }

        bool allDigits = !token.empty();
        for (unsigned char ch : token) {
            if (!std::isdigit(ch)) {
                allDigits = false;
                break;
            }
        }

        if (!allDigits) {
            std::cout << errorMessage;
            continue;
        }

        try {
            return std::stoi(token);
        } catch (const std::exception&) {
            std::cout << errorMessage;
        }
    }
}

WeightSign selectWeightSign() {
    std::cout << "\n>>> Choose weight distribution:\n";

```

```

std::cout << "  1. Positive values only\n";
std::cout << "  2. Negative values only\n";
std::cout << "  3. Mixed (positive and negative)\n";
std::cout << "> ";

int option;
while (!(std::cin >> option) || option < 1 || option > 3) {
    std::cin.clear();
    std::cin.ignore(std::numeric_limits<std::streamsize>::max
        (), '\n');
    std::cout << "Invalid. Enter 1-3: ";
}

switch (option) {
    case 1: return WeightSign::Positive;
    case 2: return WeightSign::Negative;
    case 3: return WeightSign::Mixed;
    default: return WeightSign::Positive;
}
}

void displayMatrix(const std::string& title, const WeightTable&
matrix, const std::vector<int>& vertexIds) {
    std::cout << "\n--- " << title << " ---\n";
    std::cout << "i = no path\n\n";

    // Header row
    std::cout << "      ";
    for (int id : vertexIds) {
        std::cout << std::setw(10) << id;
    }
    std::cout << "\n";

    // Separator
    std::cout << "      ";
    for (size_t i = 0; i < matrix.size(); ++i) {
        std::cout << "-----";
    }
    std::cout << "\n";

    // Data rows

```

```

    for (size_t row = 0; row < matrix.size(); ++row) {
        std::cout << std::setw(5) << vertexIds[row] << " |";
        for (size_t col = 0; col < matrix[row].size(); ++col) {
            if (matrix[row][col].has_value()) {
                std::cout << std::setw(10) << matrix[row][col].
                    value();
            } else {
                std::cout << std::setw(10) << "i";
            }
        }
        std::cout << "\n";
    }
}

void showDistributionInfo() {
    std::cout << "\n=== BINOMIAL DISTRIBUTION PARAMETERS ===\n";

    auto props = AcyclicGraphBuilder::getBinomialProperties(
        BINOMIAL_N_DEGREE, BINOMIAL_P_DEGREE);

    std::cout << "For degrees B(" << BINOMIAL_N_DEGREE << ", " <<
        BINOMIAL_P_DEGREE << "):\n";
    std::cout << "    Expected mean:    " << props.mean << "\n";
    std::cout << "    Expected var:      " << props.variance << "\n";
    std::cout << "    Expected std:      " << props.stdDev << "\n";
    std::cout << "    Mode:              " << props.mode << "\n";

    std::cout << "\nFor weights B(" << BINOMIAL_N_WEIGHT << ", "
        << BINOMIAL_P_WEIGHT << "):\n";
    std::cout << "    Expected mean:    " << (BINOMIAL_N_WEIGHT *
        BINOMIAL_P_WEIGHT) << "\n";
    std::cout << "    Expected var:      " << (BINOMIAL_N_WEIGHT *
        BINOMIAL_P_WEIGHT * (1.0 - BINOMIAL_P_WEIGHT)) << "\n";
}

void showGraphStats(const std::unique_ptr<AdjacencyGraph>& graph)
{
    if (!graph) {
        std::cout << "[!] No graph available\n";
        return;
    }
}

```

```

std::cout << "\n=== ACTUAL GRAPH STATISTICS ===\n";
auto stats = AcyclicGraphBuilder::computeDegreeStatistics(*
    graph);

std::cout << "Vertices:      " << graph->size() << "\n";
std::cout << "Edges:          " << graph->edges().size() << "
    \n";
std::cout << "Mean degree:    " << stats.meanDegree << "\n";
std::cout << "Variance:      " << stats.variance << "\n";
std::cout << "Std deviation:  " << stats.stdDev << "\n";
std::cout << "Min degree:    " << stats.minDegree << "\n";
std::cout << "Max degree:    " << stats.maxDegree << "\n";

// Compare with theory
auto props = AcyclicGraphBuilder::getBinomialProperties(
    BINOMIAL_N_DEGREE, BINOMIAL_P_DEGREE);
std::cout << "\n--- Deviation from Theory ---\n";
std::cout << "Mean error:   " << std::abs(stats.meanDegree -
    props.mean) << "\n";
std::cout << "Var error:    " << std::abs(stats.variance -
    props.variance) << "\n";
}

void showGraphDetails(const std::unique_ptr<AdjacencyGraph>&
    graph) {
    if (!graph) {
        std::cout << "[!] No graph available\n";
        return;
    }

    std::cout << "\n=== GRAPH INFORMATION ===\n";
    std::cout << "Type:          " << (graph->isDirected() ? "
        Directed" : "Undirected") << "\n";
    std::cout << "Vertices:      " << graph->size() << "\n";
    std::cout << "Edges:          " << graph->edges().size() << "\n";
    std::cout << "Vertex IDs: ";
    for (int id : graph->vertexIds()) {
        std::cout << id << " ";
    }
    std::cout << "\n";

```

```

}

void printVertexPath(const std::vector<int>& path) {
    for (size_t i = 0; i < path.size(); ++i) {
        std::cout << "v" << path[i];
        if (i + 1 < path.size()) {
            std::cout << " -> ";
        }
    }
}

void showFlowNetworkDetails(const std::unique_ptr<FlowNetwork>&
network) {
    if (!network) {
        std::cout << "[!] No flow network available\n";
        return;
    }

    std::cout << "\n=== FLOW NETWORK ===\n";
    std::cout << "Type:          " << (network->isDirected() ? "
Directed" : "Undirected") << "\n";
    std::cout << "Vertices:    " << network->size() << "\n";
    std::cout << "Edges:      " << network->edges().size() << "\n
";
    std::cout << "Vertex IDs: ";
    for (int vertexId : network->vertexIds()) {
        std::cout << vertexId << " ";
    }
    std::cout << "\n";

    std::cout << "\nEdges:\n";
    for (const FlowEdge& edge : network->edges()) {
        std::cout << "  v" << edge.from << " -> v" << edge.to
<< " | capacity = " << edge.capacity
<< ", cost = " << edge.cost
<< ", flow = " << edge.flow << "\n";
    }
}

bool sinkHasOnlyZeroCapacityIncomingEdges(const FlowNetwork&
network, int sink) {

```



```

    bool hasIncomingEdges = false;

    for (const FlowEdge& edge : network.edges()) {
        if (edge.to == sink) {
            hasIncomingEdges = true;
            if (edge.capacity > 0) {
                return false;
            }
        }
    }

    return hasIncomingEdges;
}

//
=====

// Main Program
//
=====

int main() {
    std::unique_ptr<AdjacencyGraph> currentGraph;
    std::unique_ptr<FlowNetwork> currentFlowNetwork;
    std::unique_ptr<AdjacencyGraph> currentSpanningTree;
    PruferCode lastPruferCode;
    bool hasPruferCode = false;
    std::vector<FundamentalCycle> lastFundamentalCycles; //
        cached for option 26
    int userChoice;
    int lastMaxFlow = 0;
    int lastFlowSource = -1;
    int lastFlowSink = -1;
    bool hasMaxFlowResult = false;
    bool hideLab1 = false;
    bool hideLab2 = false;
    bool hideLab3 = false;
    bool hideLab4 = false;
    bool hideLab5 = false;
    bool hideCustom = false;

```

```

do {
    showMenu(hideLab1, hideLab2, hideLab3, hideLab4, hideLab5
        , hideCustom);
    userChoice = readInteger("");

    switch (userChoice) {
        //
        =====

        case 1:  // Generate Graph
        {
            std::cout << "\n--- Graph Generation ---\n";
            int numVertices = readInteger("Number of vertices
                : ");

            std::cout << "Graph type (1=Directed, 2=
                Undirected): ";
            int typeOption;
            while (!(std::cin >> typeOption) || typeOption <
                1 || typeOption > 2) {
                std::cin.clear();
                std::cin.ignore(std::numeric_limits<std::
                    streamsize>::max(), '\n');
                std::cout << "Invalid. Enter 1 or 2: ";
            }
            bool isDirected = (typeOption == 1);

            WeightSign wSign = selectWeightSign();

            AcyclicGraphBuilder builder;
            currentGraph = builder.generateAcyclicGraph(
                numVertices, isDirected, wSign);
            currentFlowNetwork.reset();
            currentSpanningTree.reset();
            hasPruferCode = false;
            lastPruferCode = PruferCode{};
            lastFundamentalCycles.clear();
            lastMaxFlow = 0;
            lastFlowSource = -1;
            lastFlowSink = -1;

```

```

        hasMaxFlowResult = false;

        std::cout << "\n[OK] Graph created successfully!\n";
        std::cout << "    Vertices: " << numVertices << "\n";
        std::cout << "    Edges:    " << currentGraph->edges().size() << "\n";
        std::cout << "    Degree params:  n=" <<
            BINOMIAL_N_DEGREE << ", p=" <<
            BINOMIAL_P_DEGREE << "\n";
        std::cout << "    Weight params:  n=" <<
            BINOMIAL_N_WEIGHT << ", p=" <<
            BINOMIAL_P_WEIGHT << "\n";

        showDistributionInfo();
        showGraphStats(currentGraph);
        break;
    }

    //
    =====

case 2:  // Eccentricity Analysis (by edge count)
{
    if (!currentGraph) {
        std::cout << "[!] Generate a graph first\n";
        break;
    }

    std::cout << "\n--- Eccentricity Computation (by
        edge count) ---\n";
    GraphAnalyzer analyzer(*currentGraph);
    auto result = analyzer.analyze();

    std::cout << "\nVertex Eccentricities (in edges)
        :\n";
    // TO SHOW "unreachable" FOR END VERTICES:
    // Replace the loop with:
    // for (const auto& [vertex, ecc] : result.eccentricities) {

```

```

//      if (ecc >= 0) {
//          std::cout << "  v" << vertex << ": "
//          << ecc << "\n";
//      } else {
//          std::cout << "  v" << vertex << ":
//          unreachable\n";
//      }
// }
for (const auto& [vertex, ecc] : result.
    eccentricities) {
    std::cout << "  v" << vertex << ": " << ecc
        << "\n";
}

if (result.radius >= 0) {
    std::cout << "\nRadius:      " << result.
        radius << " (edges)\n";
    std::cout << "Diameter:    " << result.
        diameter << " (edges)\n";
} else {
    // TO REVERT: change this to "disconnected
    // graph" when reverting end vertices to -1
    std::cout << "\nRadius:      N/A (no vertices)
        \n";
    std::cout << "Diameter:    N/A (no vertices)\n
        ";
}

std::cout << "Center:      ";
for (int v : result.centerVertices) std::cout <<
    "v" << v << " ";
std::cout << "\n";

std::cout << "Diametrical vertices: ";
for (int v : result.diametricalVertices) std:::
    cout << "v" << v << " ";
std::cout << "\n";

// Print all diametrical paths grouped by vertex
// pair
if (!result.diametricalPathsByPair.empty()) {

```

```

size_t totalPaths = 0;
for (const auto& [pair, paths] : result.
    diametricalPathsByPair) {
    totalPaths += paths.size();
}
std::cout << "\nDiametrical paths (" <<
    totalPaths << " total):\n";

for (const auto& [pair, paths] : result.
    diametricalPathsByPair) {
    std::cout << "\n Between v" << pair.
        first << " and v" << pair.second << "
        (" << paths.size() << " paths):\n";
    for (const auto& path : paths) {
        std::cout << "      ";
        for (size_t j = 0; j < path.size();
            ++j) {
            std::cout << "v" << path[j];
            if (j < path.size() - 1) std::
                cout << " -> ";
        }
        std::cout << "\n";
    }
}
}
break;
}

//
=====

case 3: // Shimbell's Method
{
    if (!currentGraph) {
        std::cout << "[!] Generate a graph first\n";
        break;
    }

    int k = readNonNegativeIntegerStrict(
        "\nPath length k (number of edges): ",

```

```

        "[ERROR] k must be a non-negative integer
        number of edges: "
    );

    try {
        KPathCalculator calculator(*currentGraph);
        auto result = calculator.compute(k);

        auto vertexIds = currentGraph->vertexIds();
        displayMatrix("Minimum Weight Paths", result.
            minWeights, vertexIds);
        displayMatrix("Maximum Weight Paths", result.
            maxWeights, vertexIds);
    } catch (const std::exception& ex) {
        std::cout << "[ERROR] " << ex.what() << "\n";
    }
    break;
}

//
=====

case 4: // Path Counting
{
    if (!currentGraph) {
        std::cout << "[!] Generate a graph first\n";
        break;
    }

    int startVertex = readInteger("\nStart vertex: ")
        ;
    int endVertex = readInteger("End vertex: ");

    if (!currentGraph->hasVertex(startVertex) ||
        !currentGraph->hasVertex(endVertex)) {
        std::cout << "[FAIL] Invalid vertices\n";
        break;
    }

    SimplePathFinder finder(*currentGraph);

```

```

auto allPaths = finder.findAllPaths(startVertex,
    endVertex);

if (allPaths.empty()) {
    std::cout << "[FAIL] No paths found\n";
} else {
    std::cout << "[OK] Found " << allPaths.size()
        << " path(s)\n";

    // Display ALL paths
    std::cout << "Paths:\n";
    for (size_t i = 0; i < allPaths.size(); ++i)
    {
        std::cout << "  ";
        for (size_t j = 0; j < allPaths[i].size()
            ; ++j) {
            std::cout << "v" << allPaths[i][j];
            if (j < allPaths[i].size() - 1) std::
                cout << " -> ";
        }
        std::cout << "\n";
    }
    break;
}

//
=====

case 5: // Graph Details
{
    showGraphDetails(currentGraph);
    break;
}

//
=====

case 6: // Distribution Comparison
{
    showDistributionInfo();

```

```

        showGraphStats(currentGraph);
        break;
    }

    //
    =====

case 7:  // Show Adjacency Matrix
{
    if (!currentGraph) {
        std::cout << "[!] Generate a graph first\n";
        break;
    }

    std::cout << "\n=== ADJACENCY MATRIX (direct
        edges) ===\n";
    std::cout << "Shows direct edges between vertices
        (1 edge)\n";
    std::cout << "1 = edge exists, 0 = no edge\n";
    auto matrix = currentGraph->getAdjacencyMatrix();
    printAdjacencyMatrix(std::cout, matrix,
        currentGraph->vertexIds());
    break;
}

    //
    =====

case 8:  // Show Weight Matrix (Shimbell k=1)
{
    if (!currentGraph) {
        std::cout << "[!] Generate a graph first\n";
        break;
    }

    std::cout << "\n=== WEIGHT MATRIX (Shimbell, k=1)
        ===\n";
    std::cout << "Minimum weight paths with EXACTLY 1
        edge\n";
    std::cout << "i = no path with exactly 1 edge\n";
    try {

```



```

        KPathCalculator calculator(*currentGraph);
        auto result = calculator.compute(1);
        auto vertexIds = currentGraph->vertexIds();
        printWeightTable(std::cout, result.minWeights
            , vertexIds);
    } catch (const std::exception& ex) {
        std::cout << "[ERROR] " << ex.what() << "\n";
    }
    break;
}

//
=====

case 9:  // BFS Traversal (Lab 2)
{
    if (!currentGraph) {
        std::cout << "[!] Generate a graph first\n";
        break;
    }

    std::cout << "\n--- BFS Traversal ---\n";
    int startVertex = readInteger("Start vertex: ");

    if (!currentGraph->hasVertex(startVertex)) {
        std::cout << "[!] Invalid vertex\n";
        break;
    }

    BreadthFirstSearch bfs(*currentGraph);
    BFSResult result = bfs.traverseWithLevels(
        startVertex);

    std::cout << "\n[OK] BFS completed in " << result
        .iterations << " iterations\n";
    std::cout << "Maximum depth (levels): " << result
        .maxLevel << "\n";

    // Display Levels
    std::cout << "\n--- BFS Levels ---\n";

```

```

    for (int level = 0; level <= result.maxLevel; ++
        level) {
        std::cout << "Level " << level << ": [";
        const auto& vertices = result.levels.at(level
        );
        for (size_t i = 0; i < vertices.size(); ++i)
        {
            std::cout << vertices[i];
            if (i < vertices.size() - 1) std::cout <<
                ", ";
        }
        std::cout << "]\n";
    }

    // Check if graph is fully connected
    /*
    if (result.traversalOrder.size() < static_cast<
        size_t>(currentGraph->size())) {
        std::cout << "\n[!] Warning: Graph may be
            disconnected\n";
        std::cout << "    Visited " << result.
            traversalOrder.size()
                << " of " << currentGraph->size()
                << " vertices\n";
    }

        */
    break;
}

//
=====

case 10: // Dijkstra's Algorithm (Lab 2)
{
    if (!currentGraph) {
        std::cout << "[!] Generate a graph first\n";
        break;
    }

    std::cout << "\n--- Dijkstra's Shortest Path ---\n";
}

```

```

int startVertex = readInteger("Start vertex: ");
int endVertex = readInteger("End vertex: ");

if (!currentGraph->hasVertex(startVertex) ||
    !currentGraph->hasVertex(endVertex)) {
    std::cout << "[!] Invalid vertices\n";
    break;
}

DijkstraAlgorithm dijkstra(*currentGraph);
DijkstraResult result = dijkstra.
    findShortestPaths(startVertex, endVertex);

std::cout << "\n[OK] Dijkstra completed in " <<
    result.iterations << " iterations\n";

if (result.shortestPath.empty() || result.
    targetDistance == std::numeric_limits<double
    >::infinity()) {
    std::cout << "[!] No path found from v" <<
        startVertex << " to v" << endVertex << "\n
    ";
} else {
    std::cout << "Shortest distance: " << result.
        targetDistance << "\n";
    std::cout << "Path: ";
    for (size_t i = 0; i < result.shortestPath.
        size(); ++i) {
        std::cout << "v" << result.shortestPath[i
        ];
        if (i < result.shortestPath.size() - 1)
            std::cout << " -> ";
    }
    std::cout << "\n";
}

// Show distance vector
std::cout << "\n--- Distance Vector from v" <<
    startVertex << " ---\n";
for (const auto& [vertex, dist] : result.
    distances) {

```

```

        if (dist == std::numeric_limits<double>::
            infinity()) {
            std::cout << "  v" << vertex << ":
                unreachable\n";
        } else {
            std::cout << "  v" << vertex << ": " <<
                dist << "\n";
        }
    }
    break;
}

//
=====

case 11:  // Compare BFS vs Dijkstra (Lab 2)
{
    if (!currentGraph) {
        std::cout << "[!] Generate a graph first\n";
        break;
    }

    std::cout << "\n--- Algorithm Speed Comparison
        ---\n";
    int startVertex = readInteger("Start vertex: ");

    if (!currentGraph->hasVertex(startVertex)) {
        std::cout << "[!] Invalid vertex\n";
        break;
    }

    AlgorithmComparator comparator(*currentGraph);
    AlgorithmComparison comparison = comparator.
        compare(startVertex);

    std::cout << "\n=== Comparison Results ===\n";
    std::cout << "BFS iterations:          " <<
        comparison.bfsIterations << "\n";
    std::cout << "Dijkstra iterations:  " <<
        comparison.dijkstraIterations << "\n";

```

```

    if (comparison.bfsIterations > 0) {
        std::cout << "BFS is " << std::fixed << std::
            setprecision(2)
                << comparison.speedupFactor << "x
                    times faster than Dijkstra.\n";
    }

    // Reset output formatting to default
    std::cout << std::resetiosflags(std::ios_base::
        floatfield) << std::setprecision(6);

    /*
    std::cout << "\n--- Analysis ---\n";
    if (comparison.bfsIterations < comparison.
        dijkstraIterations) {
        std::cout << "BFS is faster for simple
            traversal (no weights).\n";
        std::cout << "Dijkstra requires more
            iterations due to priority queue
            operations.\n";
    } else if (comparison.bfsIterations > comparison.
        dijkstraIterations) {
        std::cout << "Unusual: Dijkstra appears
            faster. This may occur on sparse graphs.\n
                ";
    } else {
        std::cout << "Both algorithms required the
            same number of iterations.\n";
    }
    */
    break;
}

//
=====

case 12: // Build Flow Network
{
    if (!currentGraph) {
        std::cout << "[!] Generate a graph first\n";
        break;
    }
}

```

```

    }
    if (!currentGraph->isDirected()) {
        std::cout << "[!] Lab 3 requires a directed
            graph. Generate a directed graph first.\n"
            ;
        break;
    }

    FlowNetworkBuilder builder;
    currentFlowNetwork = builder.buildFromGraph(*
        currentGraph);
    lastMaxFlow = 0;
    lastFlowSource = -1;
    lastFlowSink = -1;
    hasMaxFlowResult = false;

    std::cout << "\n[OK] Flow network built from the
        current graph.\n";
    std::cout << "    Capacity(u, v) is generated
        randomly via the program generator\n";
    std::cout << "    Cost(u, v) is generated
        randomly via the program generator\n";
    std::cout << "    Vertices: " <<
        currentFlowNetwork->size() << "\n";
    std::cout << "    Edges:    " <<
        currentFlowNetwork->edges().size() << "\n";
    break;
}

//
=====

case 13: // Show Capacity Matrix
{
    if (!currentFlowNetwork) {
        std::cout << "[!] Build the flow network
            first\n";
        break;
    }

    std::cout << "\n=== CAPACITY MATRIX ===\n";

```

```

        std::cout << "i = no directed edge\n";
        printFlowMatrix(std::cout,
                        currentFlowNetwork->
                            getCapacityMatrix(),
                        currentFlowNetwork->vertexIds());
        break;
    }

    //
    =====

case 14:  // Show Cost Matrix
{
    if (!currentFlowNetwork) {
        std::cout << "[!] Build the flow network
            first\n";
        break;
    }

    std::cout << "\n=== COST MATRIX ===\n";
    std::cout << "i = no directed edge\n";
    printFlowMatrix(std::cout,
                    currentFlowNetwork->getCostMatrix
                        (),
                    currentFlowNetwork->vertexIds());

    break;
}

//
=====

case 15:  // Maximum Flow
{
    if (!currentFlowNetwork) {
        std::cout << "[!] Build the flow network
            first\n";
        break;
    }

    std::cout << "\n--- Maximum Flow (Ford-Fulkerson)
        ---\n";

```

```

int source = readInteger("Source vertex: ");
int sink = readInteger("Sink vertex: ");

if (!currentFlowNetwork->hasVertex(source) ||
    !currentFlowNetwork->hasVertex(sink) ||
    source == sink) {
    std::cout << "[!] Invalid source/sink pair\n"
        ;
    break;
}

MaxFlowSolver solver(*currentFlowNetwork);
MaxFlowResult result = solver.compute(source,
    sink);

lastMaxFlow = result.maxFlow;
lastFlowSource = source;
lastFlowSink = sink;
hasMaxFlowResult = true;

std::cout << "\n[OK] Maximum flow = " << result.
    maxFlow << "\n";
if (result.steps.empty()) {
    if (sinkHasOnlyZeroCapacityIncomingEdges(*
        currentFlowNetwork, sink)) {
        std::cout << "[!] No paths from source to
            sink: all edges leading to the sink
            have capacity = 0\n";
    } else {
        std::cout << "[!] No augmenting path
            found\n";
    }
} else {
    std::cout << "\nAugmenting steps:\n";
    for (const MaxFlowStep& step : result.steps)
    {
        std::cout << "  Step " << step.iteration
            << ": ";
        printVertexPath(step.path);
        std::cout << " | added flow = " << step.
            pathFlow

```



```

        << " | total = " << step.
        totalFlow << "\n";
    }
}

std::cout << "\nFlow matrix after max flow:\n";
printFlowMatrix(std::cout,
                currentFlowNetwork->getFlowMatrix
                (),
                currentFlowNetwork->vertexIds());
break;
}

//
=====

case 16: // Minimum-Cost Flow
{
    if (!currentFlowNetwork) {
        std::cout << "[!] Build the flow network
            first\n";
        break;
    }
    if (!hasMaxFlowResult) {
        std::cout << "[!] Compute maximum flow first\n";
        break;
    }

    const int targetFlow = (2 * lastMaxFlow) / 3;

    std::cout << "\n--- Minimum-Cost Flow ---\n";
    std::cout << "Using the same source/sink as the
        maximum-flow run: "
        << "v" << lastFlowSource << " -> v" <<
        lastFlowSink << "\n";
    std::cout << "Target flow = floor(2/3 * " <<
        lastMaxFlow
        << ") = " << targetFlow << "\n";

    MinCostFlowSolver solver(*currentFlowNetwork);

```

```

MinCostFlowResult result = solver.compute(
    lastFlowSource, lastFlowSink, targetFlow);

std::cout << "\n=== Result ===\n";
std::cout << "Target flow:      " << result.
    targetFlow << "\n";
std::cout << "Achieved flow:    " << result.
    achievedFlow << "\n";
std::cout << "Total cost:        " << result.
    totalCost << "\n";
std::cout << "Success:           " << (result.
    success ? "yes" : "no") << "\n";

if (result.achievedFlow == 0 &&
    sinkHasOnlyZeroCapacityIncomingEdges(*
        currentFlowNetwork, lastFlowSink)) {
    std::cout << "[!] No paths from source to
        sink: all edges leading to the sink have
        capacity = 0\n";
}

if (!result.steps.empty()) {
    std::cout << "\nAugmenting steps:\n";
    for (const MinCostFlowStep& step : result.
        steps) {
        std::cout << "  Step " << step.iteration
            << ": ";
        printVertexPath(step.path);
        std::cout << "\n    added flow:      "
            << step.pathFlow
                << "\n    path unit cost:    "
                << step.pathUnitCost
                << "\n    cumulative flow:  "
                << step.cumulativeFlow
                << "\n    cumulative cost:  "
                << step.cumulativeCost
                << "\n";
    }
}

```

```

        std::cout << "\nFlow matrix after minimum-cost
            flow:\n";
        printFlowMatrix(std::cout,
                        currentFlowNetwork->getFlowMatrix
                            (),
                        currentFlowNetwork->vertexIds());
        break;
    }

    //
    =====

    case 17: // Flow Network Details
    {
        showFlowNetworkDetails(currentFlowNetwork);
        break;
    }

    //
    =====

    case 18: // Count spanning trees (Kirchhoff)
    {
        if (!currentGraph) {
            std::cout << "[!] Generate a graph first\n";
            break;
        }
        if (currentGraph->isDirected()) {
            std::cout << "[!] Lab 4 requires an
                undirected graph. "
                    "Generate an undirected graph
                    first.\n";
            break;
        }

        KirchhoffCounter counter(*currentGraph);
        KirchhoffResult result = counter.compute();

        std::cout << "\n=== KIRCHHOFF MATRIX (B = D - A)
            ===\n";
        std::cout << "Diagonal: degree of vertex.\n";
    }

```

```

std::cout << "Off-diagonal: -1 if edge exists, 0
otherwise.\n\n";

const auto vertexIds = currentGraph->vertexIds();
const int n = currentGraph->size();

std::cout << "
";
for (int id : vertexIds) std::cout << std::setw
(6) << id;
std::cout << "\n
";
for (int i = 0; i < n; ++i) std::cout << "-----"
;
std::cout << "\n";
for (int i = 0; i < n; ++i) {
    std::cout << std::setw(5) << vertexIds[i] <<
" |";
    for (int j = 0; j < n; ++j) {
        std::cout << std::setw(6) << result.
kirchhoffMatrix[i][j];
    }
    std::cout << "\n";
}

std::cout << "\n[OK] Number of spanning trees = "
<< result.spanningTreeCount << "\n";
if (result.spanningTreeCount == 0 && n > 1) {
    std::cout << "[!] Zero spanning trees -- the
graph is disconnected.\n";
}
break;
}

//
=====

case 19: // Build MST (Kruskal)
{
    if (!currentGraph) {
        std::cout << "[!] Generate a graph first\n";
        break;
    }
}

```

```

if (currentGraph->isDirected()) {
    std::cout << "[!] Lab 4 requires an
        undirected graph. "
        "Generate an undirected graph
            first.\n";

    break;
}

KruskalMST kruskal(*currentGraph);
KruskalResult result = kruskal.buildMST();

std::cout << "\n=== KRUSKAL'S ALGORITHM ===\n";
std::cout << "Edges added (in order, sorted by
    ascending weight):\n";
for (size_t i = 0; i < result.chosenEdges.size();
    ++i) {
    const auto& e = result.chosenEdges[i];
    std::cout << "    " << (i + 1) << ". v" << e.
        from
            << " --- v" << e.to
            << " (weight = " << e.weight << ")
                \n";
}

std::cout << "\nTotal weight of MST: " << result.
    totalWeight << "\n";
std::cout << "Edges in MST:          " << result.
    chosenEdges.size()
        << " of " << (currentGraph->size() - 1)
        << " expected\n";

if (!result.wasConnected) {
    std::cout << "[!] Graph was disconnected --
        result is a "
            "spanning forest, not a tree.\n"
                ;
}

currentSpanningTree = std::move(result.
    spanningTree);

```

```

        hasPruferCode = false; // tree has changed,
                               invalidate old code
        lastPruferCode = PruferCode{};
        lastFundamentalCycles.clear(); // cycles depend
                                         on the tree
        std::cout << "\n[OK] Spanning tree saved as the
                      current MST.\n";
        break;
    }

    //
    =====

case 20: // Show MST details
{
    if (!currentSpanningTree) {
        std::cout << "[!] Build the spanning tree
                      first (option 19)\n";
        break;
    }

    std::cout << "\n=== SPANNING TREE ===\n";
    std::cout << "Type:      "
               << (currentSpanningTree->isDirected() ?
                    "Directed" : "Undirected")
               << "\n";
    std::cout << "Vertices: " << currentSpanningTree
               ->size() << "\n";
    std::cout << "Edges:      " << currentSpanningTree
               ->edges().size() << "\n";

    double totalWeight = 0.0;
    std::cout << "\nEdges:\n";
    for (const WeightedEdge& edge :
         currentSpanningTree->edges()) {
        std::cout << "  v" << edge.from << " --- v"
                   << edge.to
                   << " (weight = " << edge.weight <<
                   ")\n";
        totalWeight += edge.weight;
    }
}

```

```

        std::cout << "\nTotal weight: " << totalWeight <<
            "\n";
        break;
    }

    //
    =====

case 21: // Encode tree to Prufer code
{
    if (!currentSpanningTree) {
        std::cout << "[!] Build the spanning tree
            first (option 19)\n";
        break;
    }

    lastPruferCode = PruferEncoder::encode(*
        currentSpanningTree);
    hasPruferCode = true;

    const int p = currentSpanningTree->size();
    std::cout << "\n=== PRUFER CODE ===\n";
    std::cout << "Tree has " << p << " vertices, code
        length = p - 1 = "
        << (p - 1) << "\n\n";

    std::cout << "Code:   [";
    for (size_t i = 0; i < lastPruferCode.code.size()
        ; ++i) {
        std::cout << lastPruferCode.code[i];
        if (i + 1 < lastPruferCode.code.size()) std::
            cout << ", ";
    }
    std::cout << "]\n";

    std::cout << "Weights: [";
    for (size_t i = 0; i < lastPruferCode.weights.
        size(); ++i) {
        std::cout << lastPruferCode.weights[i];
        if (i + 1 < lastPruferCode.weights.size())
            std::cout << ", ";
    }

```

```

    }
    std::cout << "]\n";

    std::cout << "\n(Step i: removed smallest leaf ->
        recorded its "
                "neighbor in code[i] and the edge
                weight in weights[i])\n";
    break;
}

//
=====

case 22: // Decode Last Prufer code
{
    if (!hasPruferCode) {
        std::cout << "[!] Encode a tree first (option
            21)\n";
        break;
    }
    if (!currentSpanningTree) {
        std::cout << "[!] Need the original tree to
            know its vertex set\n";
        break;
    }

    auto decoded = PruferEncoder::decode(
        lastPruferCode, currentSpanningTree->
            vertexIds());

    std::cout << "\n=== DECODED TREE ===\n";
    std::cout << "Vertices: " << decoded->size() << "
        \n";
    std::cout << "Edges:      " << decoded->edges().
        size() << "\n\n";

    std::cout << "Edges:\n";
    for (const WeightedEdge& edge : decoded->edges())
    {
        std::cout << "  v" << edge.from << " --- v"
            << edge.to

```



```

        << " (weight = " << edge.weight <<
            ")\n";
    }

    // Round-trip check: decoded edges should match
    // the original tree.
    auto originalEdges = currentSpanningTree->edges()
        ;
    auto decodedEdges = decoded->edges();

    auto edgeKey = [](const WeightedEdge& e) {
        int a = e.from, b = e.to;
        if (a > b) std::swap(a, b);
        return std::make_tuple(a, b, e.weight);
    };

    std::vector<std::tuple<int, int, double>>
        originalKeys, decodedKeys;
    for (const auto& e : originalEdges) originalKeys.
        push_back(edgeKey(e));
    for (const auto& e : decodedEdges) decodedKeys.
        push_back(edgeKey(e));
    std::sort(originalKeys.begin(), originalKeys.end
        ());
    std::sort(decodedKeys.begin(), decodedKeys.end()
        );

    if (originalKeys == decodedKeys) {
        std::cout << "\n[OK] Round-trip successful:
            decoded tree "
                "matches the original (edges +
                    weights).\n";
    } else {
        std::cout << "\n[!] Round-trip MISMATCH
            between original and decoded tree.\n";
    }
    break;
}

//
=====

```

```

case 23: // Maximal matching
{
    if (!currentGraph) {
        std::cout << "[!] Generate a graph first\n";
        break;
    }
    if (currentGraph->isDirected()) {
        std::cout << "[!] Lab 4 requires an
            undirected graph. "
            "Generate an undirected graph
            first.\n";
        break;
    }

    std::cout << "\n--- Maximal Matching (independent
        edge set) ---\n";
    std::cout << "Run on:\n";
    std::cout << "  1. Original graph\n";
    std::cout << "  2. Spanning tree (option 19 must
        be done first)\n";
    std::cout << "> ";

    int targetOption;
    while (!(std::cin >> targetOption) ||
        (targetOption != 1 && targetOption != 2))
    {
        std::cin.clear();
        std::cin.ignore(std::numeric_limits<std::
            streamsize>::max(), '\n');
        std::cout << "Invalid. Enter 1 or 2: ";
    }

    const AdjacencyGraph* target = nullptr;
    std::string targetName;
    if (targetOption == 1) {
        target = currentGraph.get();
        targetName = "original graph";
    } else {
        if (!currentSpanningTree) {

```

```

        std::cout << "[!] Build the spanning tree
            first (option 19)\n";
        break;
    }
    target = currentSpanningTree.get();
    targetName = "spanning tree";
}

// Choose algorithm.
std::cout << "\nAlgorithm:\n";
std::cout << "  1. Greedy (maximal by inclusion
    -- per lecture definition)\n";
std::cout << "  2. Edmonds blossom (guaranteed
    MAXIMUM cardinality)\n";
std::cout << "> ";

int algoOption;
while (!(std::cin >> algoOption) ||
    (algoOption != 1 && algoOption != 2)) {
    std::cin.clear();
    std::cin.ignore(std::numeric_limits<std::
        streamsize>::max(), '\n');
    std::cout << "Invalid. Enter 1 or 2: ";
}

MatchingResult result;
std::string algoName;
if (algoOption == 1) {
    GreedyMaximalMatching matcher(*target);
    result = matcher.compute();
    algoName = "GREEDY MAXIMAL";
} else {
    EdmondsMatching matcher(*target);
    result = matcher.compute();
    algoName = "EDMONDS MAXIMUM";
}

std::cout << "\n=== " << algoName << " MATCHING
    on "
        << targetName << " ===\n";

```

```

std::cout << "Matching size: " << result.
    matchingSize << "\n";

std::cout << "\nMatching edges:\n";
if (result.matchingEdges.empty()) {
    std::cout << "  (none)\n";
} else {
    for (const WeightedEdge& edge : result.
        matchingEdges) {
        std::cout << "  v" << edge.from << " ---
            v" << edge.to
                << "  (weight = " << edge.
                    weight << ")\n";
    }
}

if (result.unmatchedVertices.empty()) {
    std::cout << "\n[OK] Matching is PERFECT --
        every vertex is covered.\n";
} else {
    std::cout << "\nUnmatched vertices: ";
    for (int v : result.unmatchedVertices) std::
        cout << "v" << v << " ";
    std::cout << "\n";
}
break;
}

//
=====

case 24: // Build Eulerian cycle/path
{
    if (!currentGraph) {
        std::cout << "[!] Generate a graph first\n";
        break;
    }
    if (currentGraph->isDirected()) {
        std::cout << "[!] Lab 5 (Eulerian cycle/path)
            requires an undirected graph.\n";
        break;
    }
}

```

```

}

EulerianCycleBuilder builder(*currentGraph);
std::cout << "\n=== EULERIAN TRAVERSAL ===\n";
EulerianCycleResult naturalTraversal =
    builder.compute(EulerizationMode::
        NonMultigraphOnly);
const bool originalWasSemiEulerian =
    naturalTraversal.success &&
    naturalTraversal.wasSemiEulerian &&
    naturalTraversal.additions.empty();

EulerianCycleResult er = originalWasSemiEulerian
    ? builder.compute(EulerizationMode::
        NonMultigraphOnly, true)
    : builder.compute(EulerizationMode::
        NonMultigraphOnly);

if (originalWasSemiEulerian) {
    if (!er.success) {
        std::cout << "[!] Graph is semi-Eulerian,
            but it could not be\n"
                "        transformed into an
                    Eulerian graph in simple-
                    graph mode.\n";

        while (!er.success) {
            std::cout << "Choose how to continue
                :\n";
            std::cout << "  a - retry conversion
                by adding edges\n";
            std::cout << "  d - try deleting
                existing edges\n";
            std::cout << "  m - allow multigraph
                conversion\n";
            std::cout << "  p - leave it semi-
                Eulerian\n";
            std::cout << "> ";

            std::string answer;
            std::cin >> answer;

```

```

if (answer.empty()) continue;

const char choice = static_cast<char
    >(std::tolower(
        static_cast<unsigned char>(answer
            [0])));

if (choice == 'a') {
    er = builder.compute(
        EulerizationMode::
        NonMultigraphOnly, true);
    if (er.success) {
        std::cout << "[i] Proceeding
            with edge-addition
            eulerization.\n";
    } else {
        std::cout << "[!] Still
            cannot make the graph
            Eulerian by adding\n"
                "edges
                without
                duplicating
                an existing
                edge.\n";
    }
    continue;
}

if (choice == 'd') {
    er = builder.compute(
        EulerizationMode::
        DeleteEdgesOnly, true);
    if (er.success) {
        std::cout << "[i] Proceeding
            with edge-deletion
            eulerization.\n";
    } else {
        std::cout << "[!] Could not
            make the graph Eulerian by
            deleting\n"

```

```

                                "    edges
                                without
                                breaking the
                                edge-bearing
                                connectivity
                                .\n";
                                }
                                continue;
                                }

if (choice == 'm') {
    er = builder.compute(
        EulerizationMode::
        AllowMultigraph, true);
    if (!er.success) {
        std::cout << "[!] Unexpected
            failure while building\n"
            "    the
            Eulerian
            cycle.\n";

        break;
    }
    std::cout << "[i] Proceeding with
        multigraph eulerization.\n";
    continue;
}

if (choice == 'p') {
    er = std::move(naturalTraversal);
    std::cout << "[i] Leaving the
        graph semi-Eulerian.\n";
    break;
}

std::cout << "Invalid option. Choose
    a, d, m, or p.\n";
}
}
} else if (!er.success && er.requiresMultigraph)
{

```

```

std::cout << "[!] Cannot make this graph
Eulerian without\n"
    "    duplicating an existing
    edge -- the only\n"
    "    possible eulerization turns
    it into a\n"
    "    multigraph unless we remove
    some edges.\n";

while (!er.success) {
    std::cout << "Choose how to continue:\n";
    std::cout << "    d - try deleting existing
    edges\n";
    std::cout << "    m - allow multigraph
    conversion\n";
    std::cout << "    n - abort\n";
    std::cout << "> ";

    std::string answer;
    std::cin >> answer;
    if (answer.empty()) continue;

    const char choice = static_cast<char>(std
::tolower(
    static_cast<unsigned char>(answer[0])
));

    if (choice == 'n') {
        std::cout << "[i] Aborted -- no
        Eulerian cycle was built.\n";
        break;
    }

    if (choice == 'd') {
        er = builder.compute(EulerizationMode
::DeleteEdgesOnly, true);
        if (er.success) {
            std::cout << "[i] Proceeding with
            edge-deletion eulerization.\n
            ";
        } else {

```



```

        std::cout << "[!] Could not make
            the graph Eulerian by deleting
            \n"
            "    edges without
            breaking the edge
            -bearing
            connectivity.\n";
    }
    continue;
}

if (choice == 'm') {
    er = builder.compute(EulerizationMode
        ::AllowMultigraph, true);
    if (!er.success) {
        std::cout << "[!] Unexpected
            failure while building\n"
            "    the Eulerian
            cycle.\n";

        break;
    }
    std::cout << "[i] Proceeding with
        multigraph eulerization.\n";
    continue;
}

std::cout << "Invalid option. Choose d, m
    , or n.\n";
}

if (!er.success) {
    break;
}

if (!er.success) {
    std::cout << "[!] Cannot build Eulerian
        traversal (graph has no edges).\n";
    break;
}

```

```

if (er.wasAlreadyEulerian) {
    std::cout << "[OK] Graph is already Eulerian
        "
                "(connected + all degrees even)
                .\n";
} else if (er.wasSemiEulerian) {
    std::cout << "[OK] Graph is semi-Eulerian "
                "(connected + exactly two odd-
                degree vertices).\n";
    if (!er.traversal.empty()) {
        std::cout << "Start: v" << er.traversal.
            front()
                    << ", end: v" << er.traversal.
            back() << "\n";
    }
} else {
    if (originalWasSemiEulerian) {
        std::cout << "[OK] Graph was semi-
            Eulerian and was transformed\n"
                    "        into an Eulerian graph
                    .\n";
    } else {
        std::cout << "[!] Graph was NOT Eulerian.
            Modifications applied:\n";
    }
    for (size_t i = 0; i < er.additions.size();
        ++i) {
        const auto& a = er.additions[i];
        std::cout << "    " << (i + 1) << ". "
                    << (a.added ? "added edge v" :
                        "removed edge v")
                    << a.from
                    << " --- v" << a.to
                    << " [" << a.reason << "]\n";
    }
    std::cout << "Total modifications: " << er.
        additions.size() << "\n";
}

const bool isCycle = er.traversalKind ==
    EulerTraversalKind::Cycle;

```

```

std::cout << "\nEulerian " << (isCycle ? "cycle"
    : "path")
    << " (" << (er.traversal.size() - 1)
    << " edges, " << er.traversal.size() <<
        " vertex stops):\n";
std::cout << " ";
for (size_t i = 0; i < er.traversal.size(); ++i)
{
    std::cout << "v" << er.traversal[i];
    if (i + 1 < er.traversal.size()) std::cout <<
        " -> ";
    // Wrap long lines for readability.
    if ((i + 1) % 12 == 0 && i + 1 < er.traversal
        .size()) {
        std::cout << "\n ";
    }
}
std::cout << "\n";
break;
}

//
=====

case 25: // Fundamental cycle system
{
    if (!currentGraph) {
        std::cout << "[!] Generate a graph first\n";
        break;
    }
    if (currentGraph->isDirected()) {
        std::cout << "[!] Lab 5 requires an
            undirected graph.\n";
        break;
    }
    if (!currentSpanningTree) {
        std::cout << "[!] Build the spanning tree
            first (option 19)\n";
        break;
    }
}

```

```

FundamentalCycleSystem system(*currentGraph, *
    currentSpanningTree);
lastFundamentalCycles = system.compute();

std::cout << "\n=== FUNDAMENTAL CYCLE SYSTEM ===\n";
std::cout << "One fundamental cycle per chord (
    non-tree edge).\n";
std::cout << "Total cycles: " <<
    lastFundamentalCycles.size()
    << " (= |E| - |V| + 1 for a connected
        graph).\n";

if (lastFundamentalCycles.empty()) {
    std::cout << "[!] No chords -- the graph is
        itself a tree, "
        "no cycles exist.\n";
    break;
}

for (size_t i = 0; i < lastFundamentalCycles.size
    ()); ++i) {
    const FundamentalCycle& cycle =
        lastFundamentalCycles[i];
    std::cout << "\n  C" << (i + 1)
        << " (chord v" << cycle.chordFrom
        << " --- v" << cycle.chordTo << ")
        :\n";

    std::cout << "    Vertex sequence: ";
    for (size_t j = 0; j < cycle.vertexSequence.
        size(); ++j) {
        std::cout << "v" << cycle.vertexSequence[
            j];
        if (j + 1 < cycle.vertexSequence.size())
            std::cout << " -> ";
    }
    std::cout << "\n";

    std::cout << "    Edges (" << cycle.edges.
        size() << "): ";

```

```

        for (size_t j = 0; j < cycle.edges.size(); ++
            j) {
            std::cout << "(v" << cycle.edges[j].first
                << ",v" << cycle.edges[j].
                    second << ")";
            if (j + 1 < cycle.edges.size()) std::cout
                << ", ";
        }
        std::cout << "\n";
    }
    break;
}

//
=====

case 26: // Combine cycles via symmetric difference
{
    if (lastFundamentalCycles.empty()) {
        std::cout << "[!] Build the fundamental cycle
            system first (option 25)\n";
        break;
    }

    std::cout << "\n--- Symmetric Difference of
        Cycles ---\n";
    std::cout << "Available fundamental cycles: 1.."
        << lastFundamentalCycles.size() << "\n"
        ;
    std::cout << "Enter cycle numbers separated by
        spaces and press Enter.\n";
    std::cout << "> ";

    std::vector<int> picks;
    if (std::cin.peek() == '\n') {
        std::cin.get();
    }
    while (true) {
        picks.clear();
        std::string line;
        std::getline(std::cin, line);

```

```

std::istringstream input(line);
std::string token;
bool badInput = false;

while (input >> token) {
    int x = 0;
    try {
        size_t pos = 0;
        x = std::stoi(token, &pos);
        if (pos != token.size()) {
            throw std::invalid_argument("
                trailing characters");
        }
    } catch (const std::exception&) {
        std::cout << "Invalid token. Enter
            the cycle numbers again:\n> ";
        badInput = true;
        break;
    }

    if (x < 1 || x > static_cast<int>(
        lastFundamentalCycles.size())) {
        std::cout << "[!] Cycle C" << x << "
            does not exist. "
                "Enter the cycle numbers
                again:\n> ";
        badInput = true;
        break;
    }
    picks.push_back(x);
}

if (!badInput) break;
}

std::vector<std::pair<int,int>> acc;
if (picks.empty()) {
    std::cout << "\nResult of empty input:\n";
} else {
    // Apply symmetric difference left-to-right.

```

```

// Since this operation is associative, the
// final edge set
// does not depend on how the expression is
// parenthesized.
acc = lastFundamentalCycles[picks[0] - 1].
    edges;
for (size_t i = 1; i < picks.size(); ++i) {
    acc = FundamentalCycleSystem::
        symmetricDifference(
            acc, lastFundamentalCycles[picks[i] -
                1].edges);
}

std::cout << "\nResult of C" << picks[0];
for (size_t i = 1; i < picks.size(); ++i) {
    std::cout << " /_\ C" << picks[i];
}
std::cout << ":\n";
}
std::cout << "  Edges (" << acc.size() << "): ";
if (acc.empty()) {
    std::cout << "(empty set of edges)";
} else {
    for (size_t j = 0; j < acc.size(); ++j) {
        std::cout << "(v" << acc[j].first
            << ",v" << acc[j].second << ")"
            ;
        if (j + 1 < acc.size()) std::cout << ", "
            ;
    }
}
std::cout << "\n";

if (!acc.empty()) {
    auto cycles = FundamentalCycleSystem::
        decomposeIntoCycles(acc);
    if (cycles.empty()) {
        std::cout << "  Could not reconstruct
            simple cycles from the edge set.\n";
        std::cout << "  The result is still an
            even subgraph, but not represented\n";
    }
}

```

```

        std::cout << " here as a clean cycle
            decomposition.\n";
    } else if (cycles.size() == 1) {
        std::cout << " Reconstructed cycle: ";
        for (size_t i = 0; i < cycles[0].size();
            ++i) {
            std::cout << "v" << cycles[0][i];
            if (i + 1 < cycles[0].size()) std::
                cout << " -> ";
        }
        std::cout << "\n";
    } else {
        std::cout << " Decomposition into cycles
            :\n";
        for (size_t i = 0; i < cycles.size(); ++i
        ) {
            std::cout << "      " << (i + 1) << ".
                ";
            for (size_t j = 0; j < cycles[i].size
                ()); ++j) {
                std::cout << "v" << cycles[i][j];
                if (j + 1 < cycles[i].size()) std
                    ::cout << " -> ";
            }
            std::cout << "\n";
        }
    }
}

// std::cout << "(Note: after symmetric
// difference the result is always an\n";
// std::cout << " even subgraph. It may be one
// cycle or a union of several\n";
// std::cout << " cycles, so we reconstruct as
// much as the edge set allows.)\n";
break;
}

//
=====

```



```

case 1001: // Toggle hiding legacy sections
{
    hideLab1 = !hideLab1;
    std::cout << (hideLab1
        ? "[OK] Lab 1 is now hidden in the menu.\n"
        : "[OK] Lab 1 is now shown in the menu.\n");
    break;
}

case 1002: // Toggle hiding legacy sections
{
    hideLab2 = !hideLab2;
    std::cout << (hideLab2
        ? "[OK] Lab 2 is now hidden in the menu.\n"
        : "[OK] Lab 2 is now shown in the menu.\n");
    break;
}

case 1003: // Toggle hiding legacy sections
{
    hideLab3 = !hideLab3;
    std::cout << (hideLab3
        ? "[OK] Lab 3 is now hidden in the menu.\n"
        : "[OK] Lab 3 is now shown in the menu.\n");
    break;
}

case 1004: // Toggle hiding legacy sections
{
    hideLab4 = !hideLab4;
    std::cout << (hideLab4
        ? "[OK] Lab 4 is now hidden in the menu.\n"
        : "[OK] Lab 4 is now shown in the menu.\n");
    break;
}

case 1005: // Toggle hiding legacy sections
{
    hideLab5 = !hideLab5;
    std::cout << (hideLab5
        ? "[OK] Lab 5 is now hidden in the menu.\n"

```

```

        : "[OK] Lab 5 is now shown in the menu.\n");
    break;
}

case 1006: // Toggle hiding legacy sections
{
    hideCustom = !hideCustom;
    std::cout << (hideCustom
        ? "[OK] Custom menu is now hidden in the menu
        .\n"
        : "[OK] Custom menu is now shown in the menu
        .\n");
    break;
}

//
=====

case 102: // Export Flow Network Mermaid
{
    if (!currentFlowNetwork) {
        std::cout << "[!] Build the flow network
        first\n";
        break;
    }

    try {
        const auto outputPath = MermaidExporter::
            exportFlowNetwork(*currentFlowNetwork);
        std::cout << "\n[OK] Flow-network Mermaid
            code written to:\n"
            << "      " << outputPath.string() <<
            "\n";
    } catch (const std::exception& ex) {
        std::cout << "[ERROR] " << ex.what() << "\n";
    }
    break;
}

//
=====

```

```

case 101:  // Export Graph Mermaid
{
    if (!currentGraph) {
        std::cout << "[!] Generate a graph first\n";
        break;
    }

    try {
        const auto outputPath = MermaidExporter::
            exportGraph(*currentGraph);
        std::cout << "\n[OK] Mermaid code written to
            :\n"
                    << "      " << outputPath.string() <<
                        "\n";
    } catch (const std::exception& ex) {
        std::cout << "[ERROR] " << ex.what() << "\n";
    }
    break;
}

//
=====

case 103:  // Export Spanning Tree Mermaid
{
    if (!currentSpanningTree) {
        std::cout << "[!] Build the spanning tree
            first (option 19)\n";
        break;
    }

    try {
        const auto outputPath = MermaidExporter::
            exportGraph(
                *currentSpanningTree, "
                generated_spanning_tree.mmd");
        std::cout << "\n[OK] Spanning-tree Mermaid
            code written to:\n"
                    << "      " << outputPath.string() <<
                        "\n";
    }
}

```

```

        } catch (const std::exception& ex) {
            std::cout << "[ERROR] " << ex.what() << "\n";
        }
        break;
    }

    //
    =====

    case 0:  // Exit
    {
        std::cout << "Exiting program...\n";
        break;
    }

    //
    =====

    default:  // Invalid Option
    {
        std::cout << "[!] Invalid choice. Please try
            again.\n";
    }
}

} while (userChoice != 0);

return 0;
}

```